

Architektura urządzeń sieci komputerowych

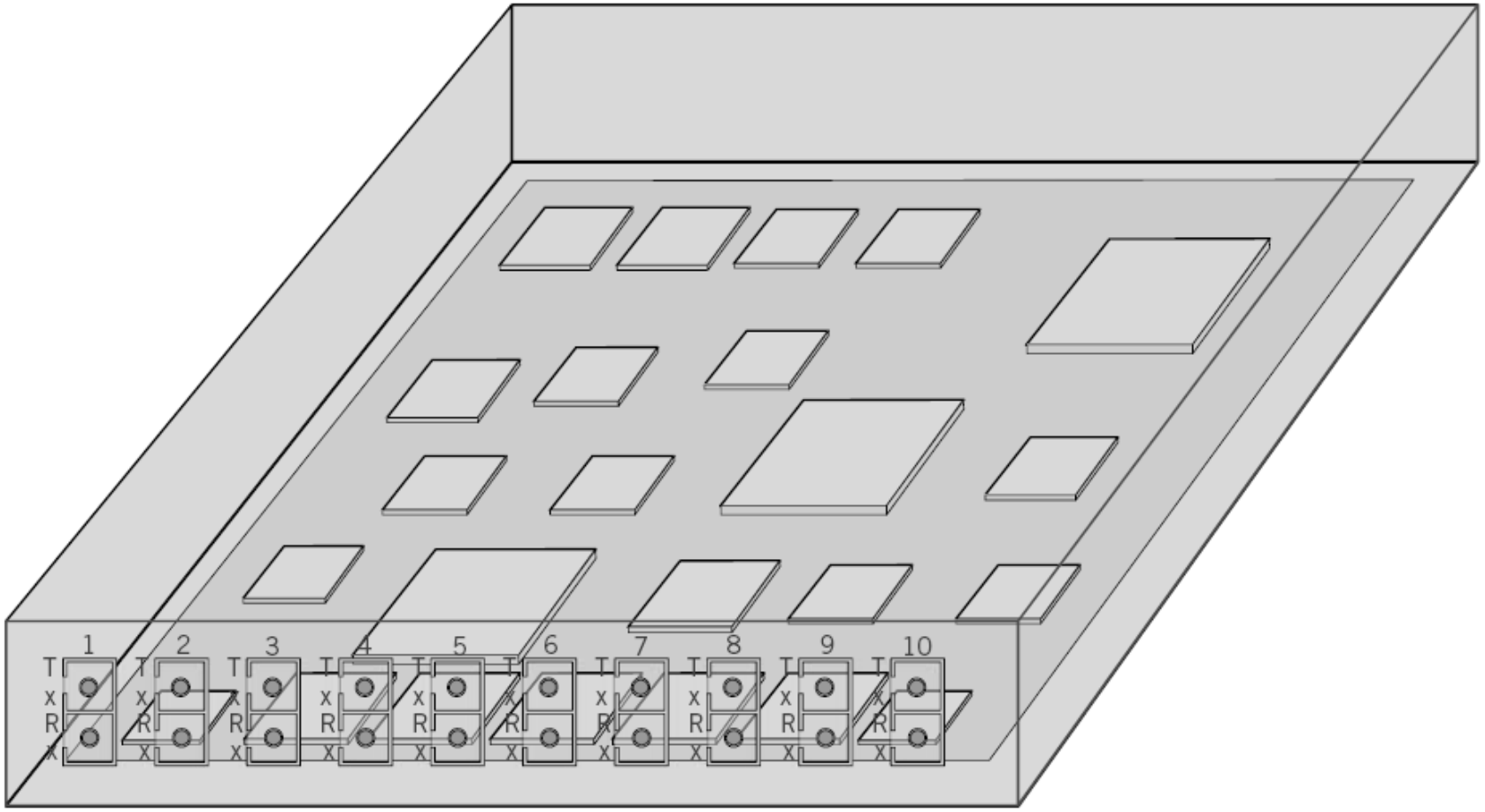
Procesory sieciowe

Zagadnienia

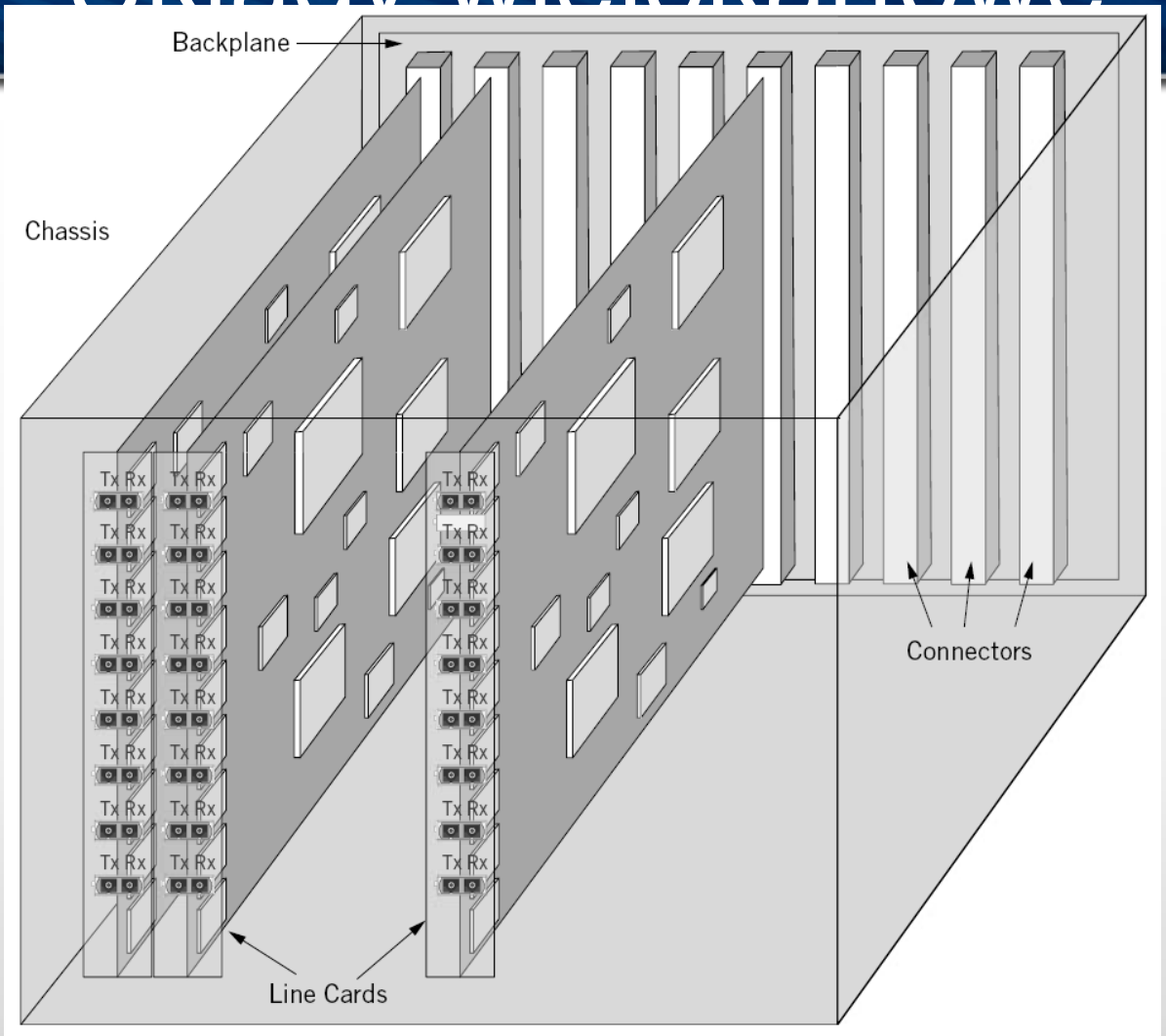
- Rodzaje urządzeń
- Wbudowane funkcje procesorów
- Obsługa strumieni
- Architektura procesorów sieciowych
- Procesory EZChip

RODZAJE URZĄDZEŃ

Urządzenia typu ,,pizza-box”



Układy wielokartowe





WBUDOWANE FUNKCJE PROCESORÓW

Przetwarzanie pakietów

- Ramkowanie
- Parsowanie i klasyfikacja
- Przeszukiwanie i przekazywanie
- Modyfikacja
- Kompresja i kodowanie
- Kolejkowanie i zarządzanie ruchem

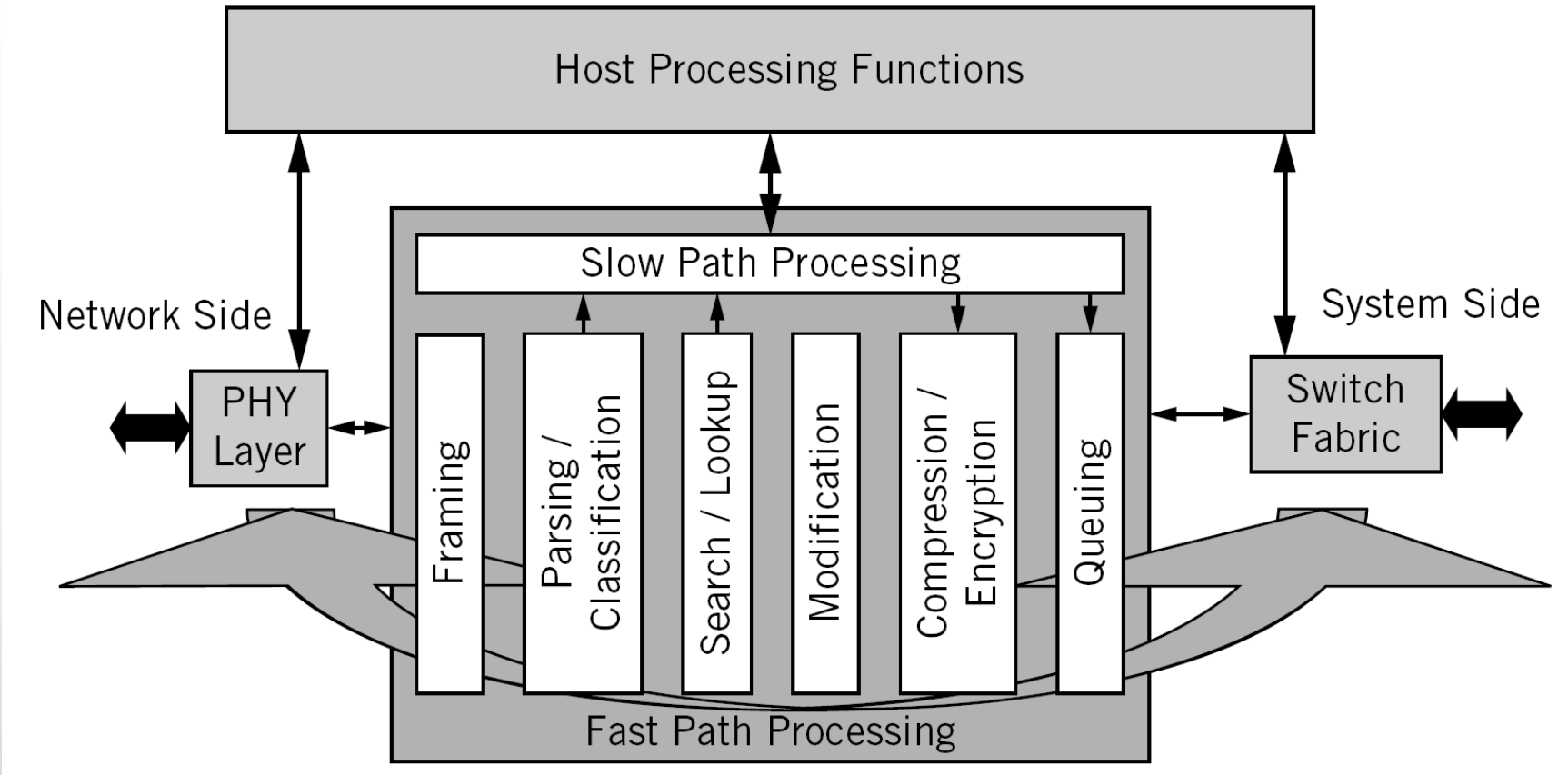
Rodzaje ścieżek

- Ścieżka danych
- Ścieżka sterująca

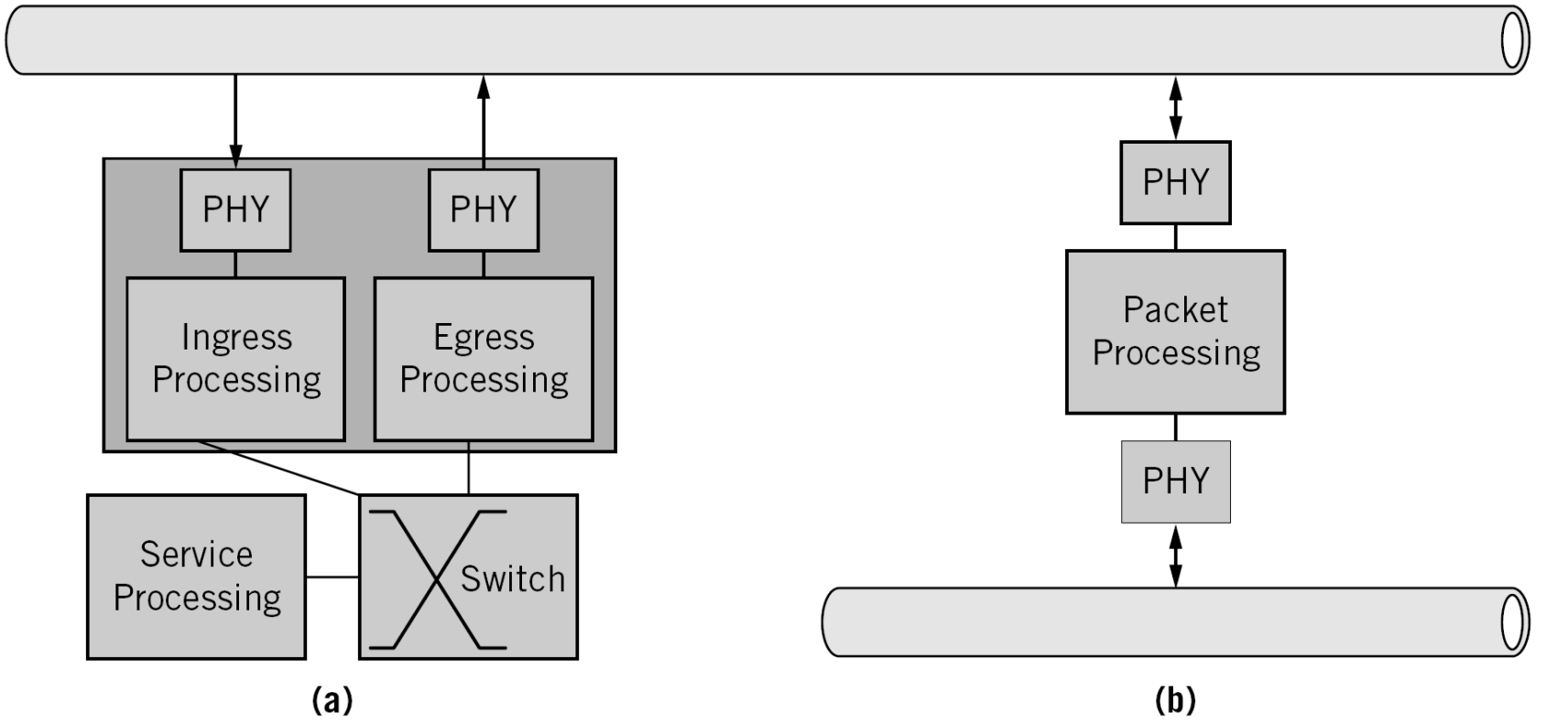
Kierunki przetwarzania

- Wejściowe (Ingress)
- Wyjściowe (Egress)
- Połączone (Ingress-Egress)

Ogólny schemat przetwarzania pakietów



Implementacje procesorów sieciowych



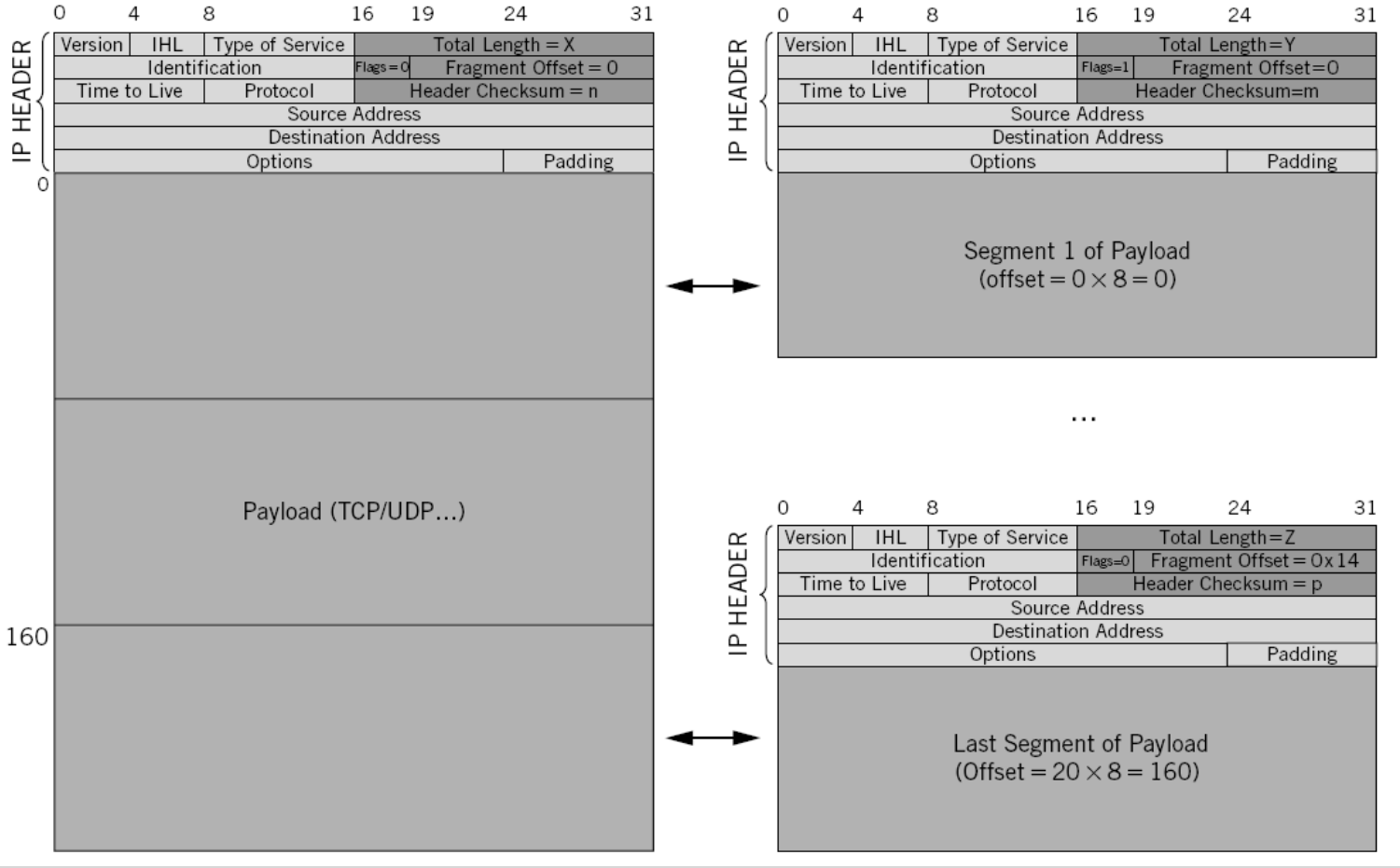
Przetwarzanie wejściowe

- Kontrola błędów
- Bezpieczeństwo i dekodowanie
- Klasyfikacja / demultipleksacja
- Zarządzanie ruchem
- Przeszukiwanie
- Manipulacje nagłówkiem
- Składanie pakietów
- Priorytezacja pakietów i kolejkowanie
- Przekazywanie pakietów.

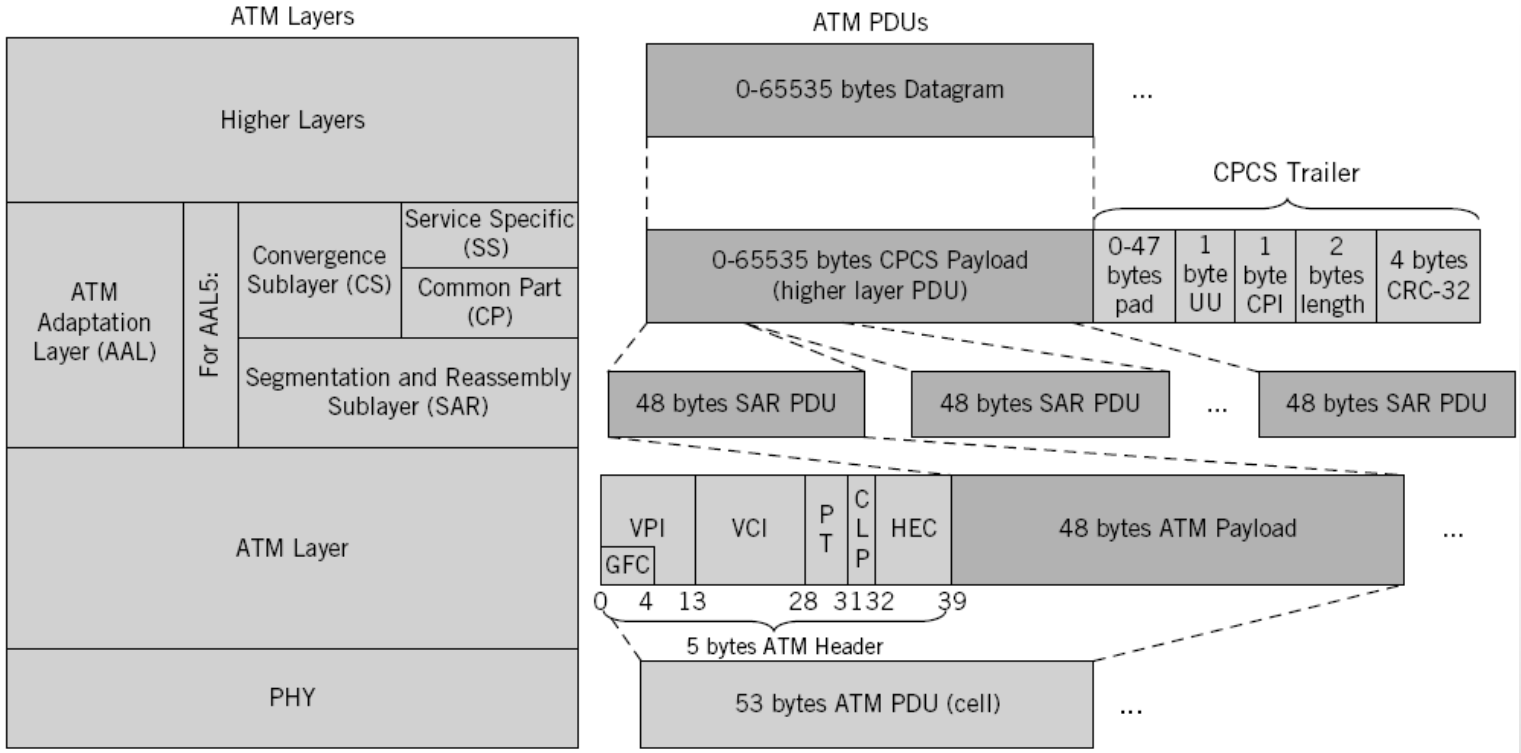
Przetwarzanie wyjściowe

- Wyliczanie i dodawania sumy kontrolnej
- Poszukiwanie adresu (łącza wyjściowego)
- Przekazywanie pakietu
- Segmentacja i fragmentacja pakietów
- Zarządzanie ruchem
- Priorytezacja i kolejkowanie pakietów
- Bezpieczeństwo (kodowanie)

IP SAR


0 4 8 16 19 24 31
IP HEADER
Version
IHL
Type of Service
Total Length = X
Identification
Flags = 0
Fragment Offset = 0
Time to Live
Protocol
Header Checksum = n
Source Address
Destination Address
Options
Padding
0
Payload (TCP/UDP...)
160
Total Length=Y
Flags=1
Fragment Offset=0
Header Checksum=m
Segment 1 of Payload
(offset = 0 × 8 = 0)
...
Total Length=Z
Flags=0
Fragment Offset = 0x14
Header Checksum = p
Last Segment of Payload
(Offset = 20 × 8 = 160)

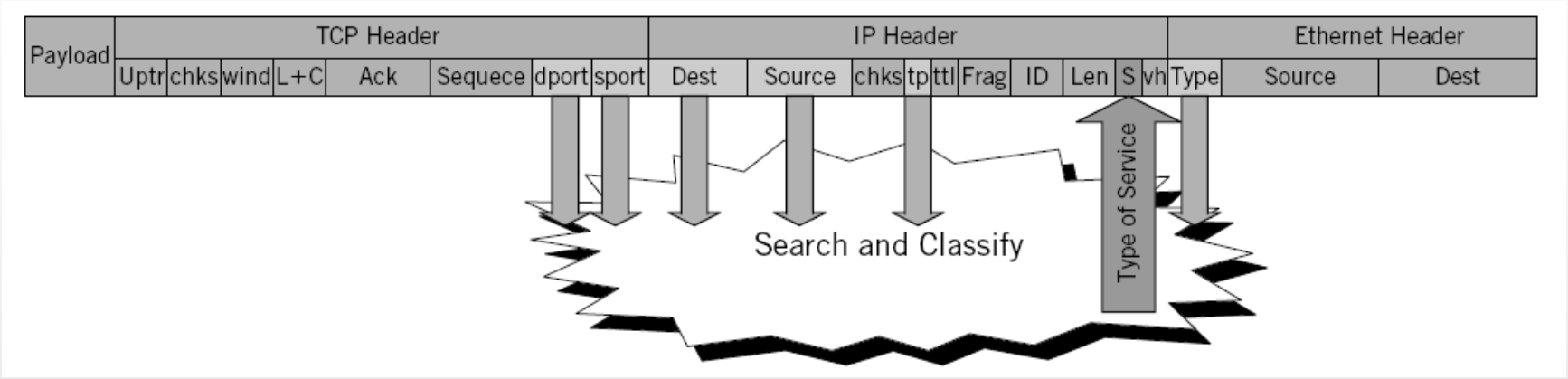
ATM SAR



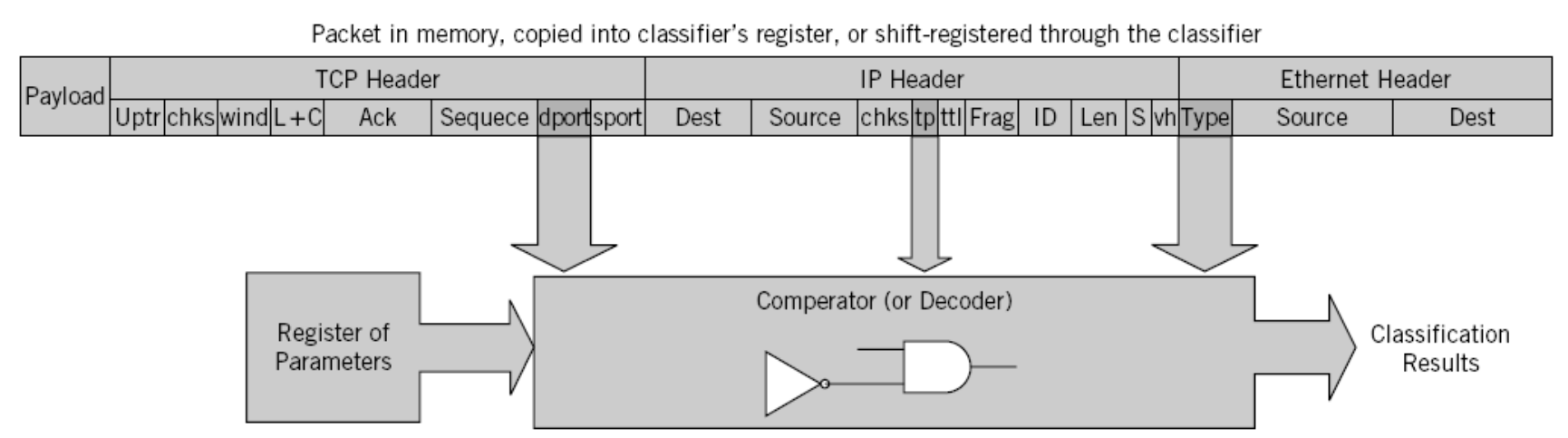
PDU—Protocol Data Unit
CPCS—Common Part Convergence Sublayer
UU—User-to-user Indication
CPI—Common Part Indicator
CLP—Cell Loss Priority

GFC—Generic flow control, only in User-Network-Interface (UNI)
VPI—Virtual Path Identifier
VCI—Virtual Channel Identifier
PT—Payload Type
HEC—Header Error Control

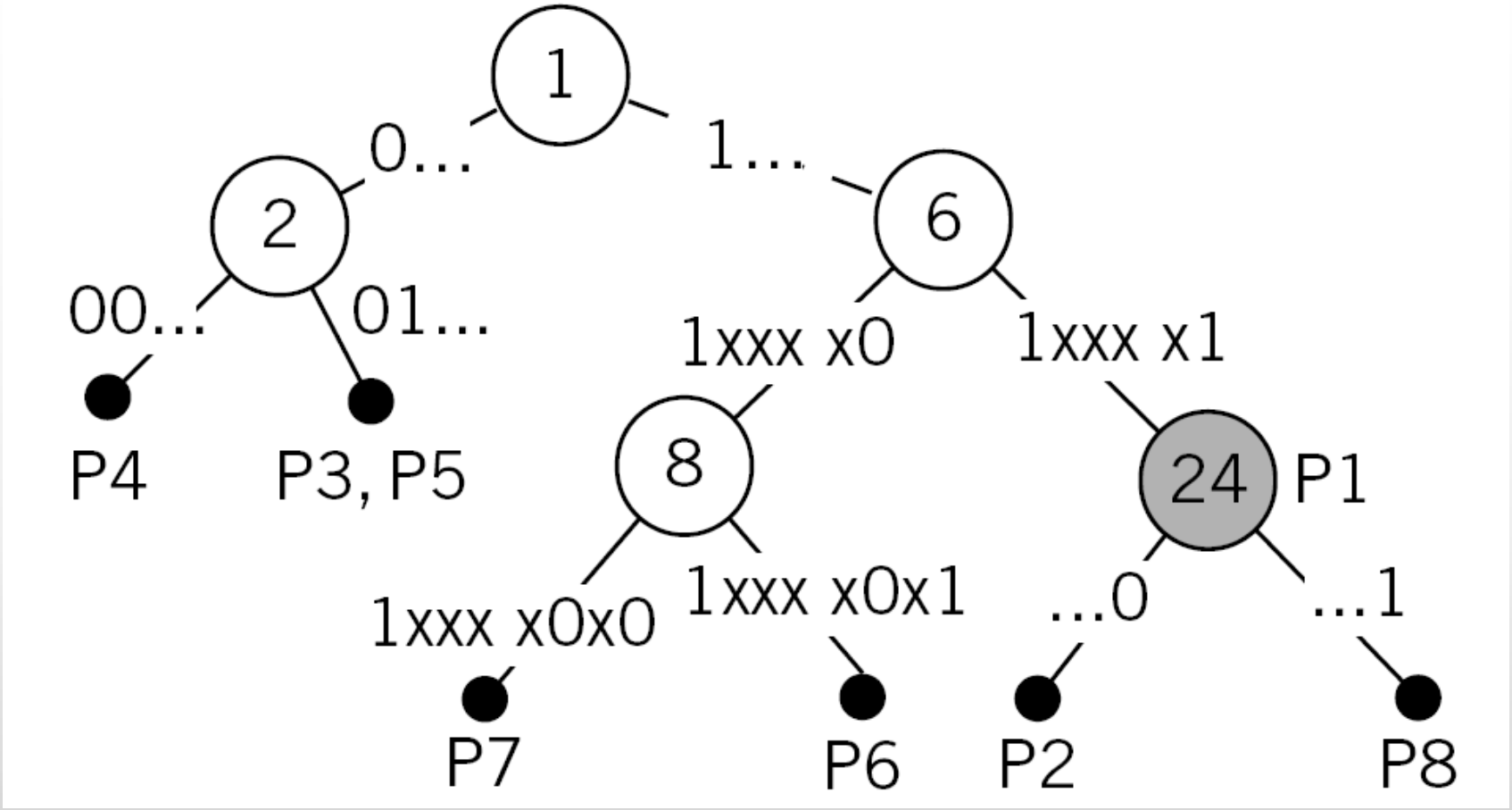
Klasyfikacja pakietów IP



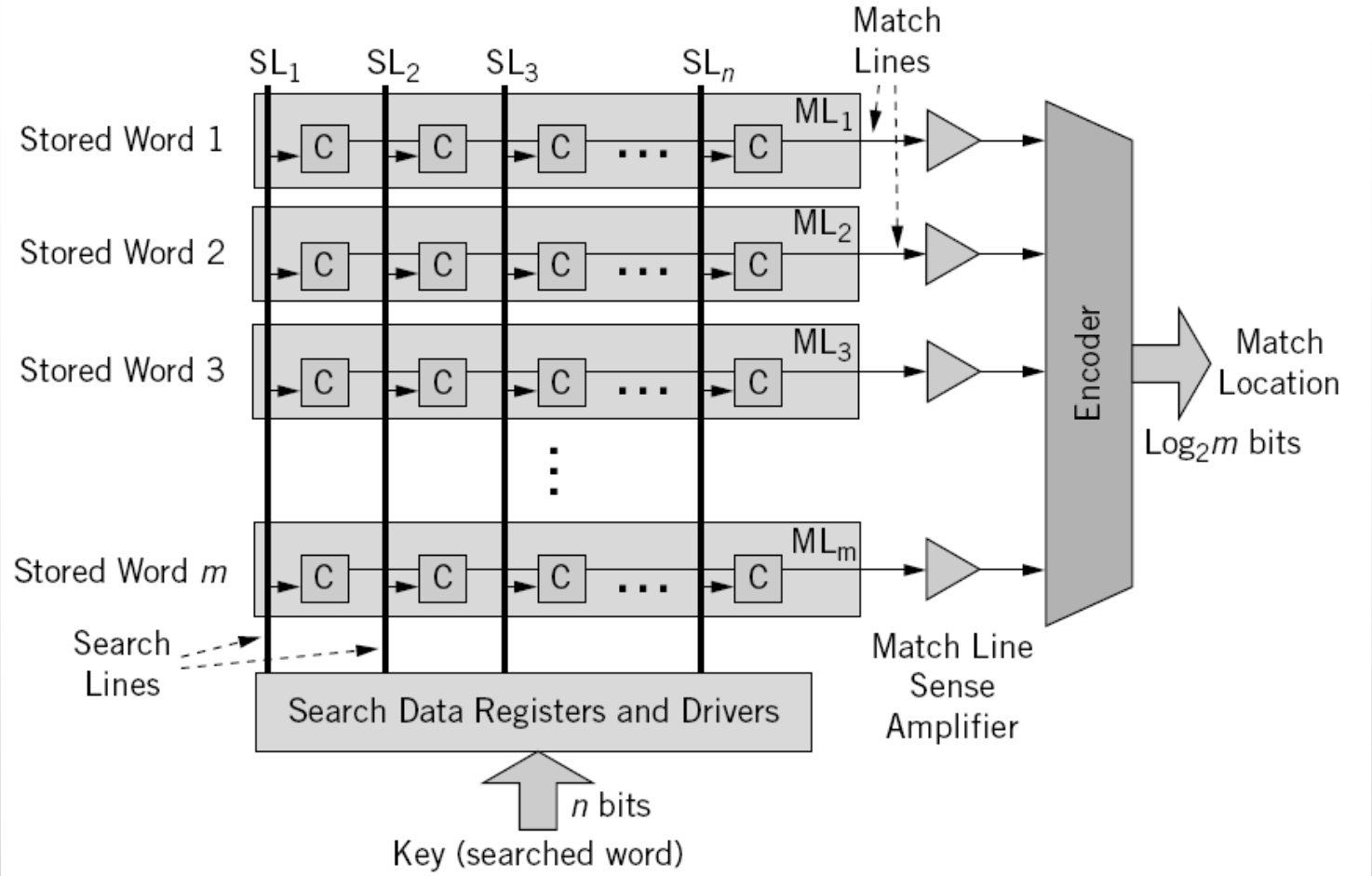
Klasyfikacja sprzętowa



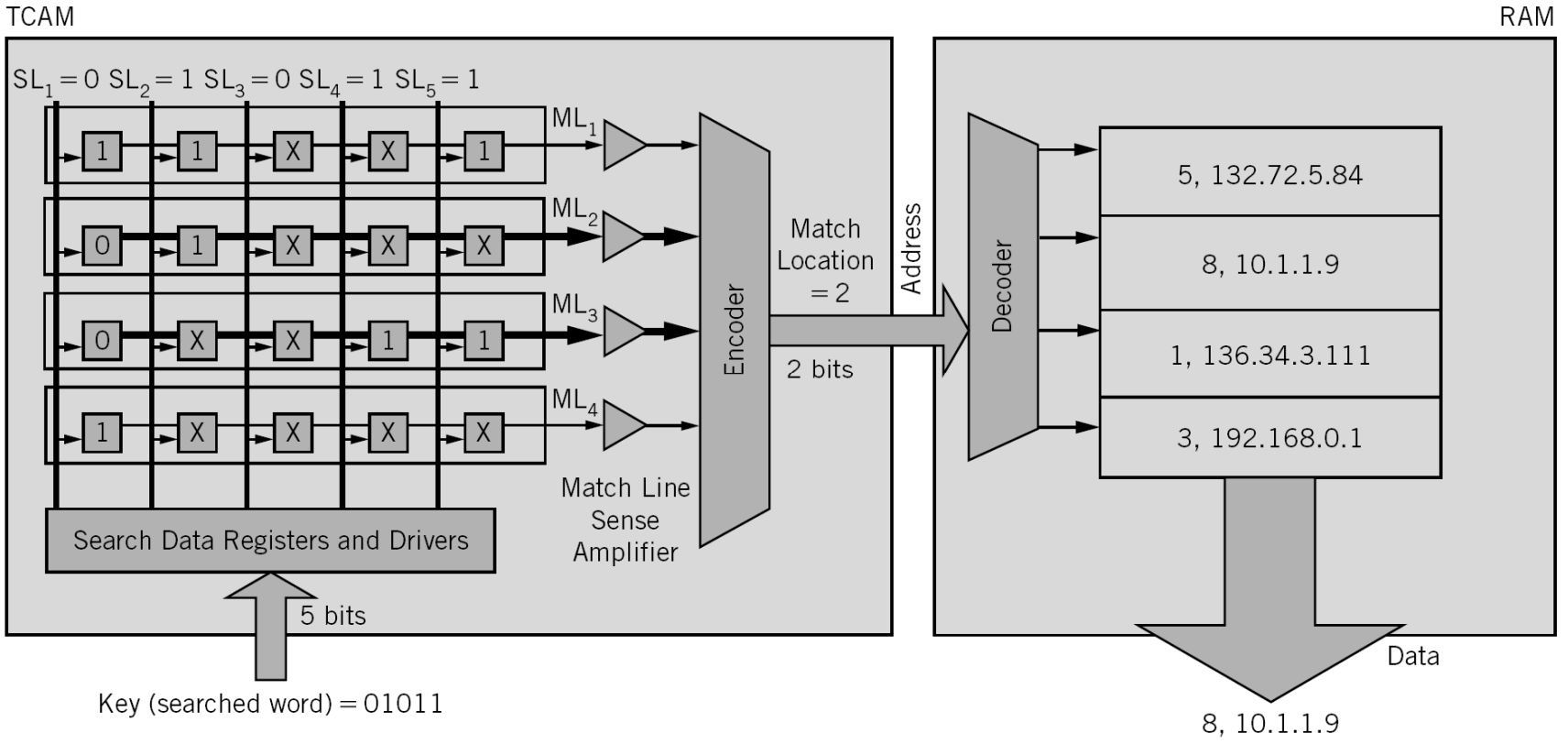
Przeszukiwanie



Pamięci CAM


Match Lines
SL1
SL2
SL3
SLn
ML1
ML2
ML3
MLm
Stored Word 1
Stored Word 2
Stored Word 3
Stored Word m
C
Encoder
Match Location
Log2m bits
Search Lines
Search Data Registers and Drivers
Match Line Sense Amplifier
n bits
Key (searched word)

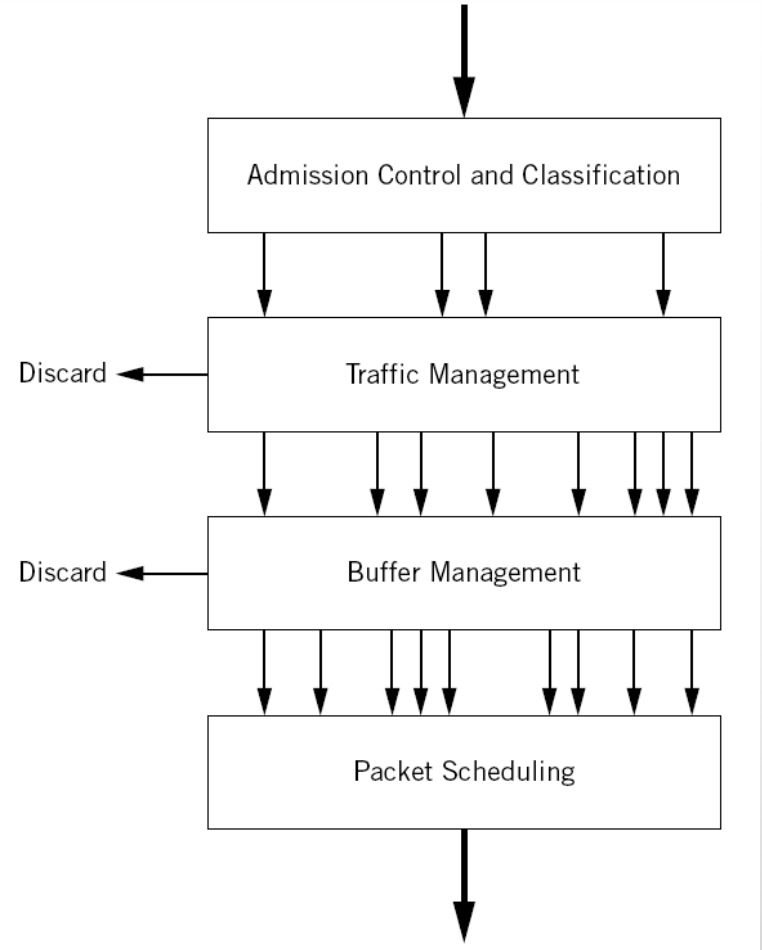
Pamięci TCAM



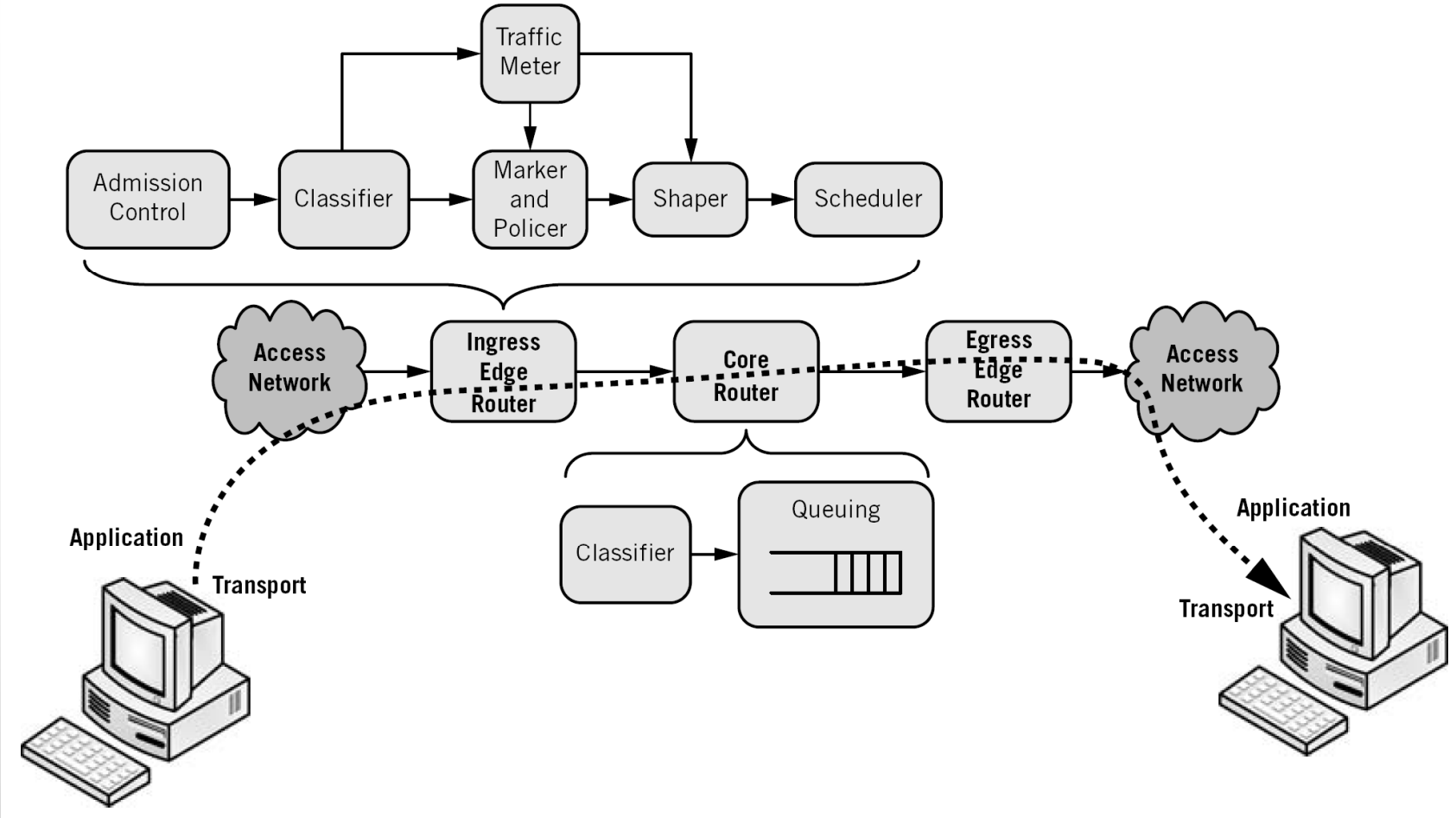


OBSŁUGA STRUMIENI

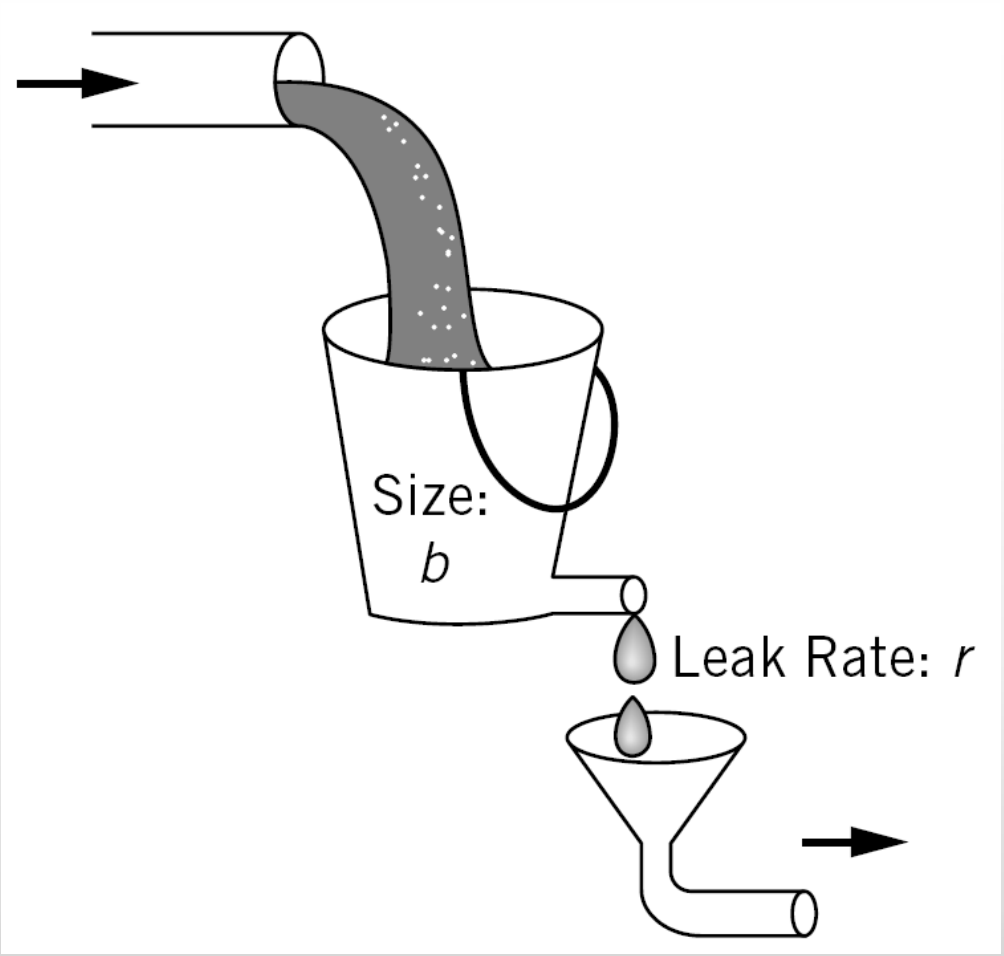
Funkcje QoS


Admission Control and Classification
Discard
Traffic Management
Discard
Buffer Management
Packet Scheduling

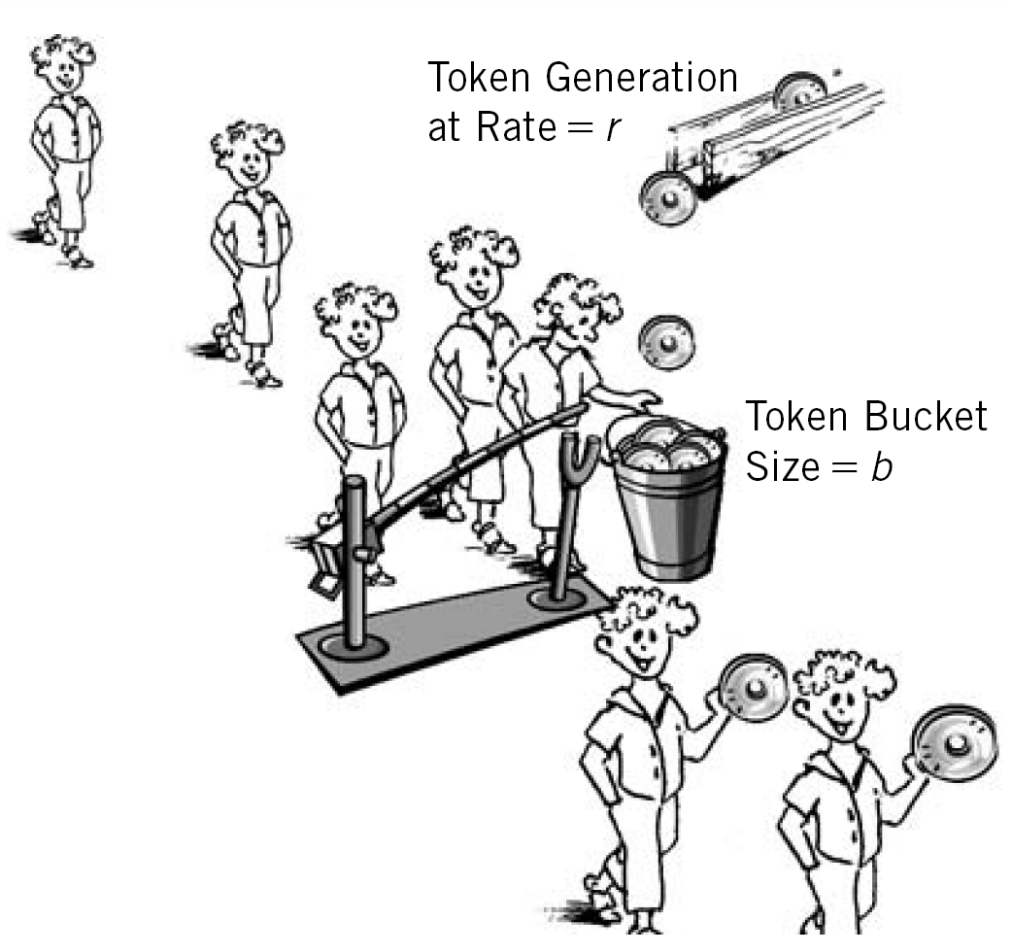
Mechanizmy QoS End-to-End



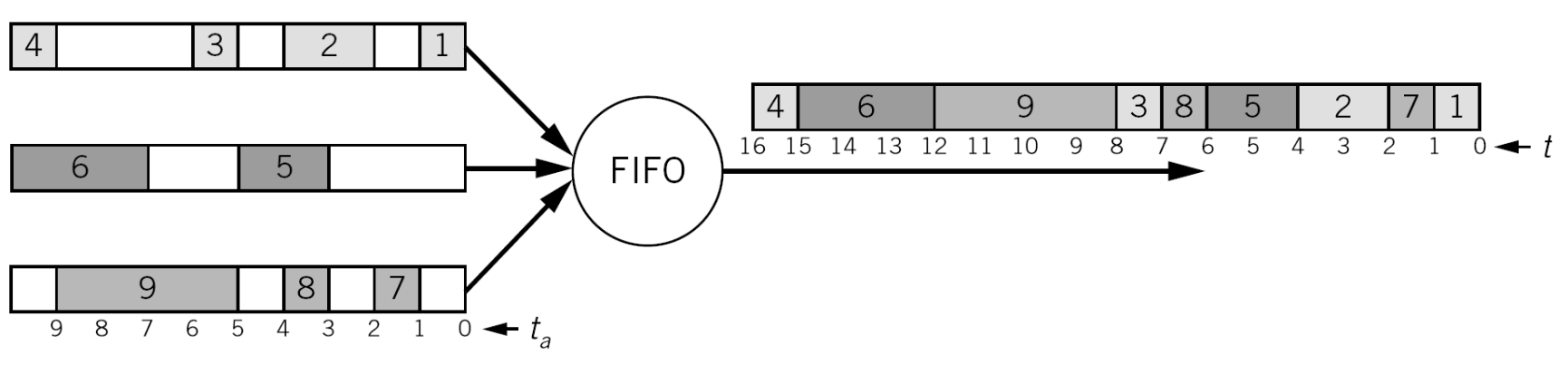
,,Dziurawe wiadro’’



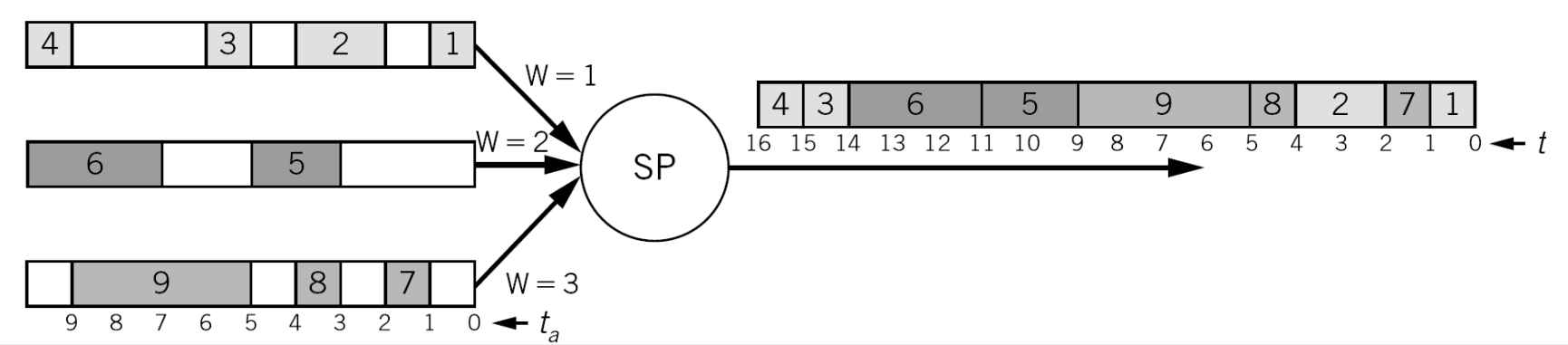
„Wiadro z żetonami”



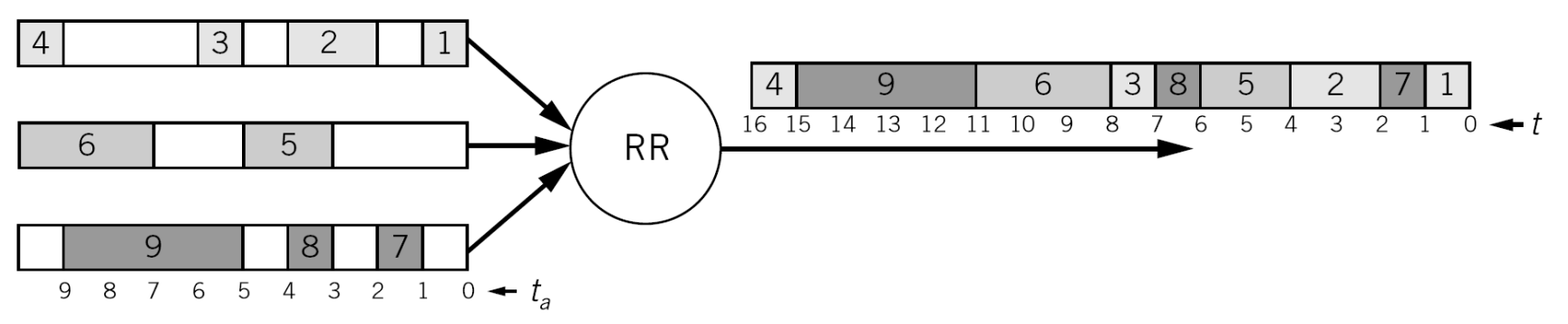
FIFO


4
3
2
1
6
5
9
8
7
9 8 7 6 5 4 3 2 1 0
t_a
FIFO
4
6
9
3
8
5
2
7
1
16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
t

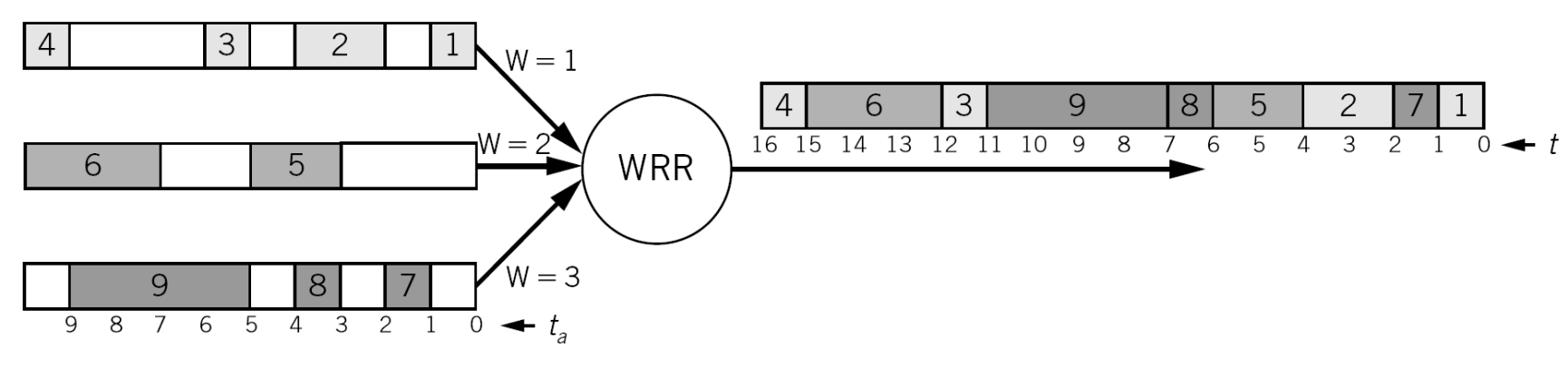
Priorytezacja



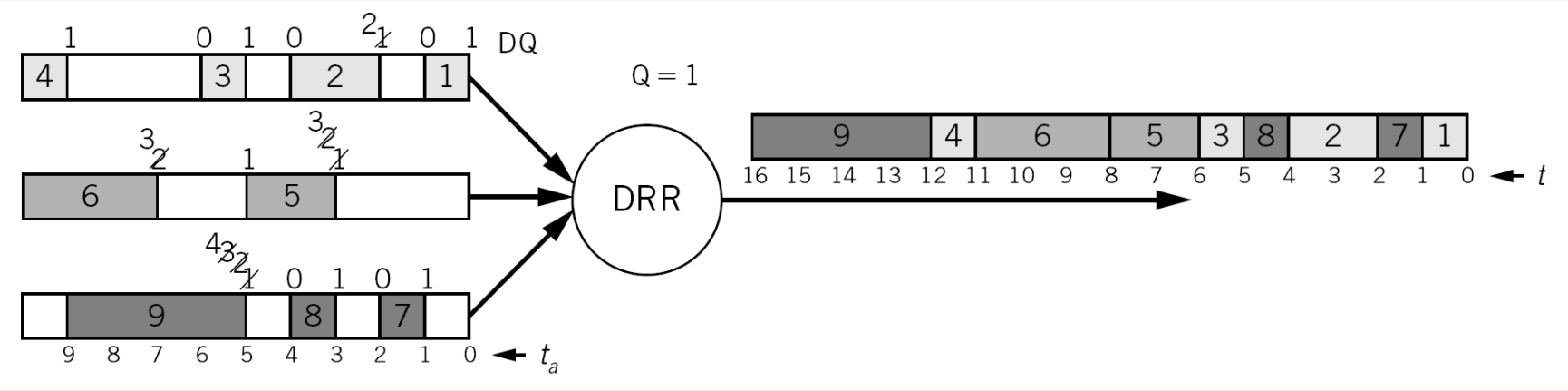
Round-robin


4
3
2
1
6
5
9
8
7
9 8 7 6 5 4 3 2 1 0
t_a
RR
4
9
6
3
8
5
2
7
1
16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
t

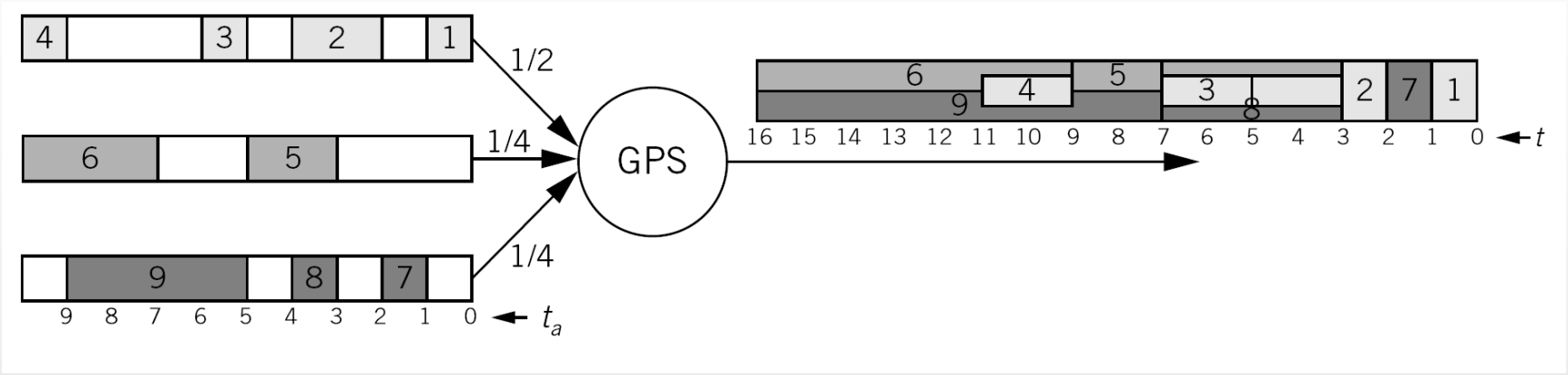
WRR


4
3
2
1
W = 1
6
5
W = 2
9
8
7
W = 3
9 8 7 6 5 4 3 2 1 0
t_a
WRR
4
6
3
9
8
5
2
7
1
16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
t

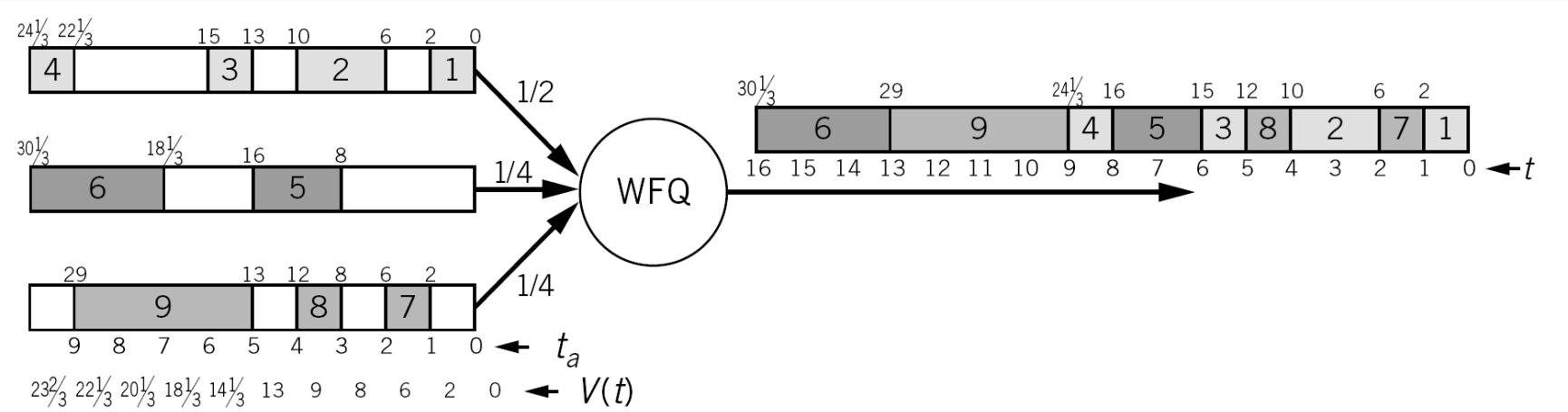
DRR



GPS


4
3
2
1
1/2
6
5
1/4
9
8
7
1/4
9 8 7 6 5 4 3 2 1 0
t_a
GPS
6
9
4
5
3
8
2
7
1
16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
t

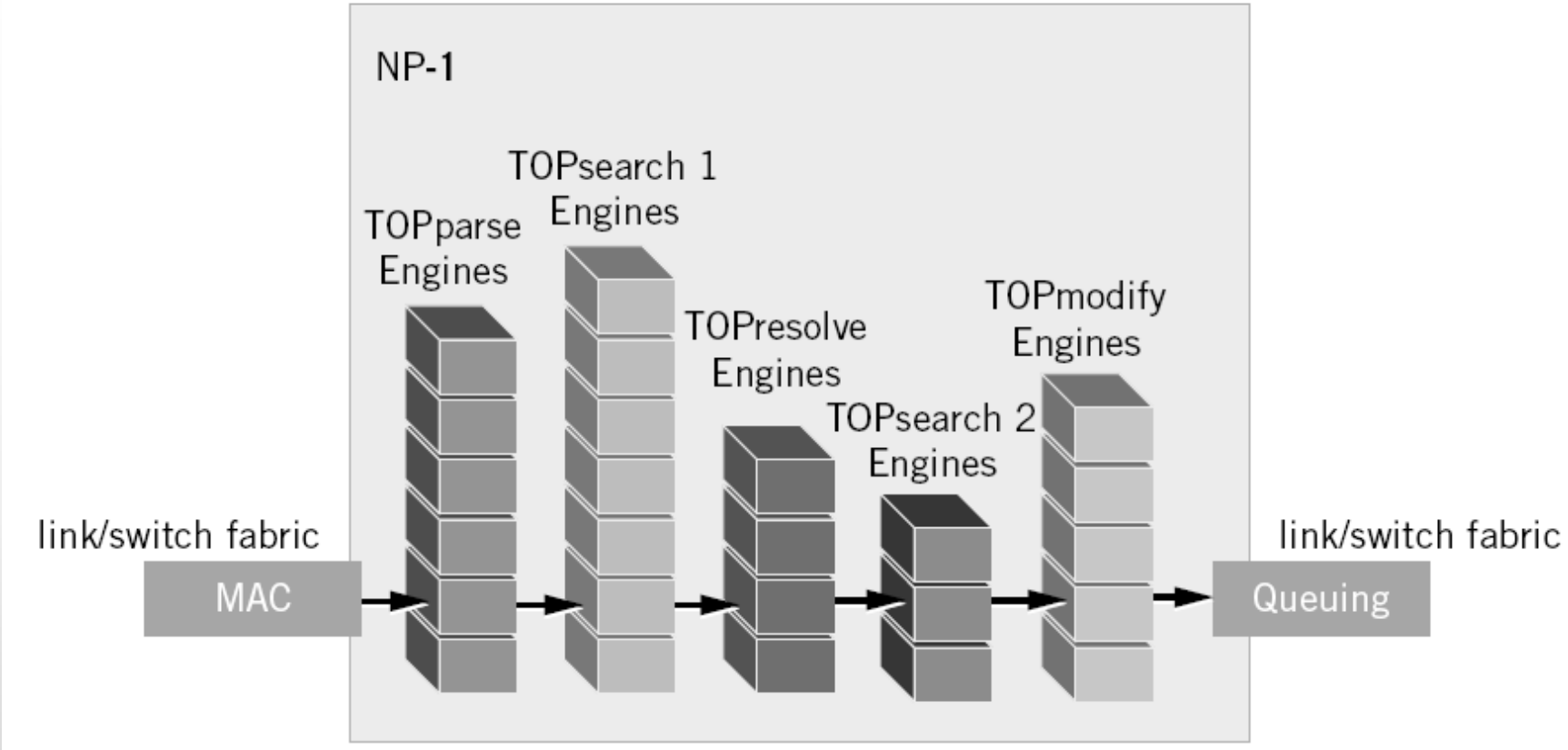
WFQ



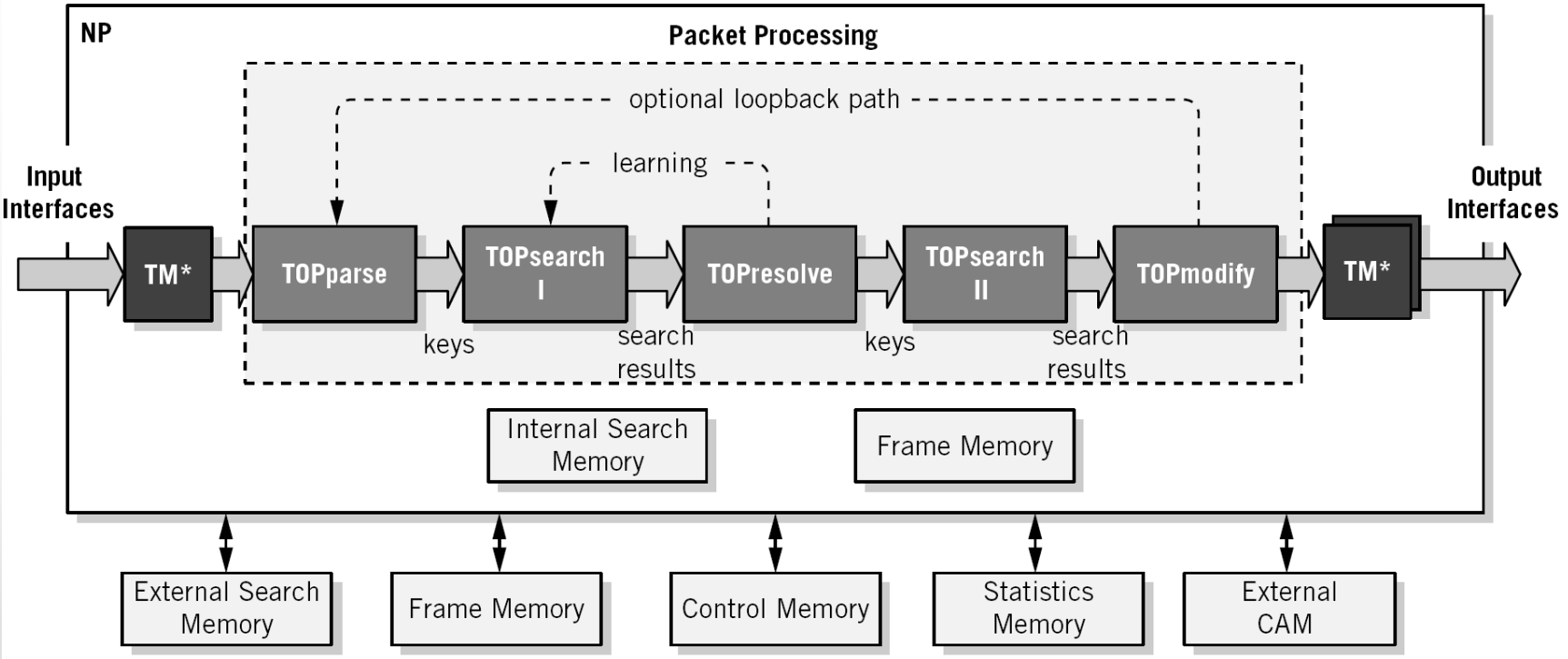


ARCHITEKTURA PROCESORÓW SIECIOWYCH

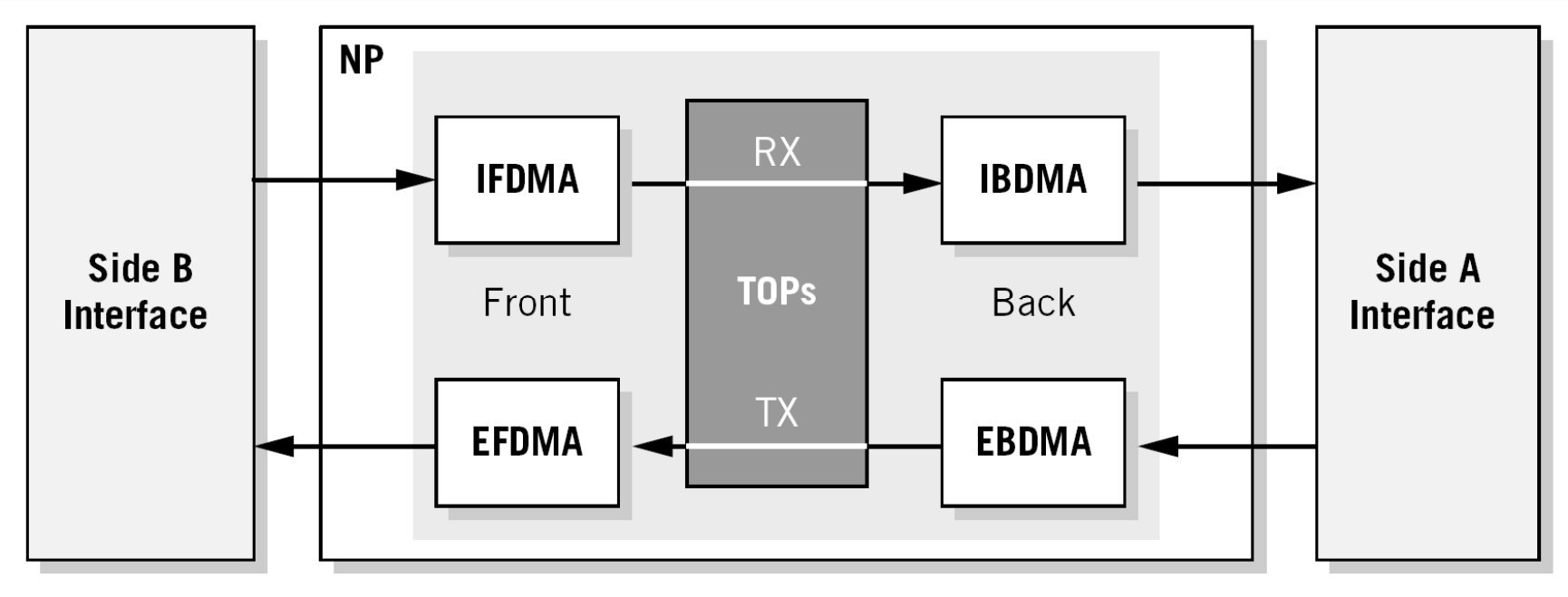
Architektura procesorów sieciowych



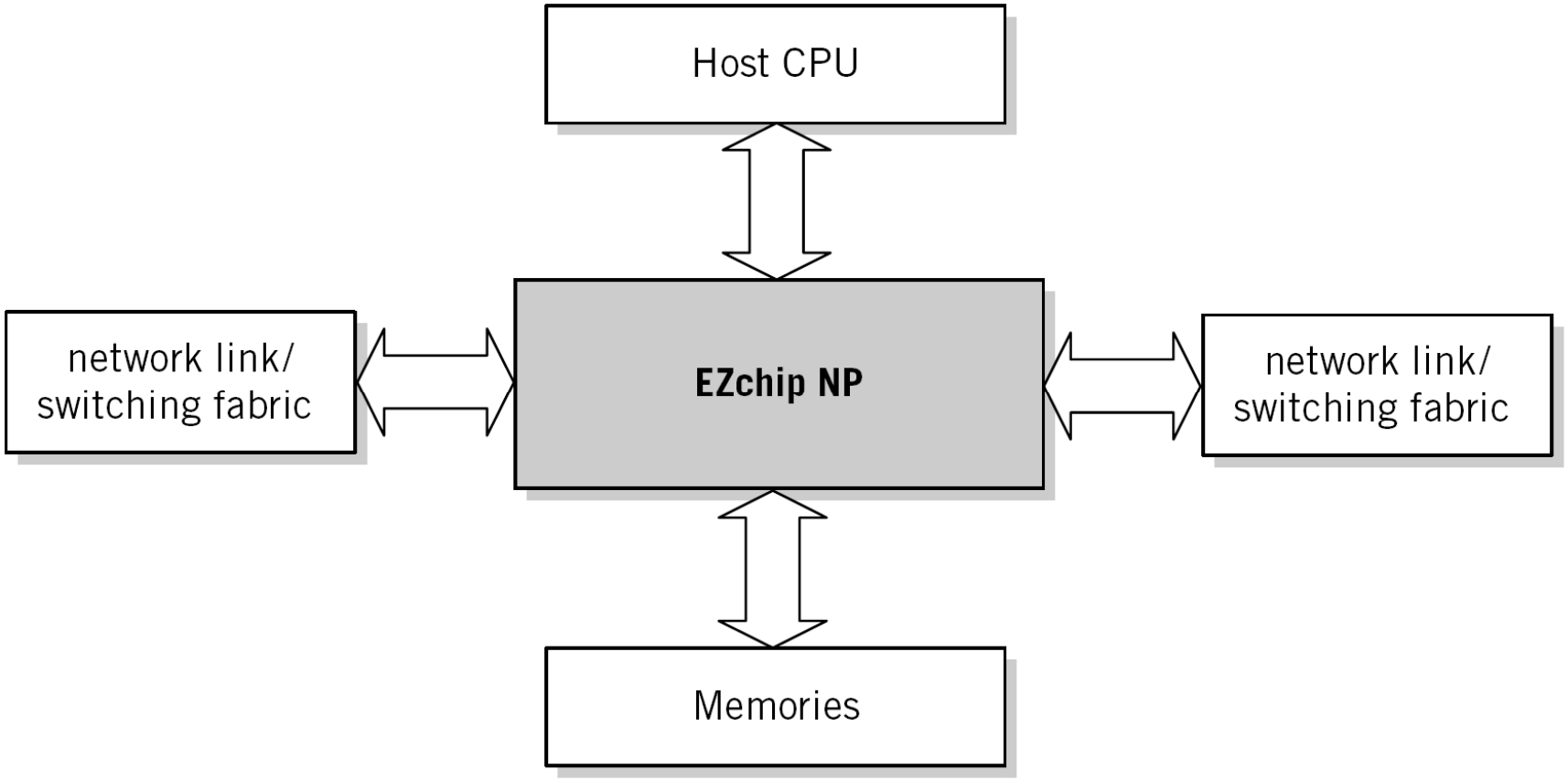
Architektura procesorów sieciowych



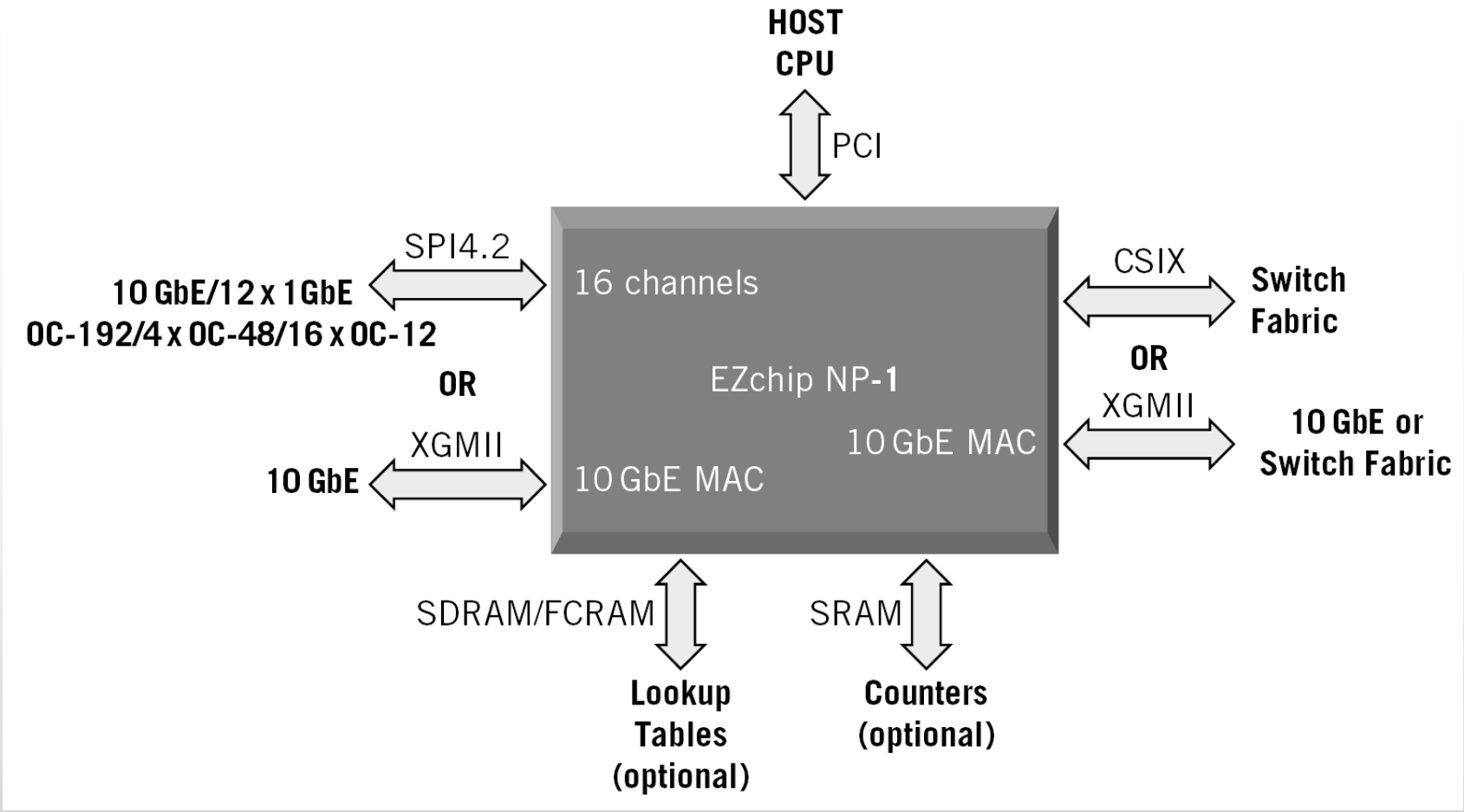
Kierunki przetwarzania



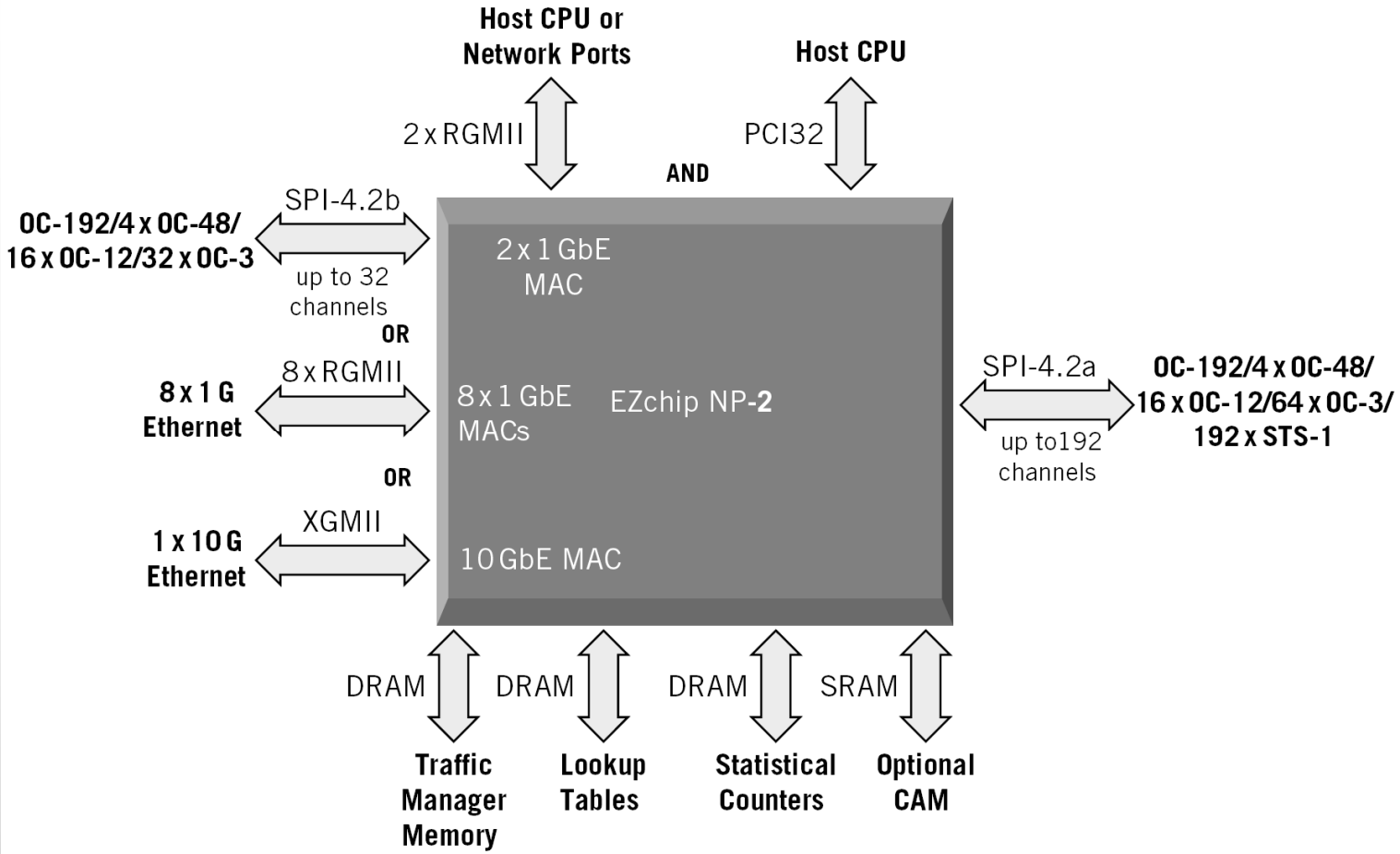
Typy interfejsów



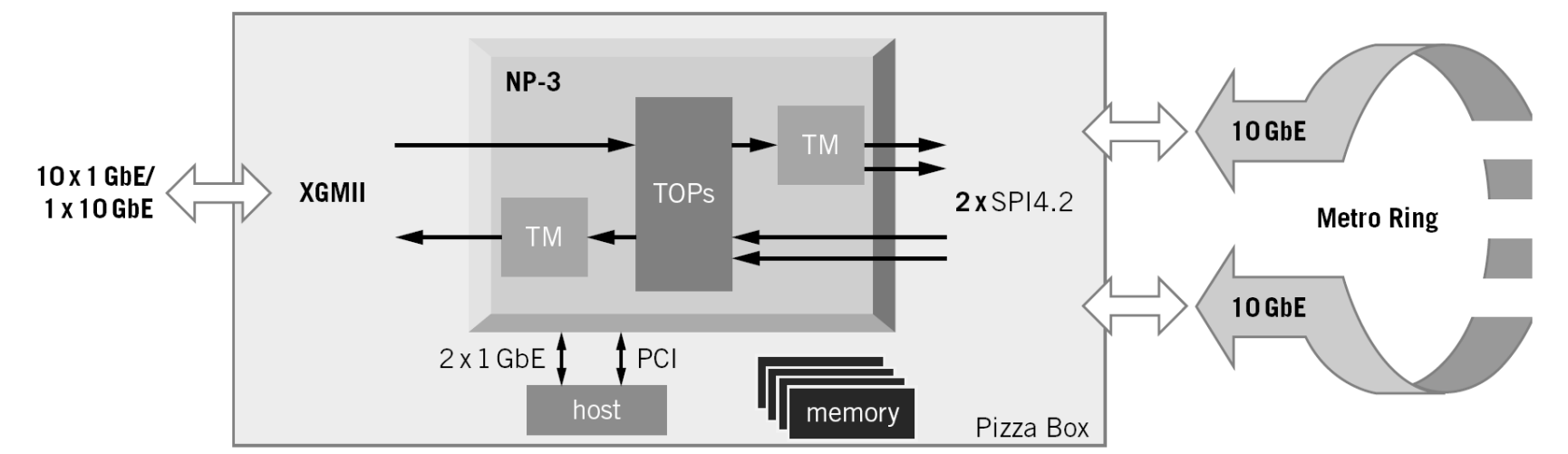
Procesor NP1



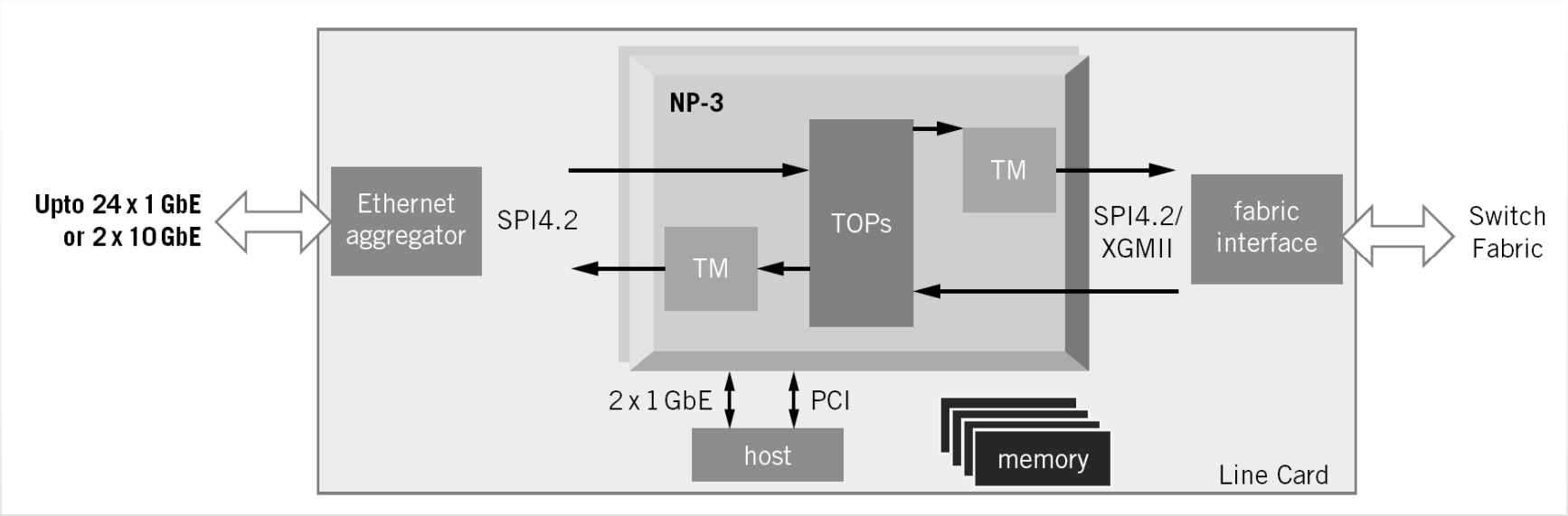
Procesor NP 2



Implementacja pojedyncza



Implementacja karty liniowej



Karta usługowa

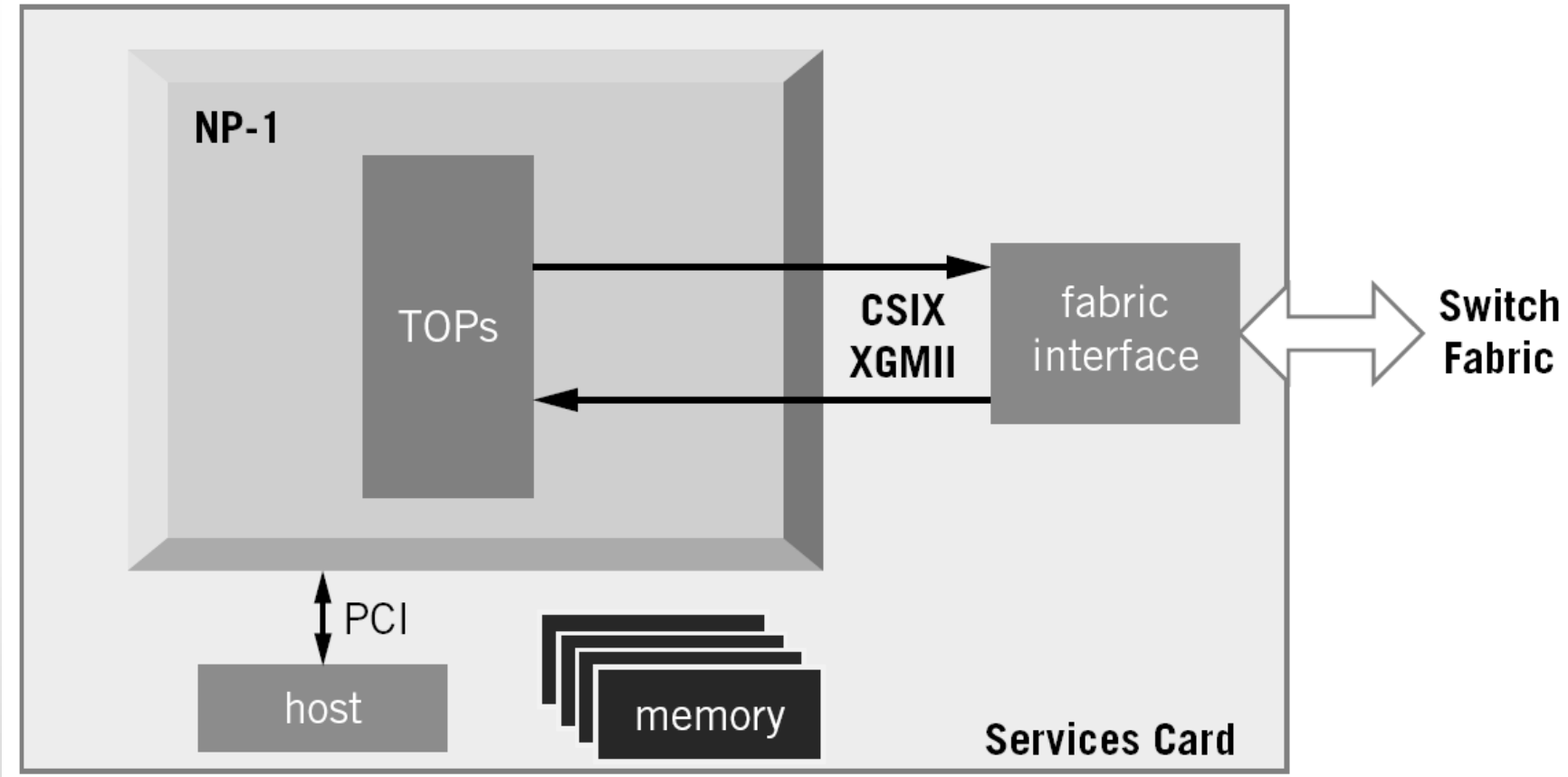
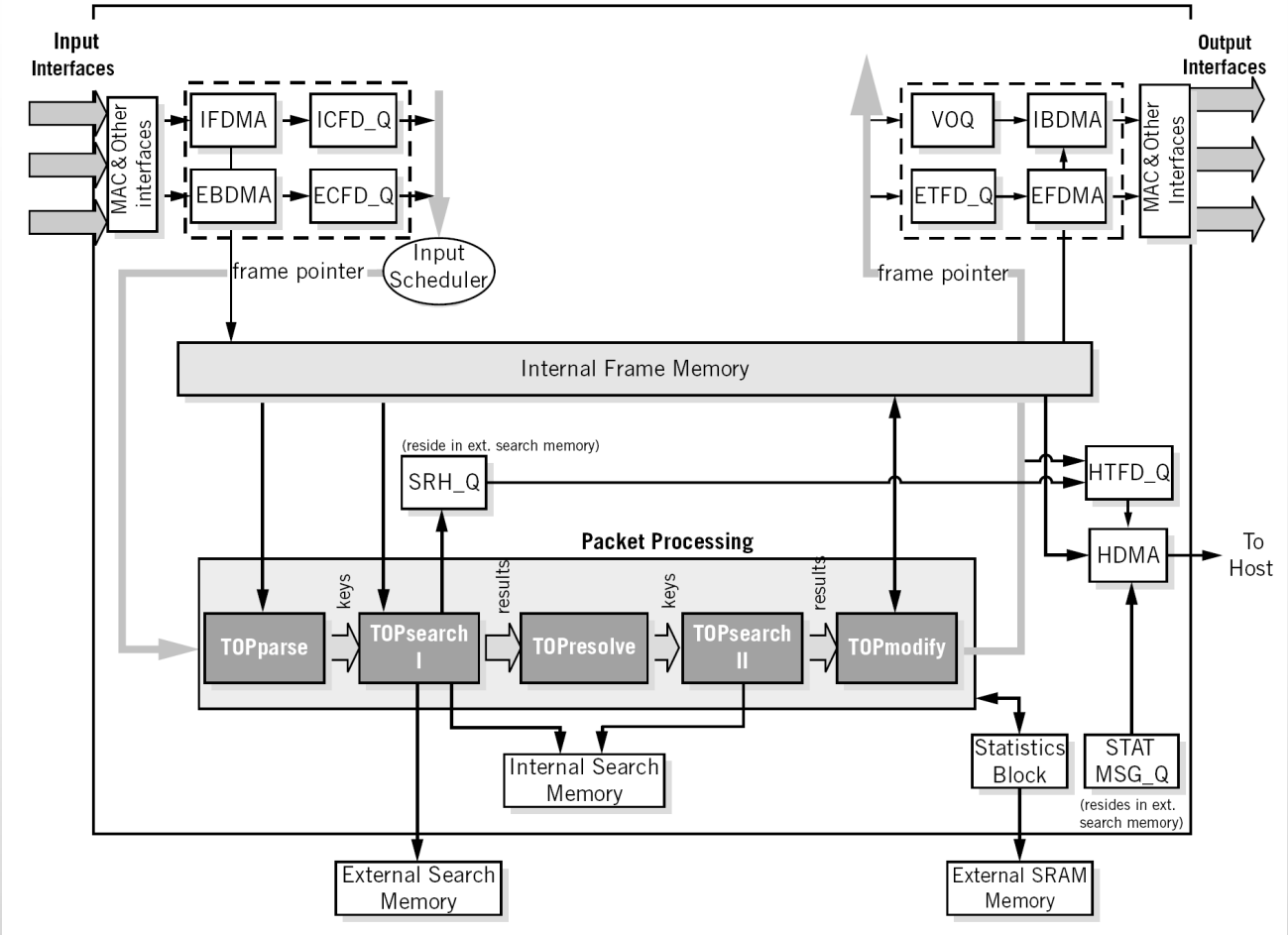


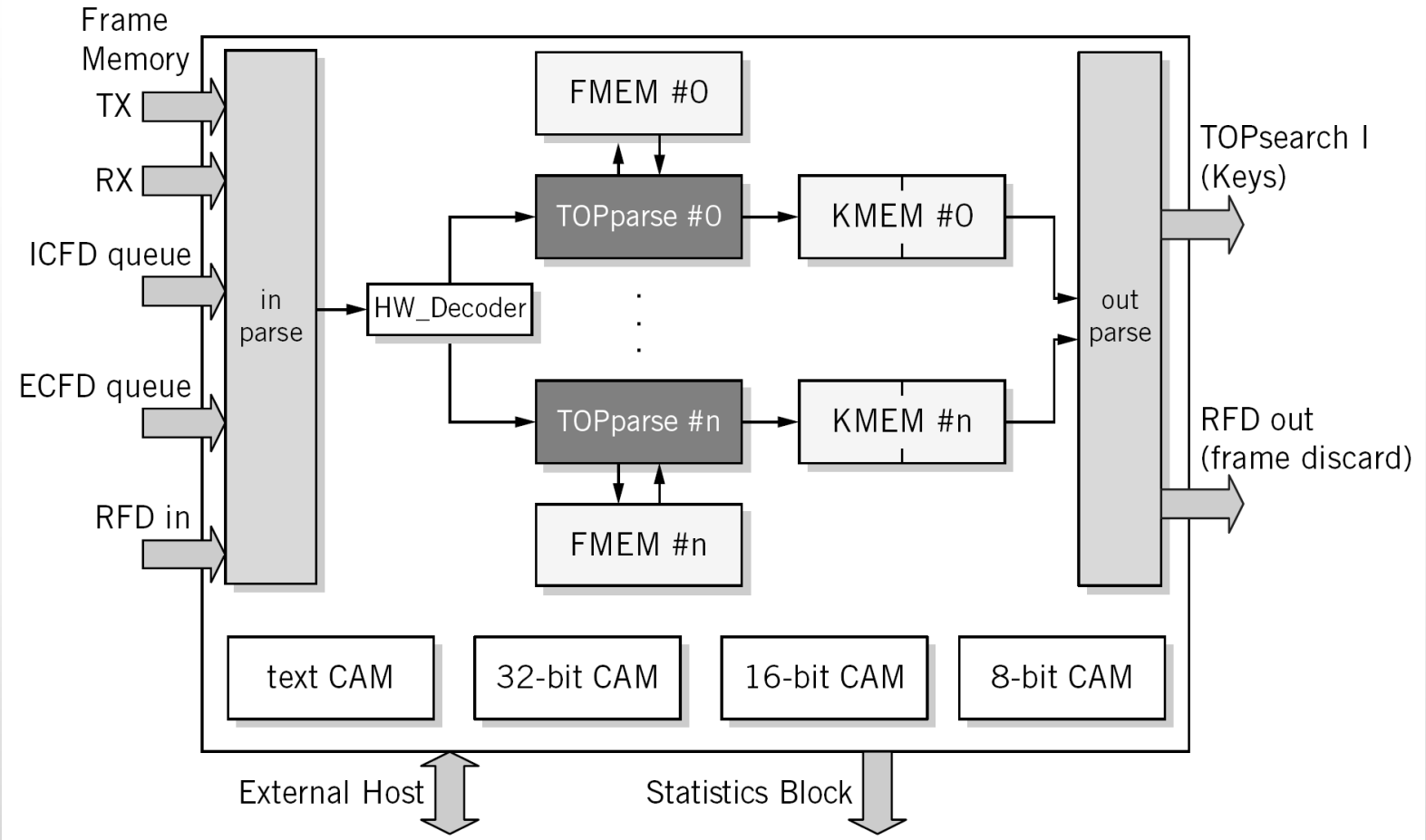
Diagram przepływów



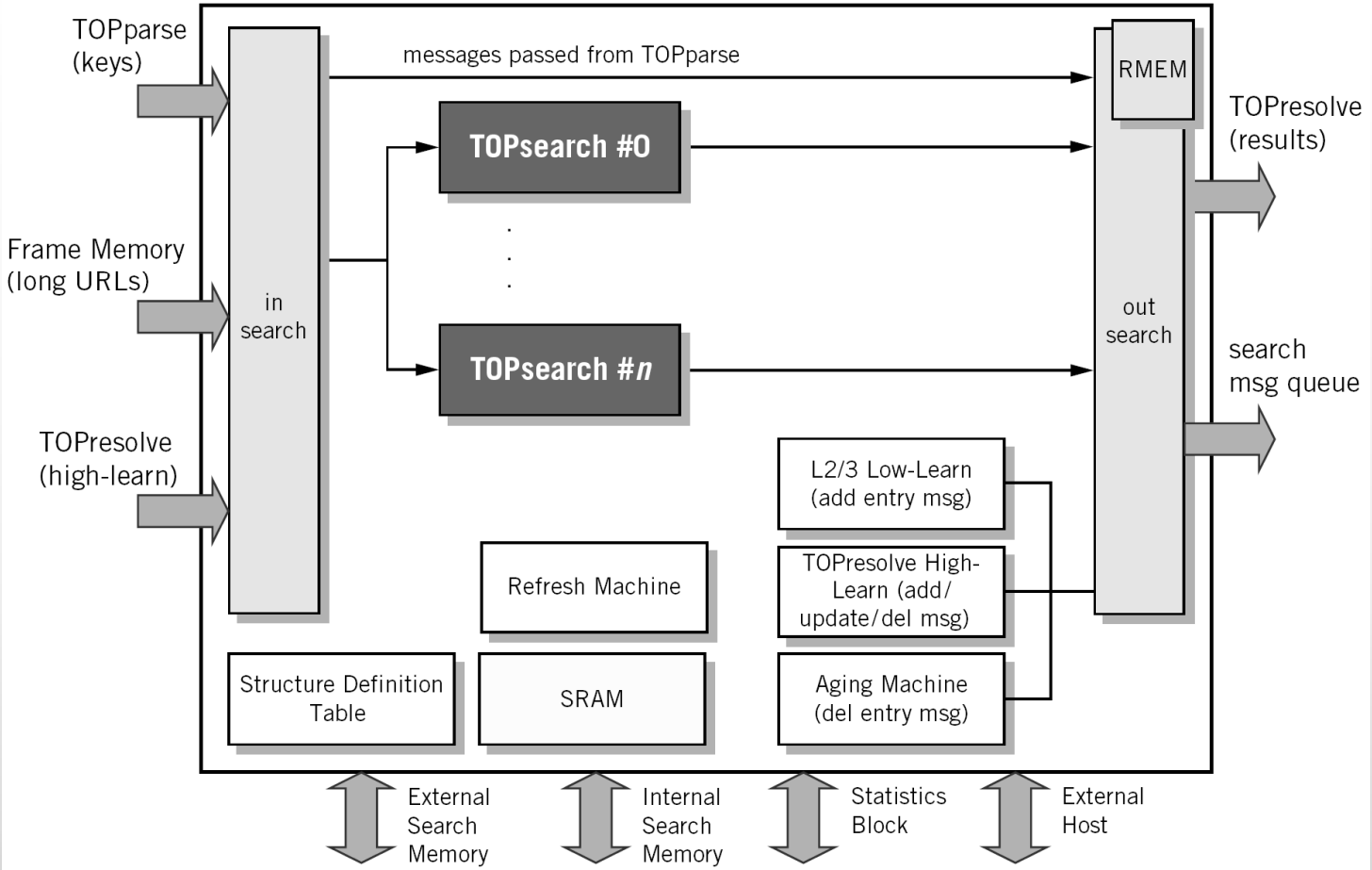


PROCESORY EZCHIP

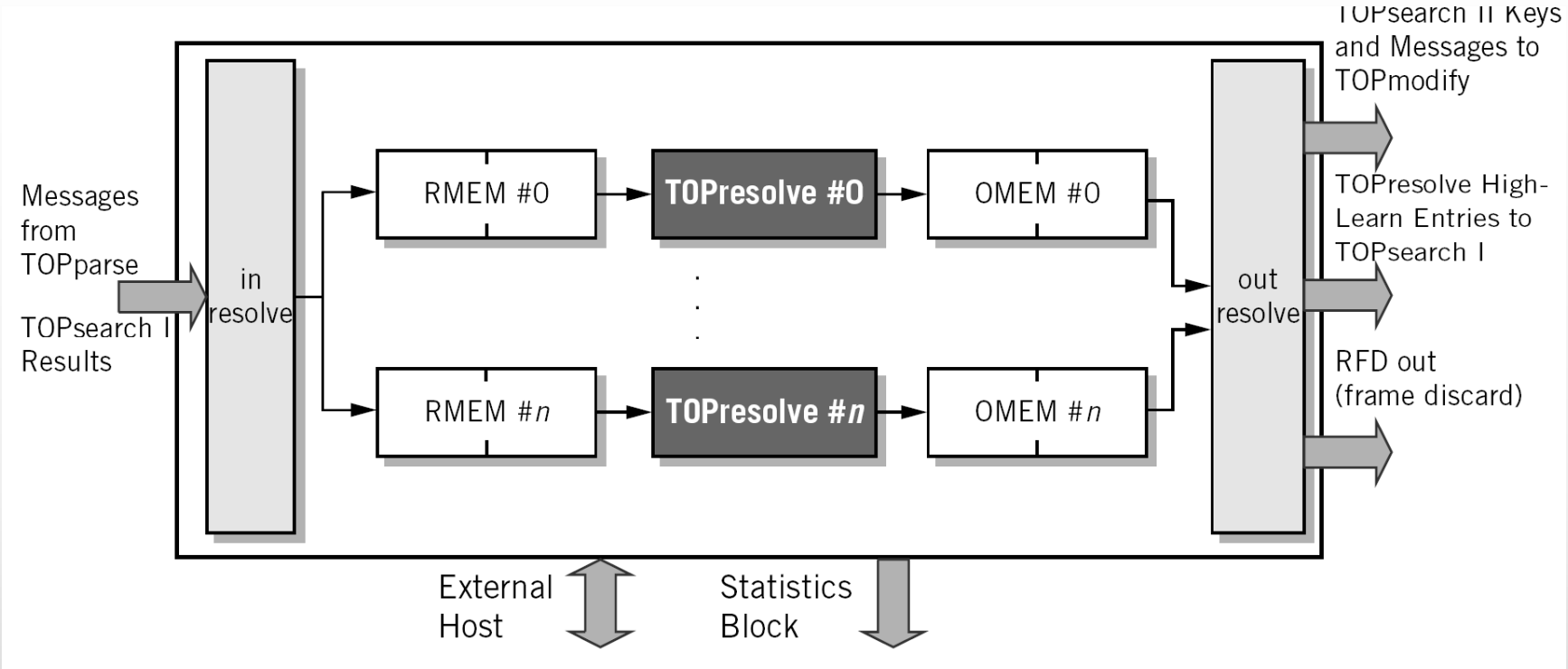
TOPparse


Frame
Memory
TX
RX
ICFD queue
ECFD queue
RFD in
in
parse
HW_Decoder
FMEM #0
TOPparse #0
KMEM #0
TOPparse #n
KMEM #n
FMEM #n
out
parse
TOPsearch I
(Keys)
RFD out
(frame discard)
text CAM
32-bit CAM
16-bit CAM
8-bit CAM
External Host
Statistics Block

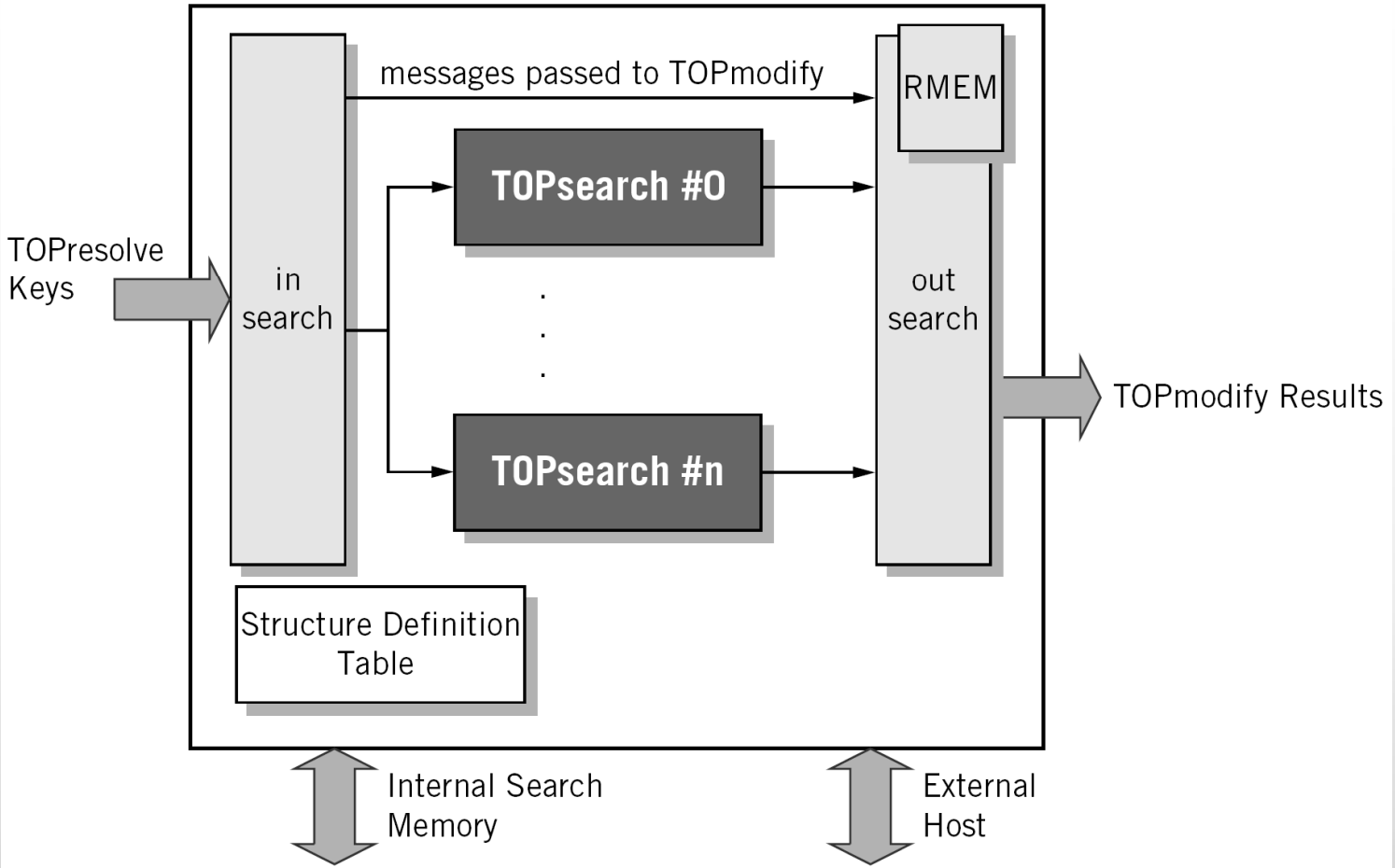
TOPSearch I


TOPparse
(keys)
Frame Memory
(long URLs)
TOPresolve
(high-learn)
in
search
messages passed from TOPparse
TOPsearch #0
TOPsearch #n
RMEM
out
search
TOPresolve
(results)
search
msg queue
L2/3 Low-Learn
(add entry msg)
TOPresolve High-
Learn (add/
update/del msg)
Aging Machine
(del entry msg)
Refresh Machine
SRAM
Structure Definition
Table
External
Search
Memory
Internal
Search
Memory
Statistics
Block
External
Host

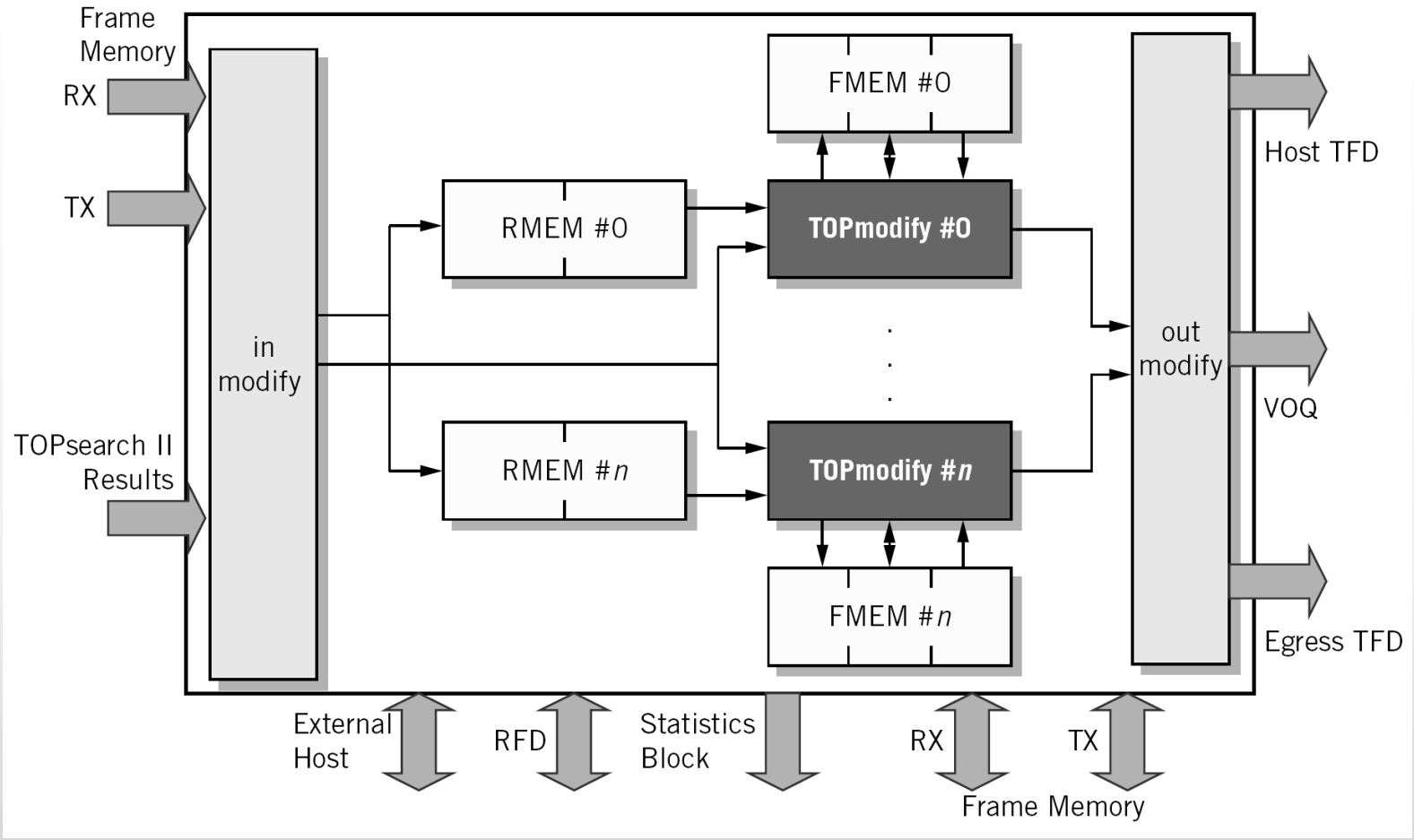
TOPresolve


Messages from TOPparse
TOPsearch I Results
in resolve
RMEM #0
TOPresolve #0
OMEM #0
RMEM #n
TOPresolve #n
OMEM #n
out resolve
TOPsearch II Keys and Messages to TOPmodify
TOPresolve High-Learn Entries to TOPsearch I
RFD out (frame discard)
External Host
Statistics Block

TOPsearch II



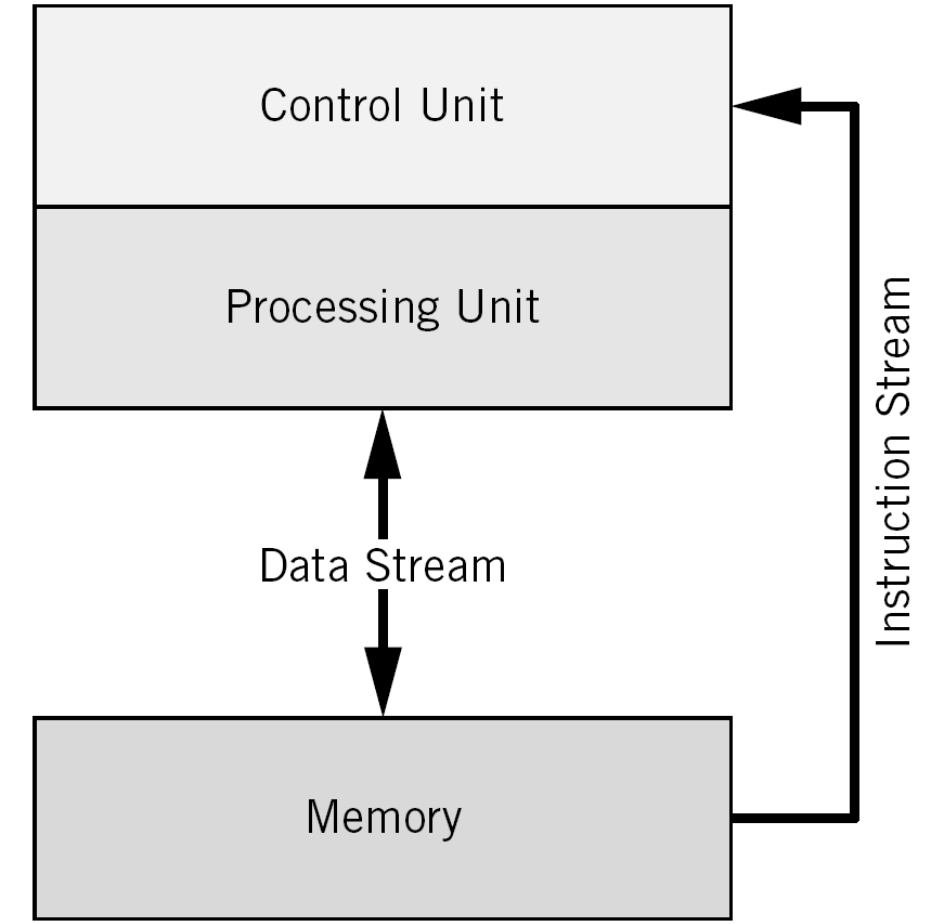
TOPmodify


Frame Memory
RX
TX
TOPsearch II Results
in modify
RMEM #0
RMEM #n
FMEM #0
TOPmodify #0
TOPmodify #n
FMEM #n
out modify
Host TFD
VOQ
Egress TFD
External Host
RFD
Statistics Block
RX
TX
Frame Memory

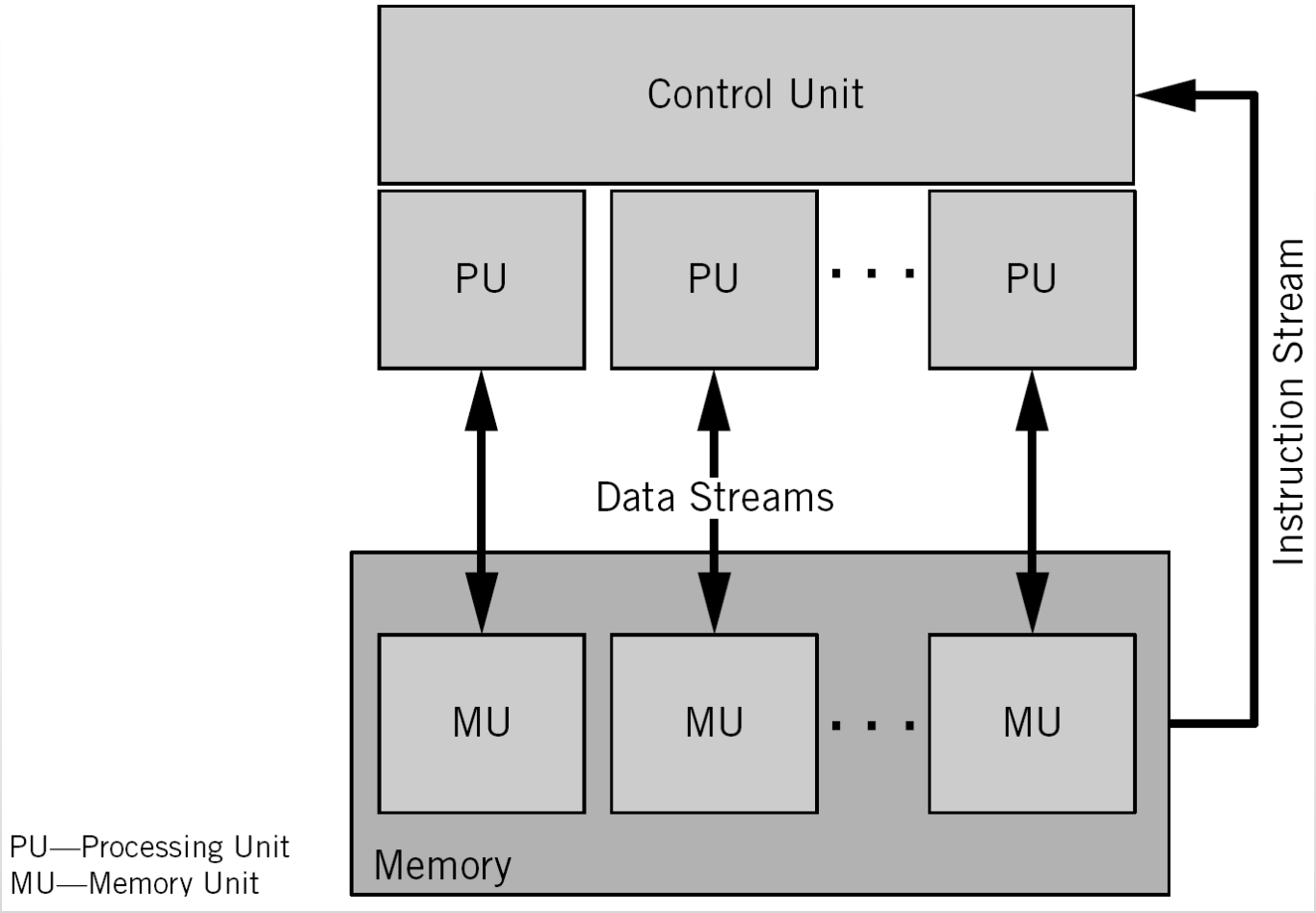


SYSTEMY WIELOPROCESOROWE

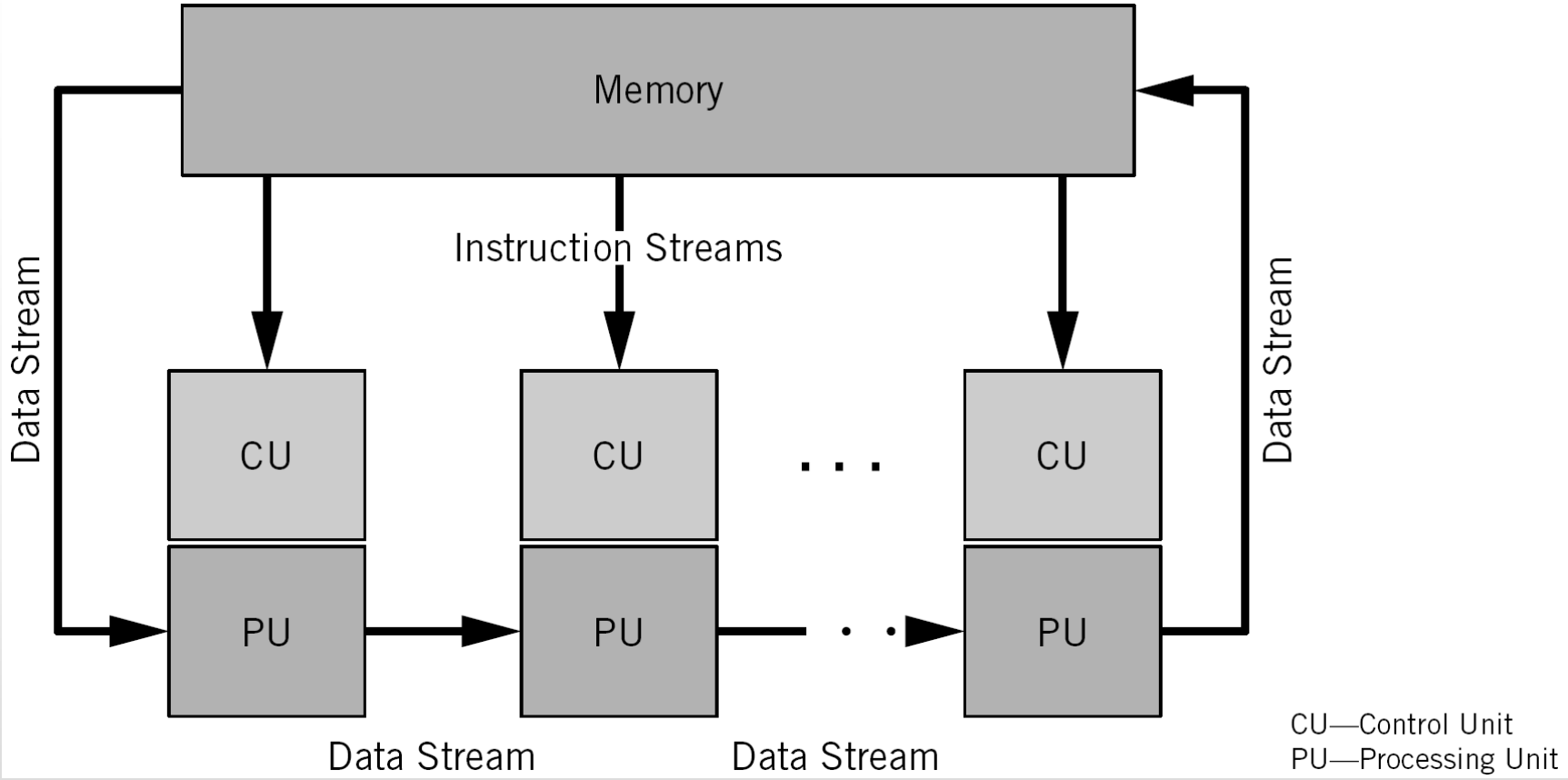
SISD


Control Unit
Processing Unit
Data Stream
Memory
Instruction Stream

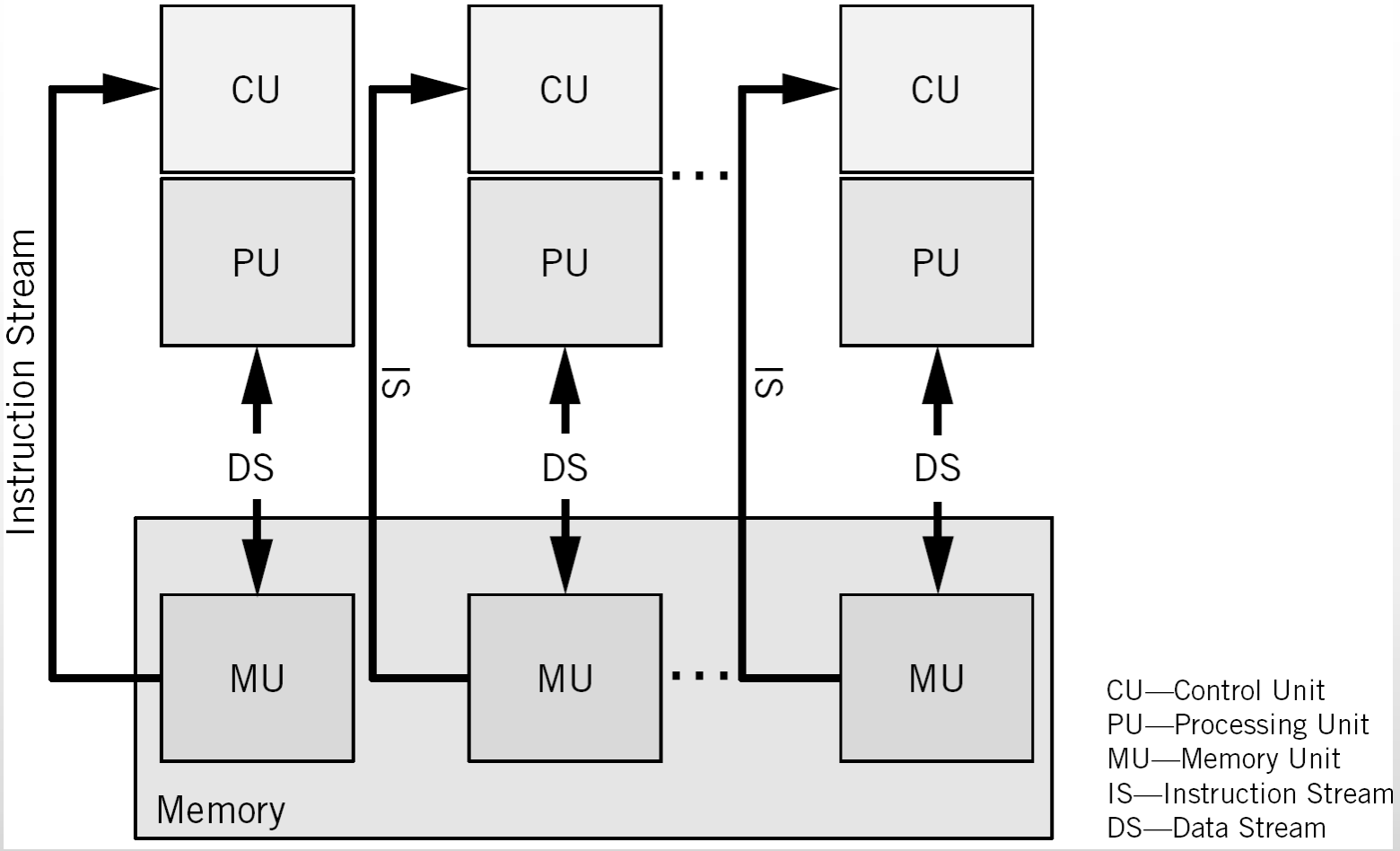
SIMD


Control Unit
PU
PU
PU
Data Streams
Instruction Stream
MU
MU
MU
Memory
PU—Processing Unit
MU—Memory Unit

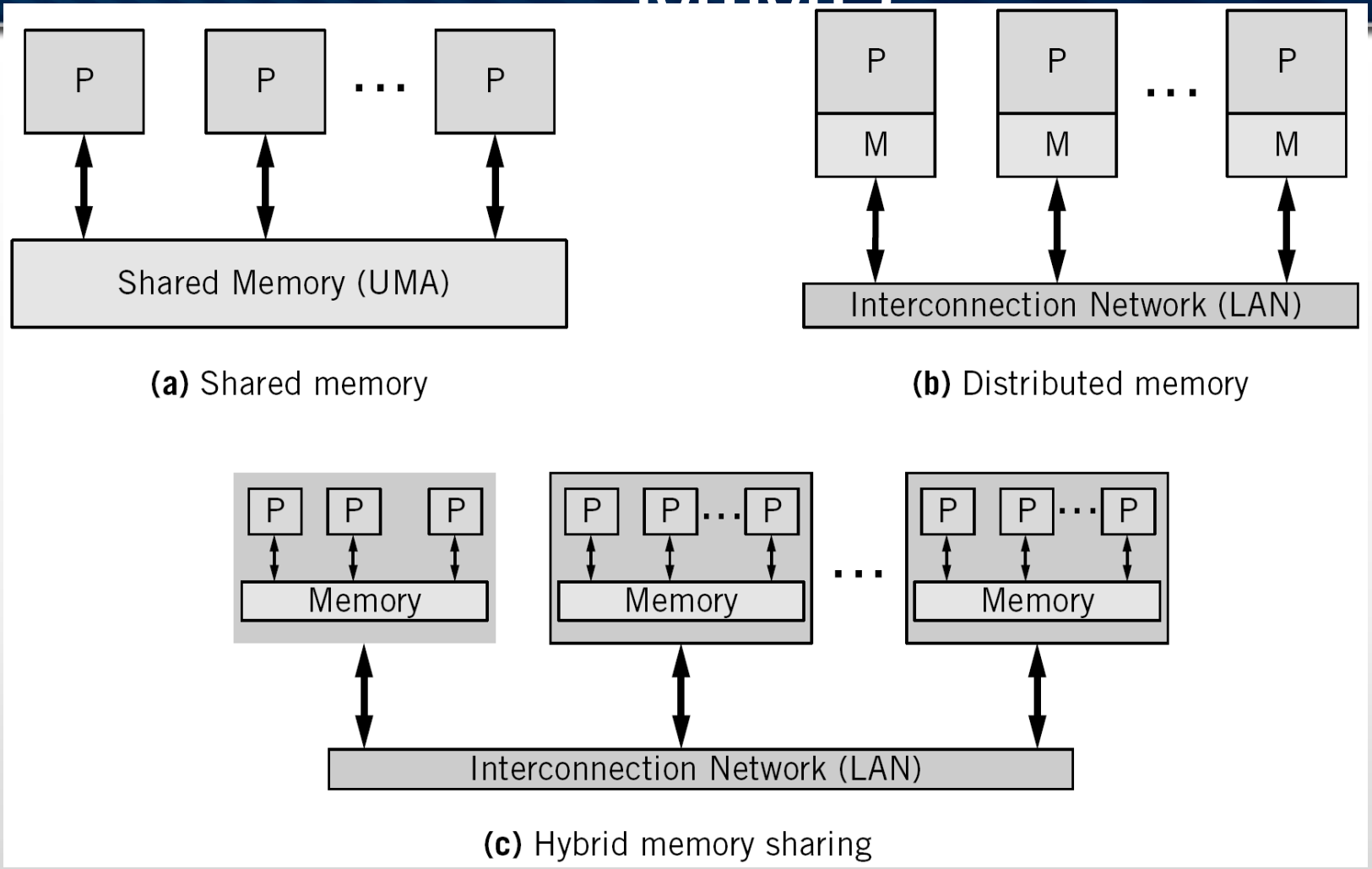
MISD


Memory
Instruction Streams
Data Stream
Data Stream
CU
CU
CU
PU
PU
PU
Data Stream
Data Stream
CU—Control Unit
PU—Processing Unit

MIMD


CU
CU
CU
PU
PU
PU
Instruction Stream
SI
SI
DS
DS
DS
MU
MU
MU
Memory
CU—Control Unit
PU—Processing Unit
MU—Memory Unit
IS—Instruction Stream
DS—Data Stream

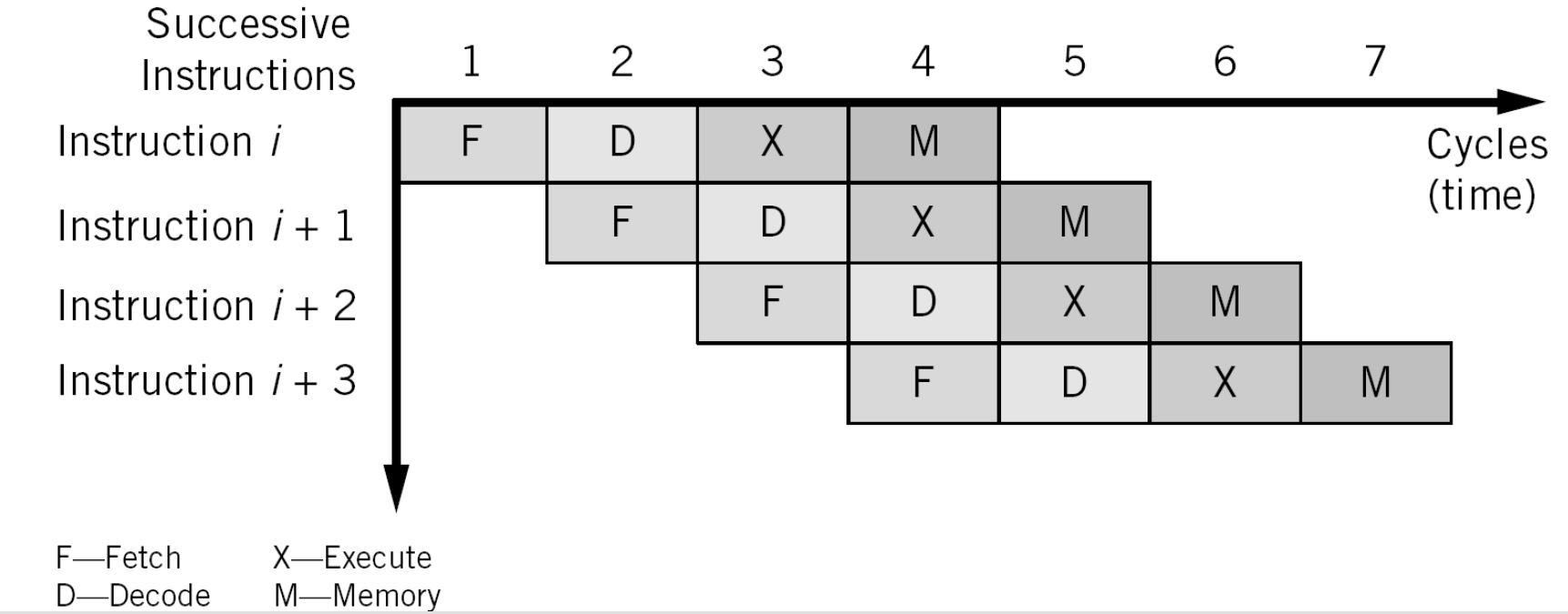
Wykorzystanie pamięci w MIMD



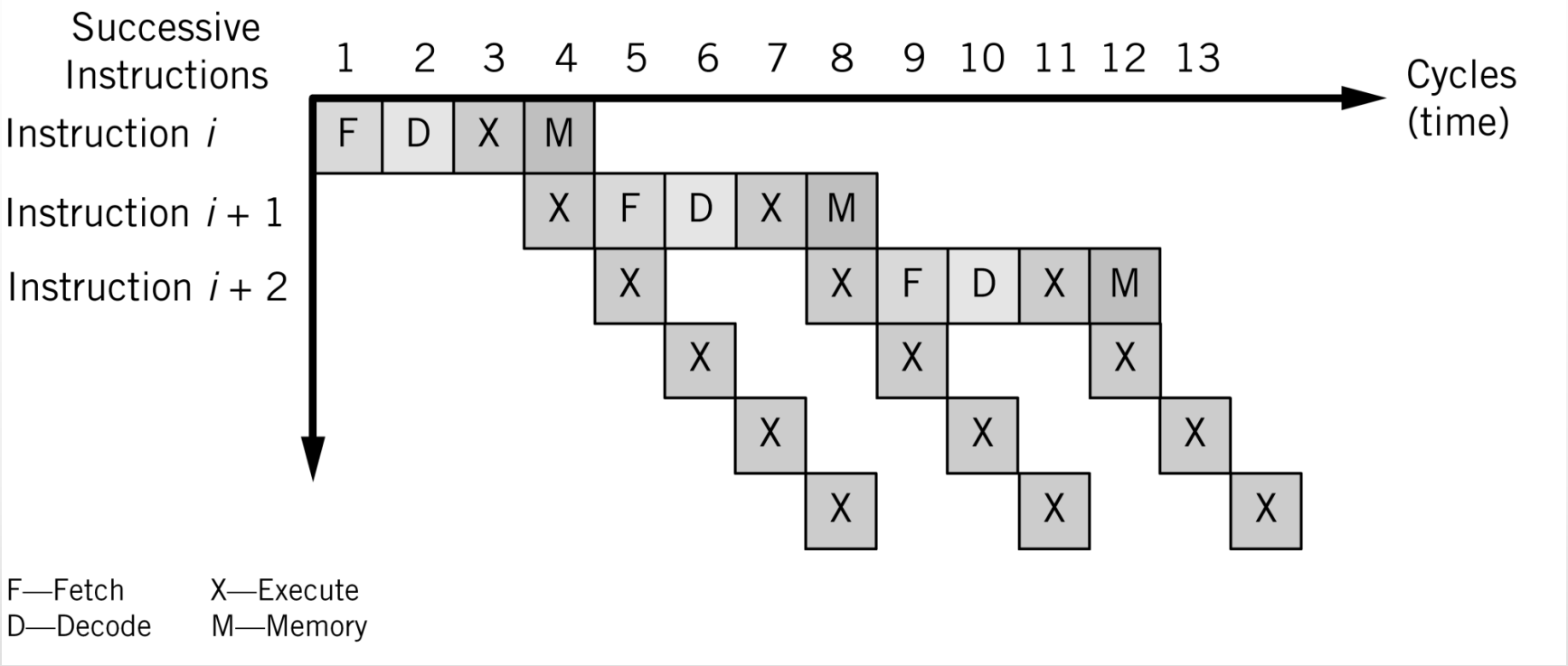


PRZETWARZANIE INSTRUKCJI

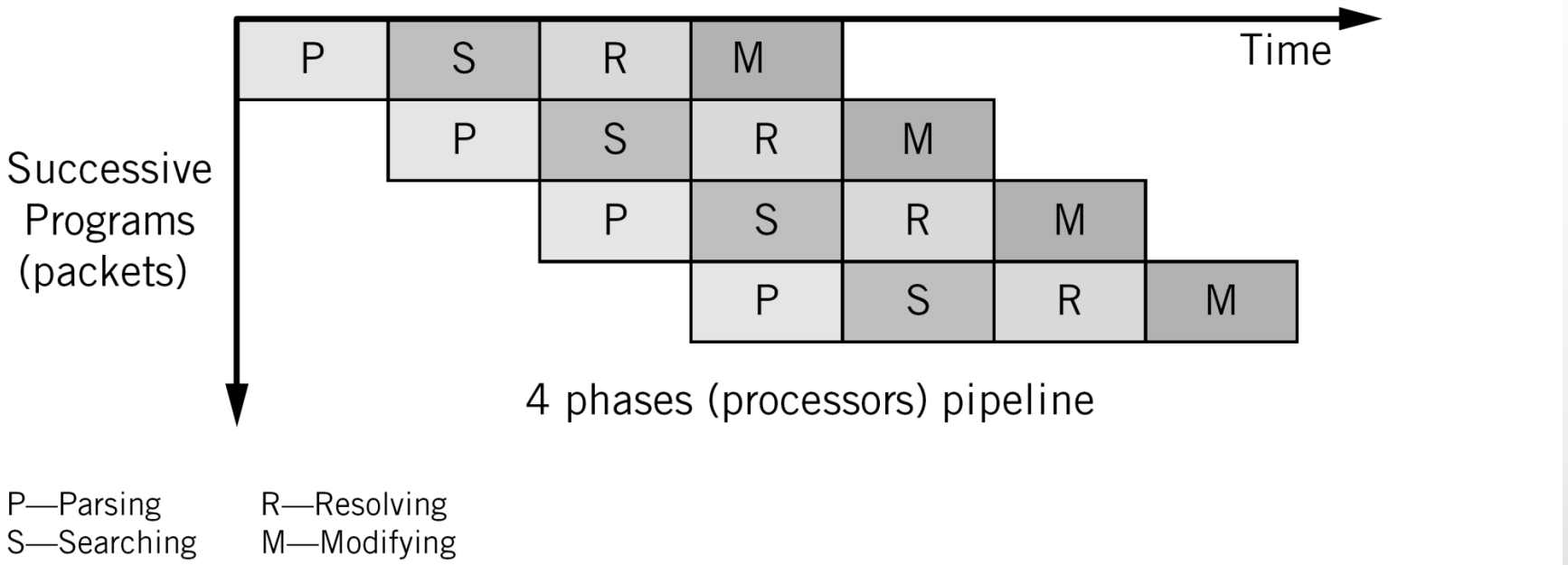
Przetwarzanie potokowe



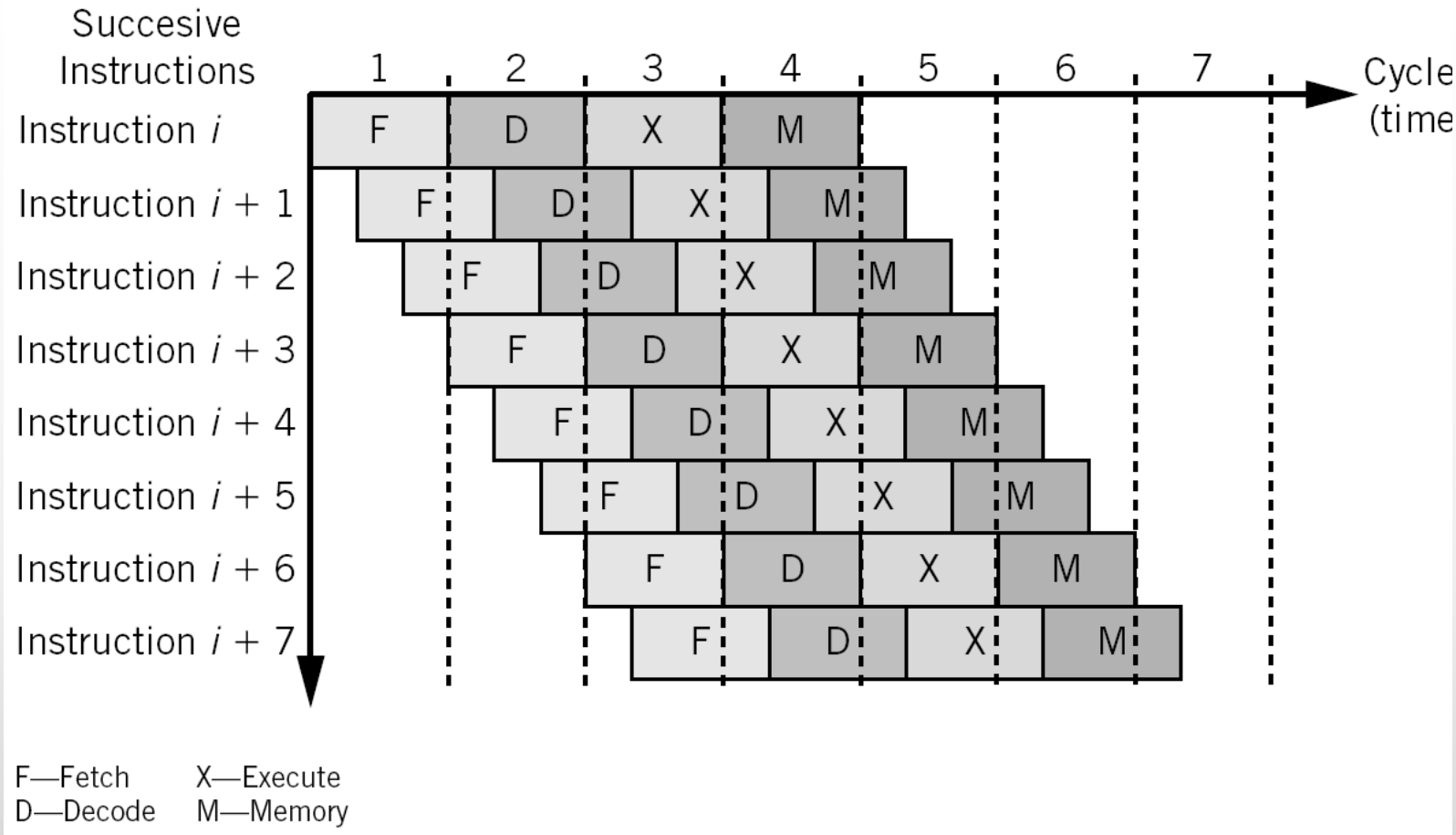
Wektorowe przetwarzanie potokowe



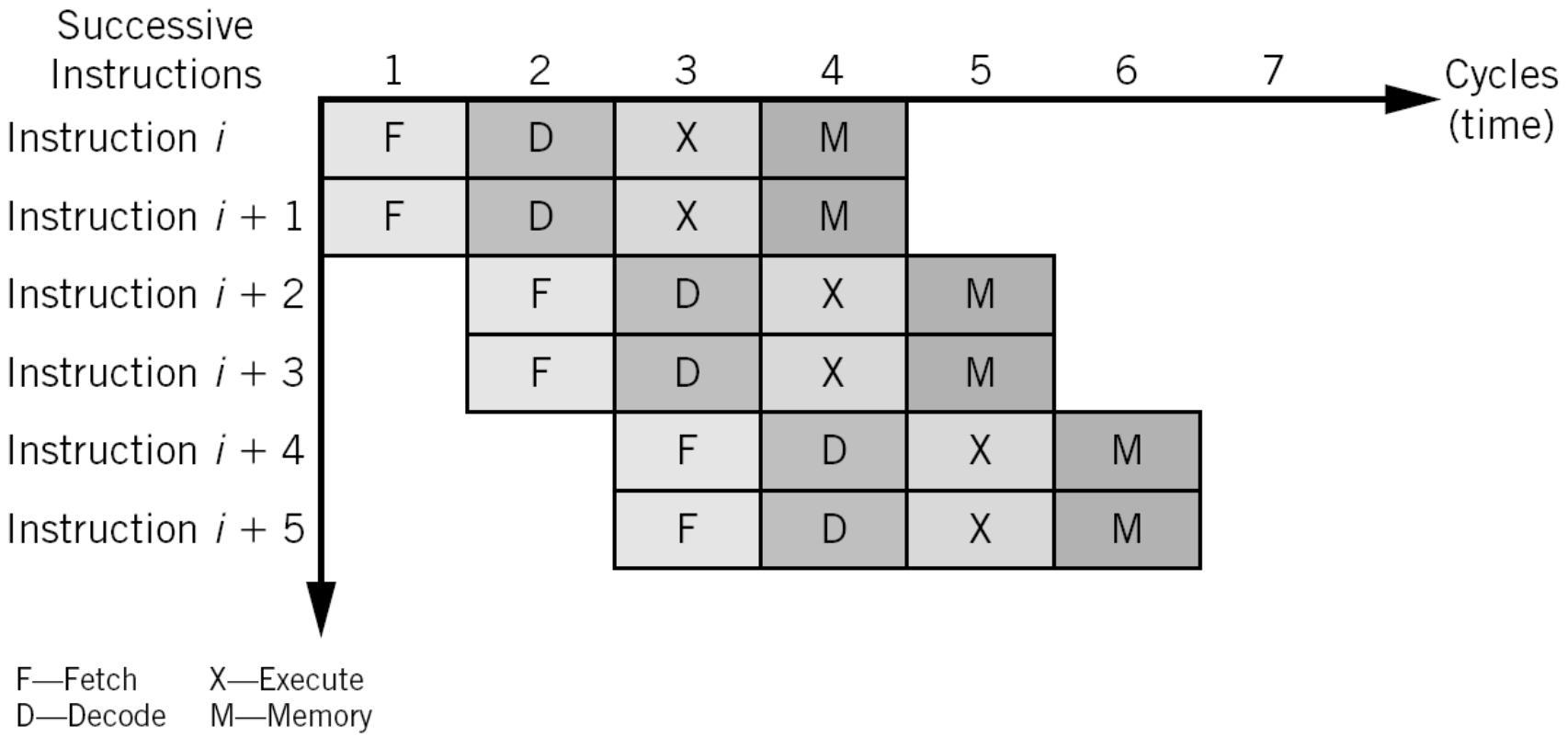
Przetwarzanie potokowe programów



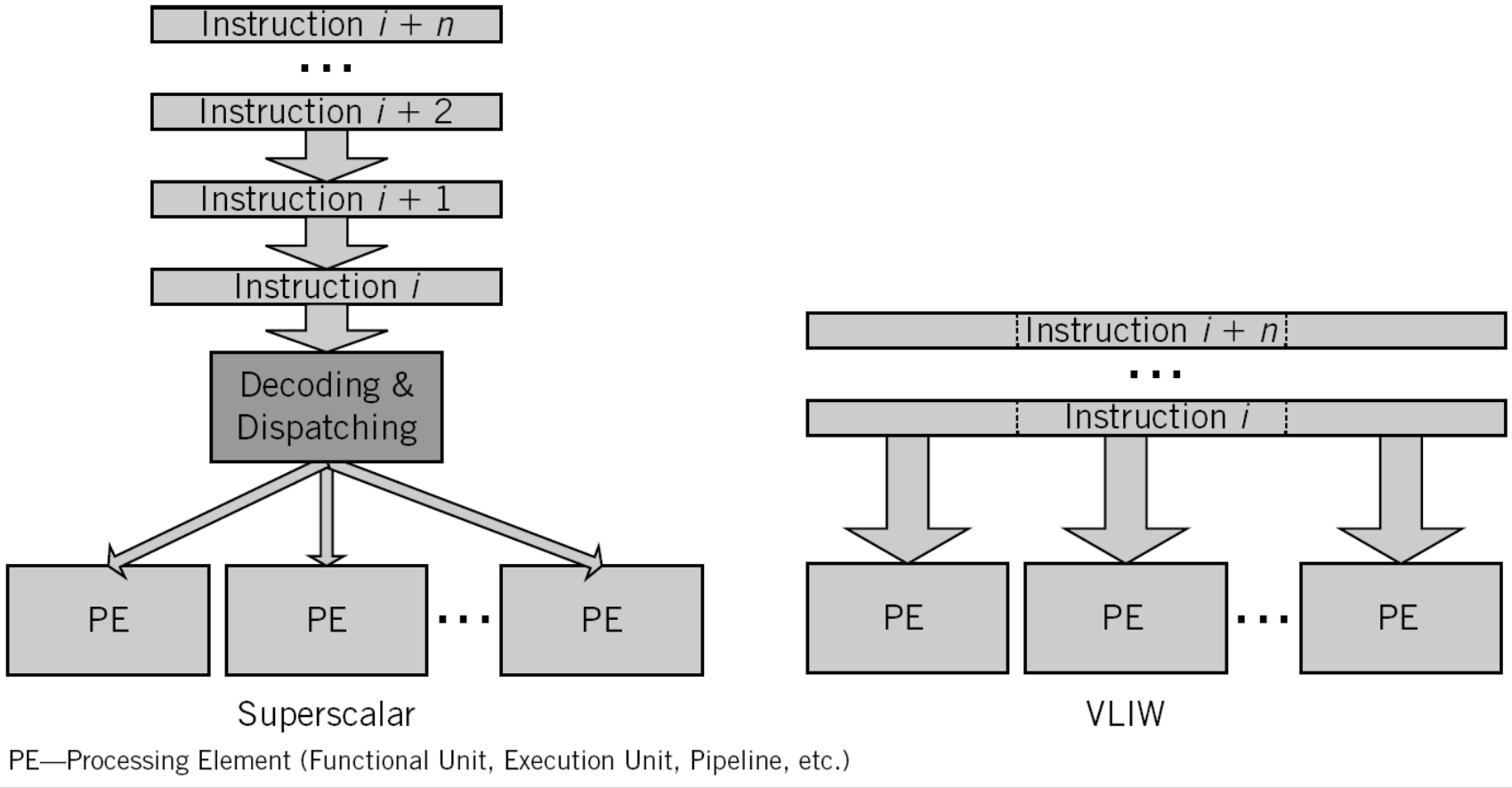
Przetwarzanie potokowe stopnia 3



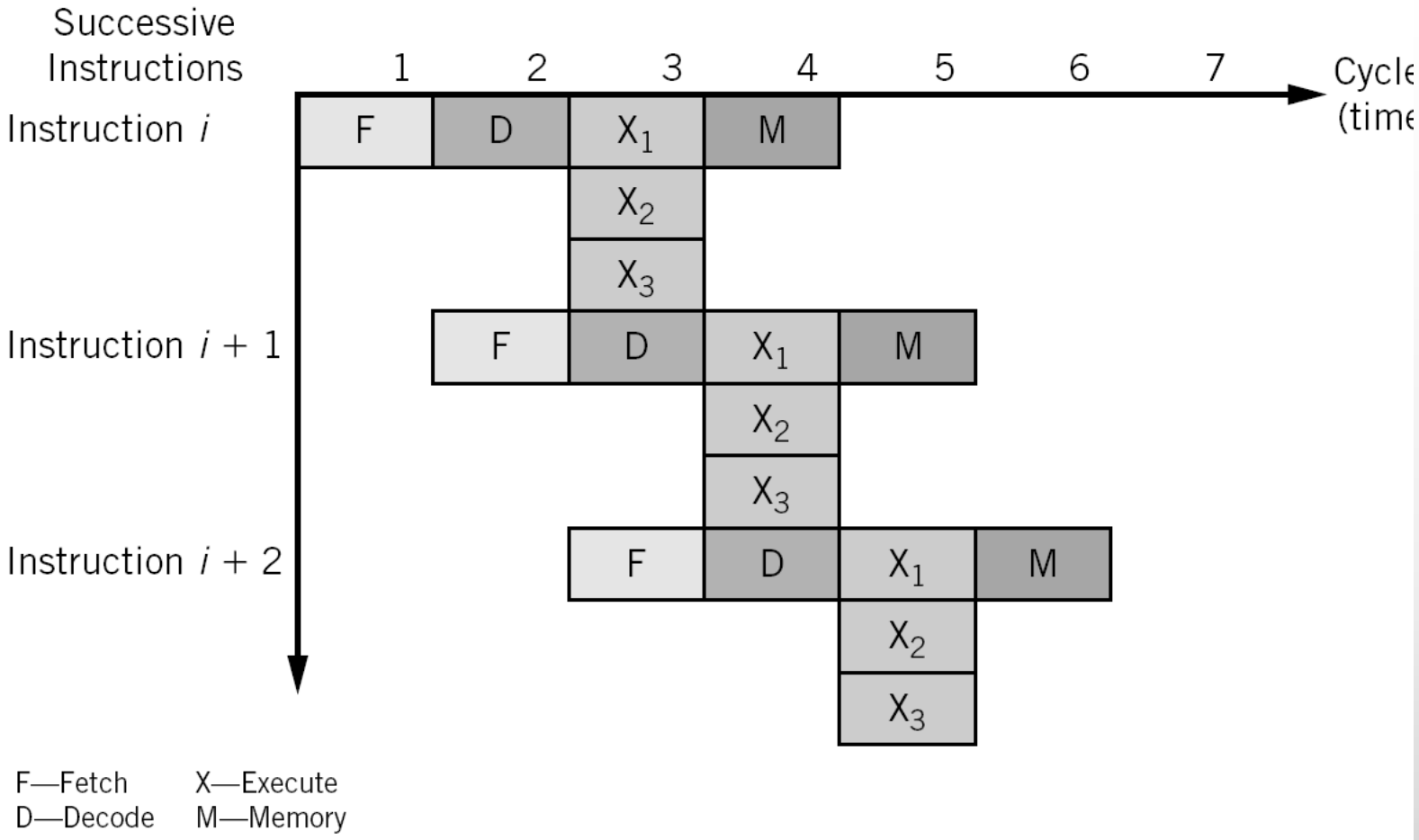
Przetwarzanie superskalarne stopnia 3



VLIM i przetwarzanie superskalarne



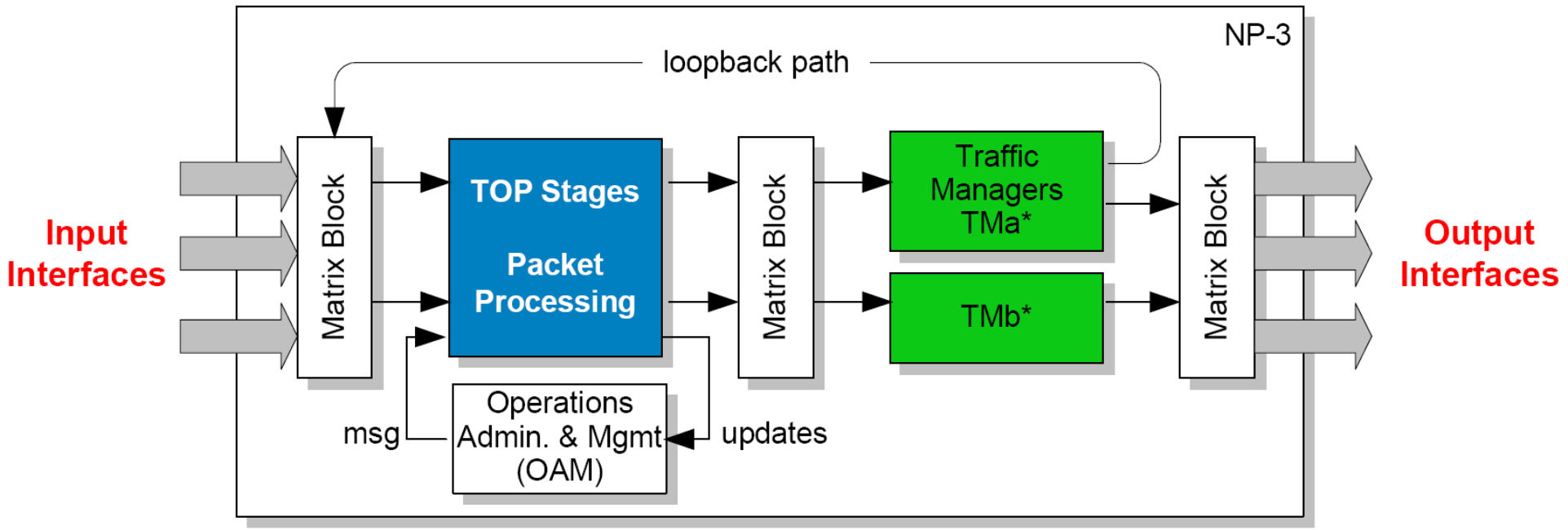
VLIM


Successive Instructions
1
2
3
4
5
6
7
Cycle (time
Instruction i
Instruction i + 1
Instruction i + 2
F
D
X1
X2
X3
M
F—Fetch
D—Decode
X—Execute
M—Memory

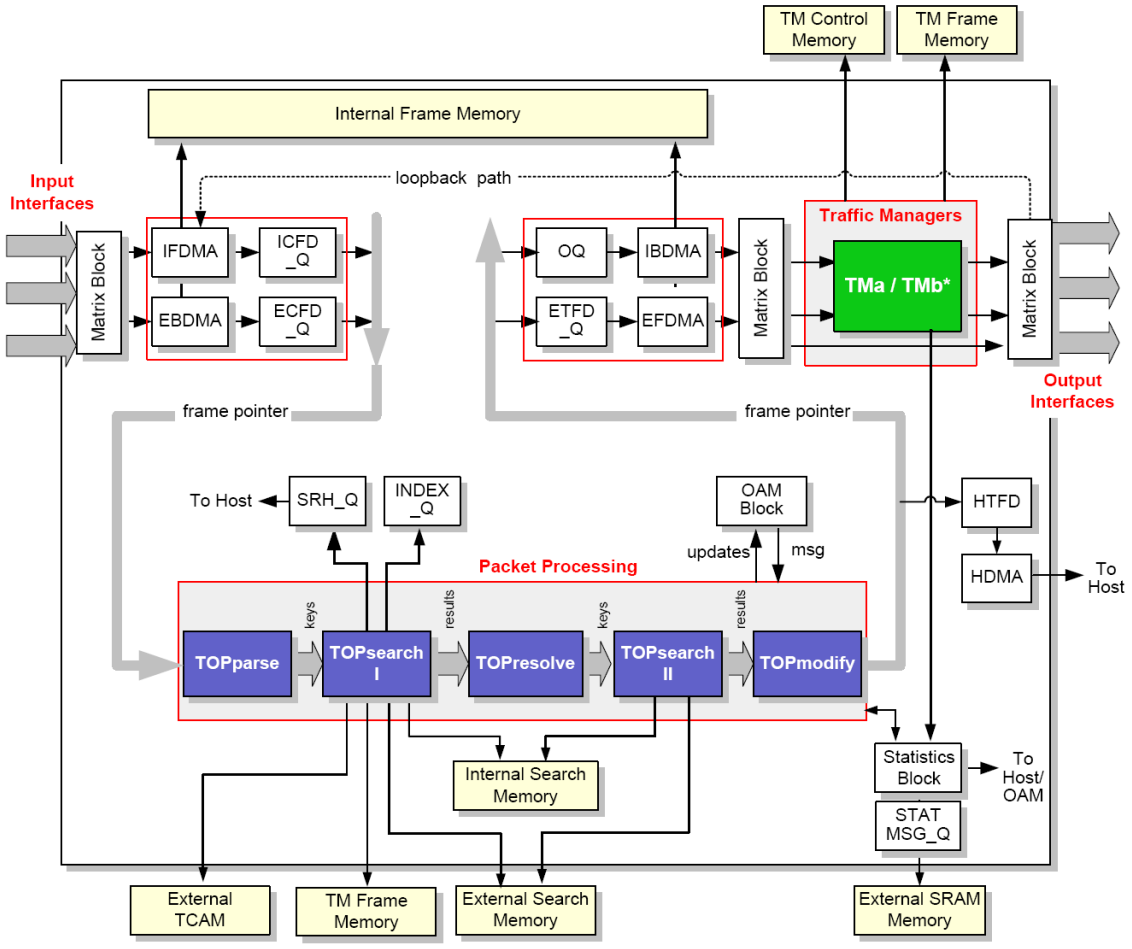


PROGRAMOWANIE PROCESORÓW SIECIOWYCH

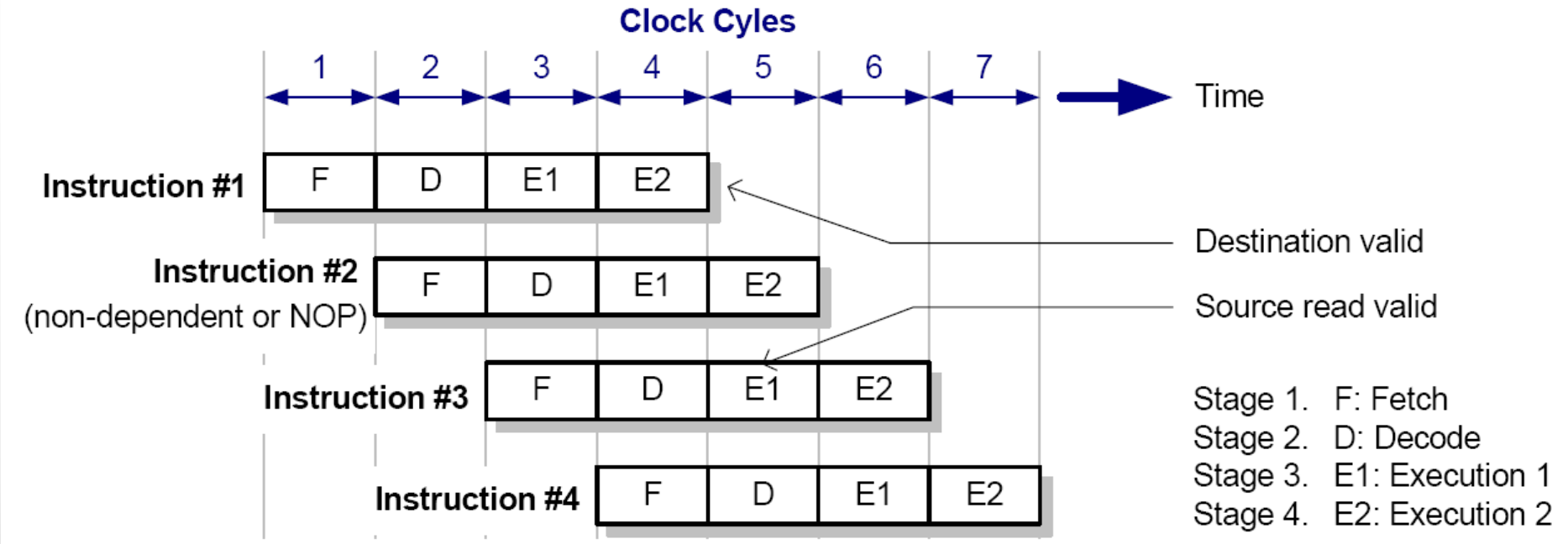
Diagram przepływu danych



Przepływ danych przez procesor

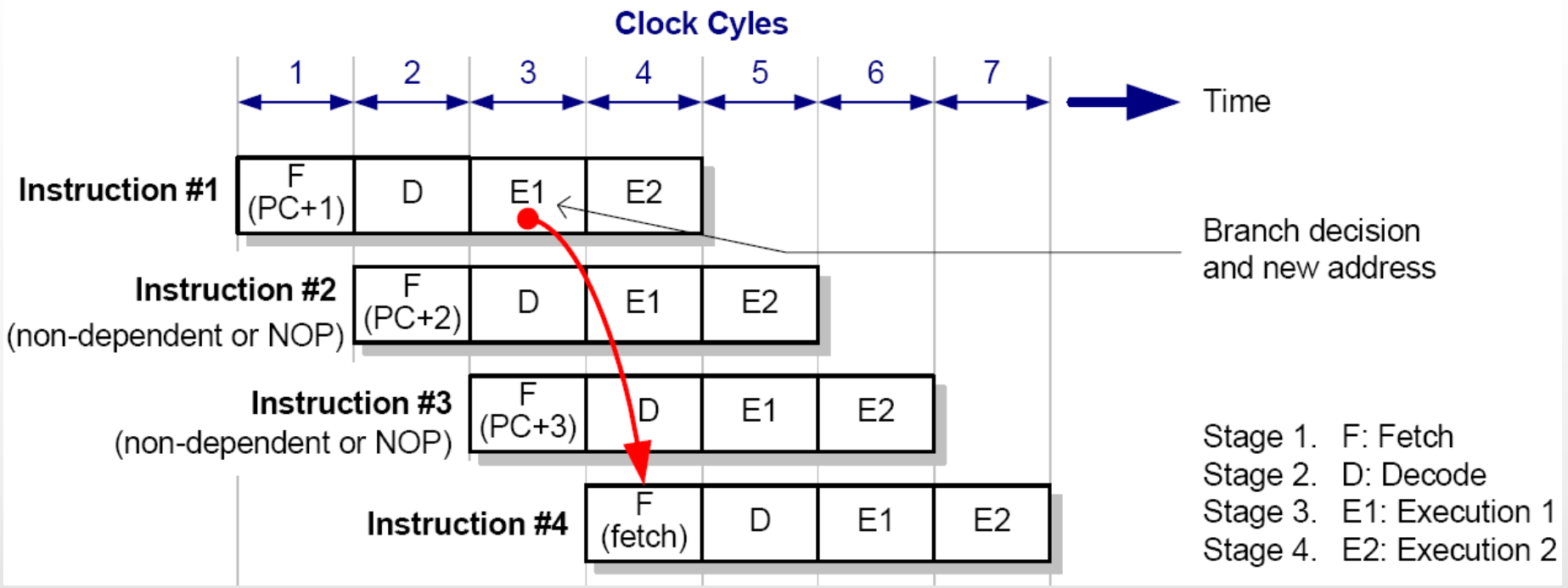


Potokowe przetwarzanie instrukcji



Niebezpieczne fragmenty kodu

```
Mov    UREG[ 4 ], UREG[2].BYTE[2], 2;
Mov    ALU, UREG [ 4 ], 2;                    // UREG[ 4 ] is not ready
```



Przykład 1

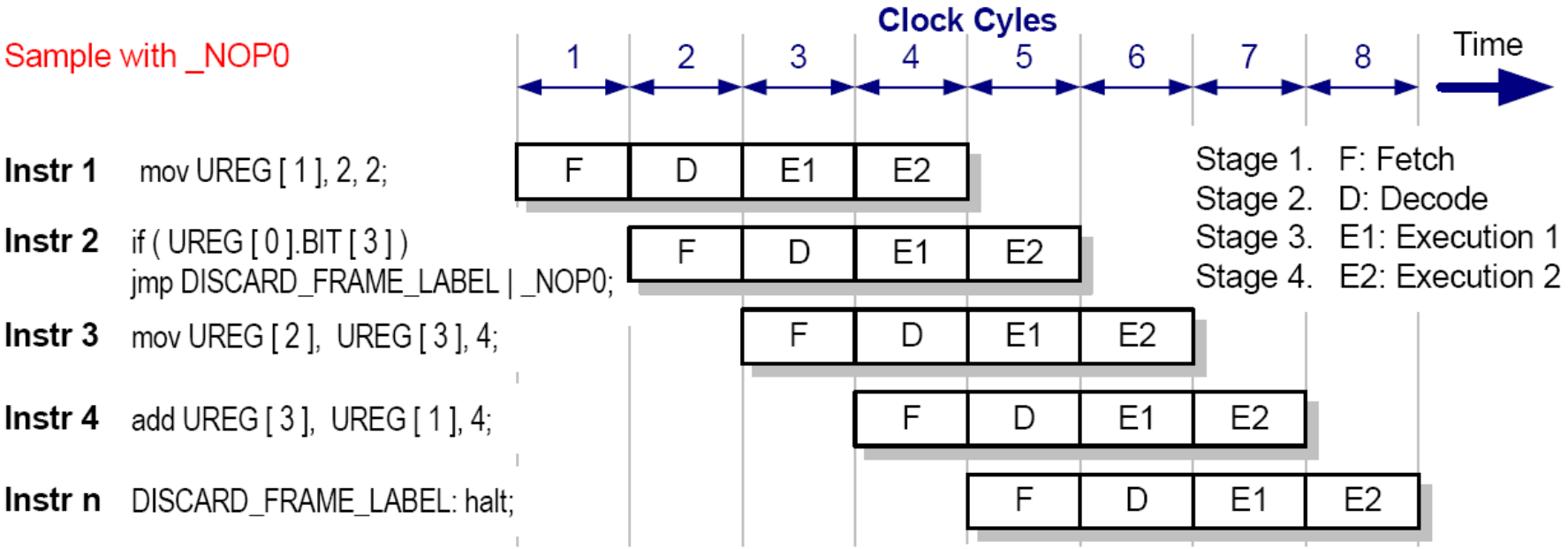
```
if (!UREG[0].BIT[3]) Jmp NEWLABEL |_NOP0;
Get UREG[6], 0(RD_PTR), 4;
Mov UREG[7], 0x8000, 4;
NEXT_COMMAND:
...
NEWLABEL: ...
```

Przykład 2

```
1    mov UREG [ 1 ], 2, 2;
2    if ( UREG [ 0 ].BIT [ 3 ] )
        jmp DISCARD_FRAME_LABEL | _NOPx;
3    mov UREG [ 2 ],  UREG [ 3 ], 4;
4    add UREG [ 3 ],  UREG [ 1 ], 4;
     ...
n    DISCARD_FRAME_LABEL:
        halt;
```

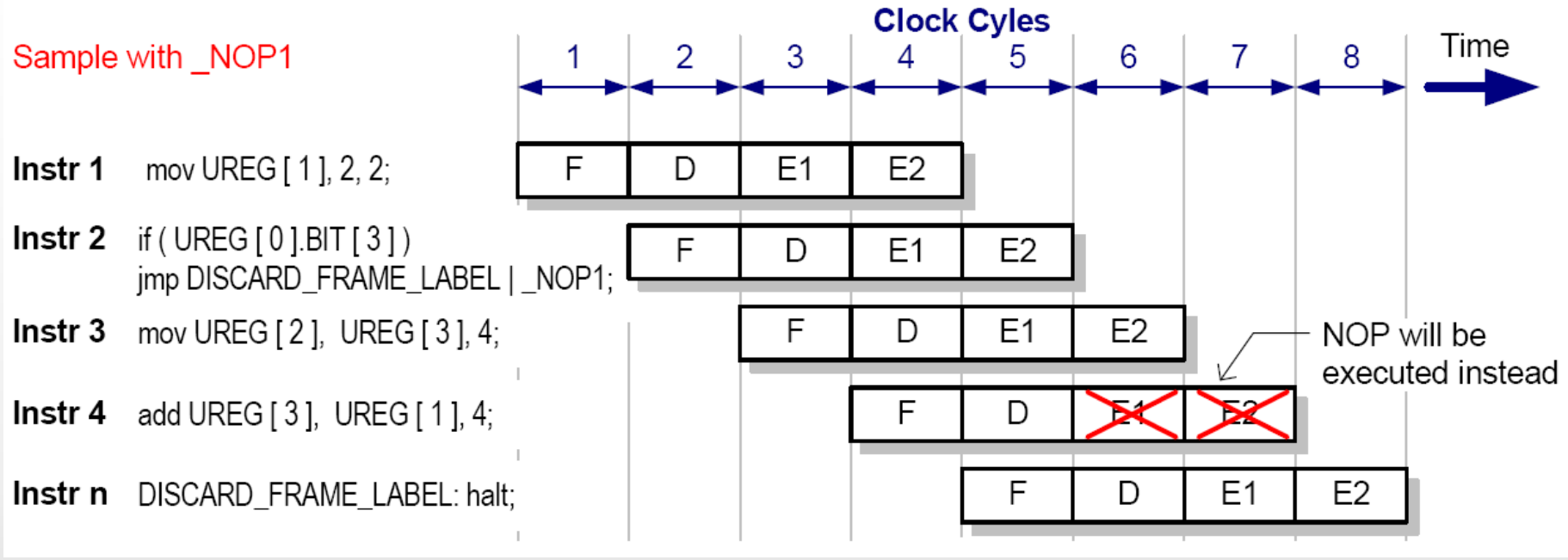
NOP0

Sample with _NOP0



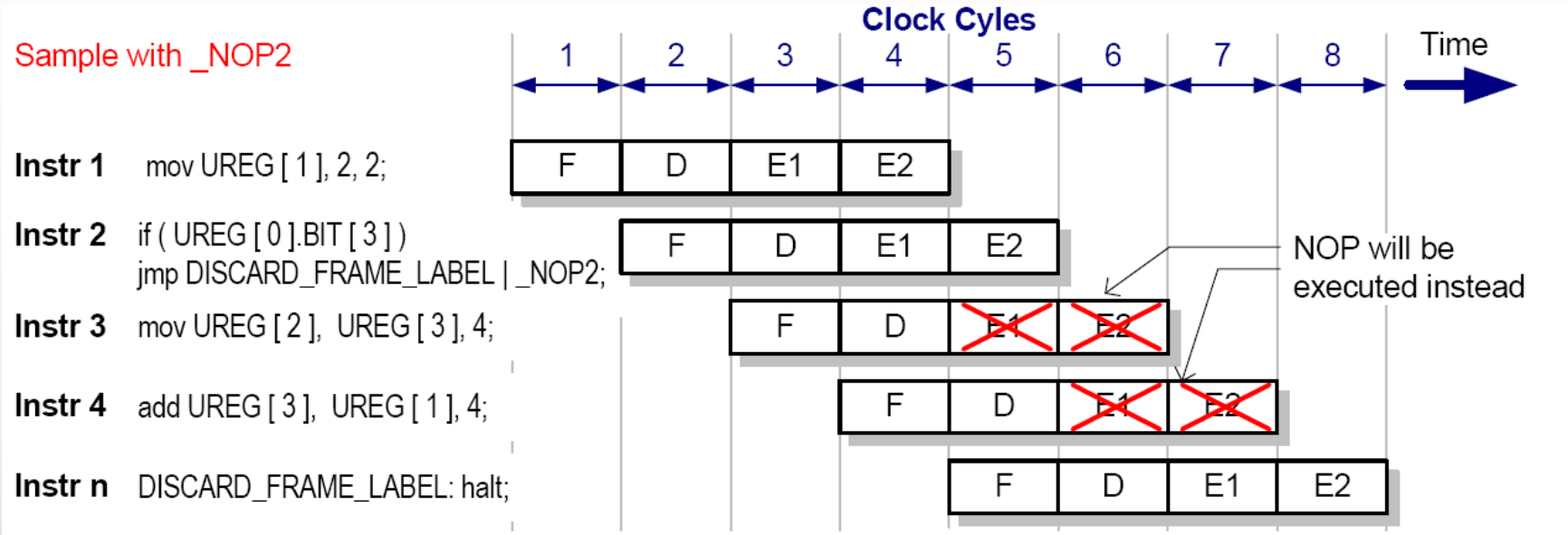
NOP1

Sample with _NOP1

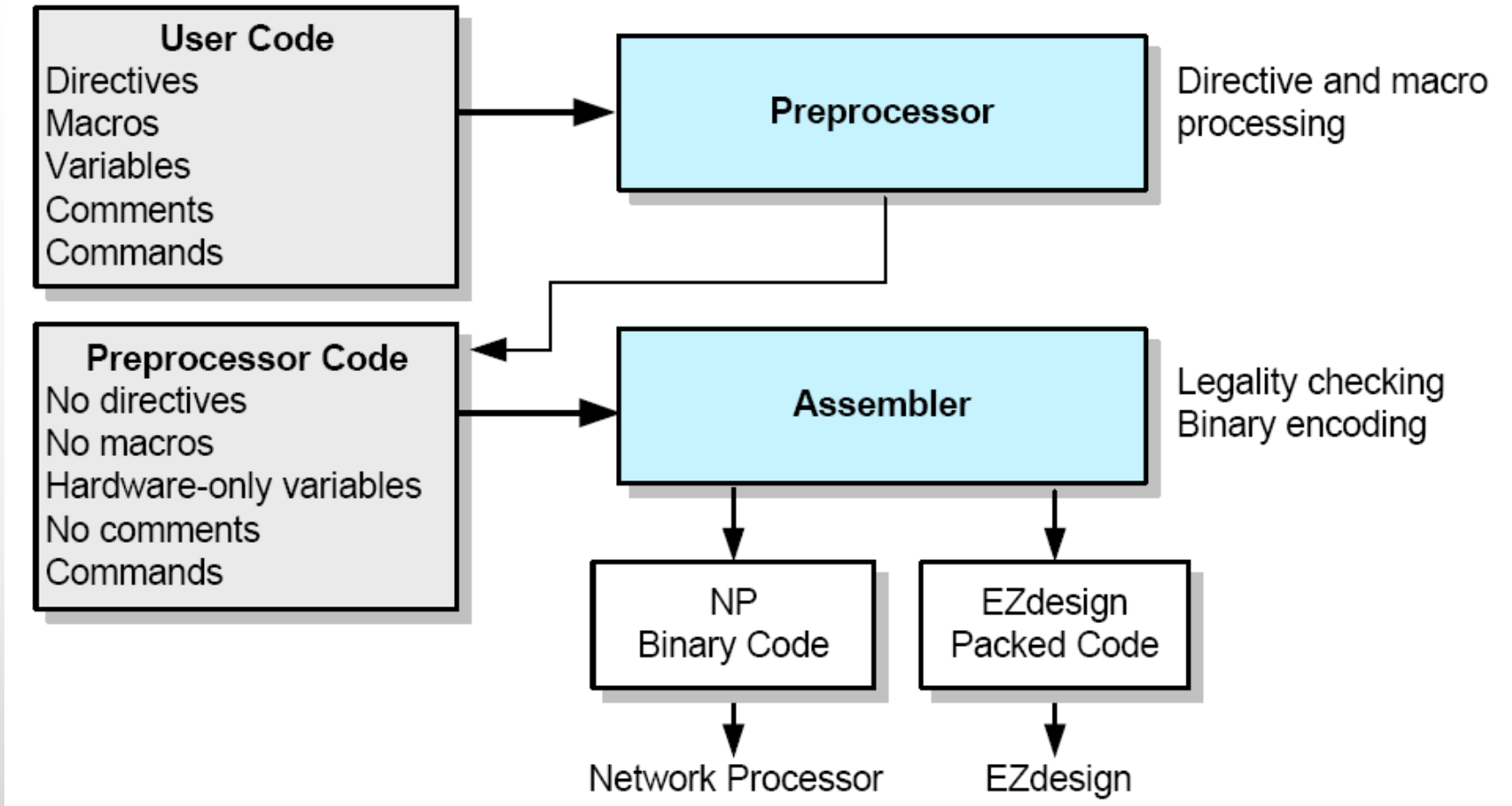


NOP2

Sample with _NOP2



Przetwarzanie programu



Ogólna składnia

```
<LABEL:>  <if  (CONDITION)>  COMMAND  <OPERANDS> [OPERANDS];
```

```
…
BREAK_POINT_LAB:
       mov CNT,  2, 2;
       mov SREG [ 5 ], 0, 2;
WRITE_ALL_UREGS_LAB:
       if ( !HOST_RDY )
              jmp  WRITE_ALL_UREGS_LAB | _NOP2; // insert 2 NOPs in pipeline
       loop WRITE_ALL_UREGS_LAB;                // no NOPs inserted in pipeline
```

```
/*comment
and comment*/
//comment until a line end
```

Symbole specjalne

SYMBOL	DESCRIPTION	EXAMPLE
$	The current PC (program counter) value.	`mov UREG[0], $, 2;` `jz $ + 10;`
+ following ")"	Auto-increment base register or index register after a read/write operation.	`get UREG [1], 20 (RD_PTR)+, 2;`
&	Create immediate data with indirect access format (see the following section).	`mov UREG[1], &UREG[ 0 ].BYTE[ 2 ], 2;`

Znaki specjalne

COMBINATION	*DESCRIPTION*
\r	The “carriage return” character, ASCII 10.
\n	The “new line” character, ASCII 13.
\t	The tab character.
\\	The backslash character.
\”	The quotation mark character.
xx, where *xx* is a two-digit hexadecimal number	An ASCII character with the code *xx*.

Identyfikacja zasobów

bit#	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	x	x	x	x	Register index				x	x	x	byte #		bit #		

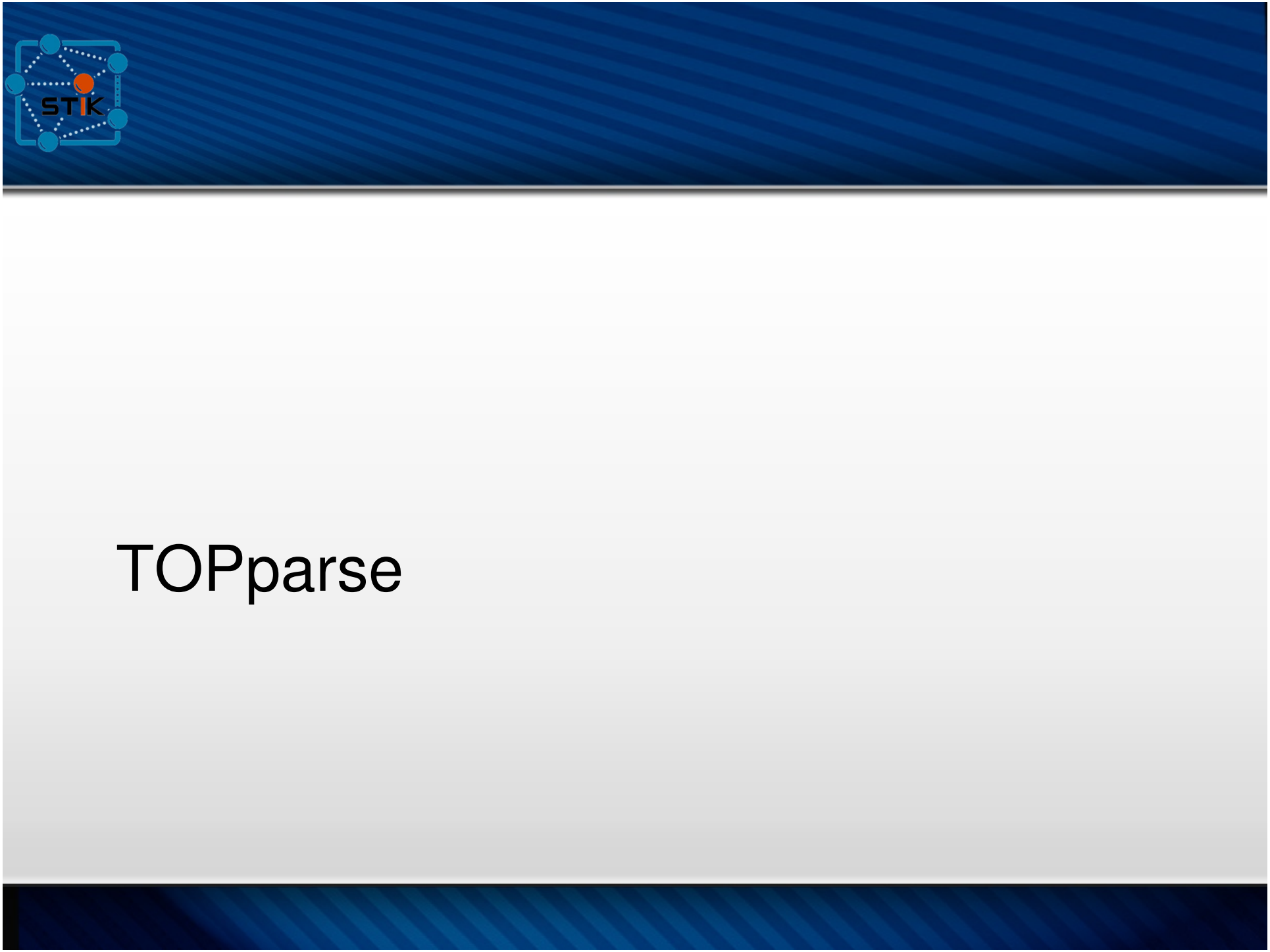
```
...
mov SREG [ 5 ], &UREG[2].BYTE[3], 2;          // Now SREG[5] contains a reference
                                              // to UREG [ 2 ].BYTE[3]
...
mov UREG[ 0 ], UREG [ SREG [ 5 ]], 1;         // move 1 byte from UREG [ 2 ].BYTE[3]
add SREG [ 5 ], SREG [ 5 ], 0x0100, 2;        // create next index using the syntax
                                              // shown
...
mov UREG[ 0 ], UREG [ SREG [ 5 ]], 1;         // move 1 byte from UREG [ 3 ].BYTE[3]
```

Sposoby adresowania

Immediate	123 or 0x12 or “abcd” or $ or label	123 - decimal number 0x12 - hexadecimal number “abcd” - 4 bytes = 0x61626364 $ - program counter (PC) label - program label
Register	*device* **[** *number* **]**	For the device, see the devices listed for each TOP in its relevant section. *number* - index of the register in the device array
Register (byte-specific)	*device* **[** *number1* **].byte [** *number2* **]**	*number1* - index of the register in the device array *number2* - byte number in this register
Register (bit-specific)	*device* **[** *number1* **].bit [** *number2* **]**	*number2* - bit number in this register
Register (four-bit)	*device* **[** *number1* **].bits [***n1*,*n2*,*n3*,*n4***]**	*n1*..*n4* - specific bits in this register
Indirect	*device* **[***IND_REG***]**	
Base-index	*offset* **(** *base* **)** or *offset* **(** *base* **)+**	*base* - base registers. ▸ *See the devices listed for each TOP in its relevant section.* *offset* - immediate or some register.

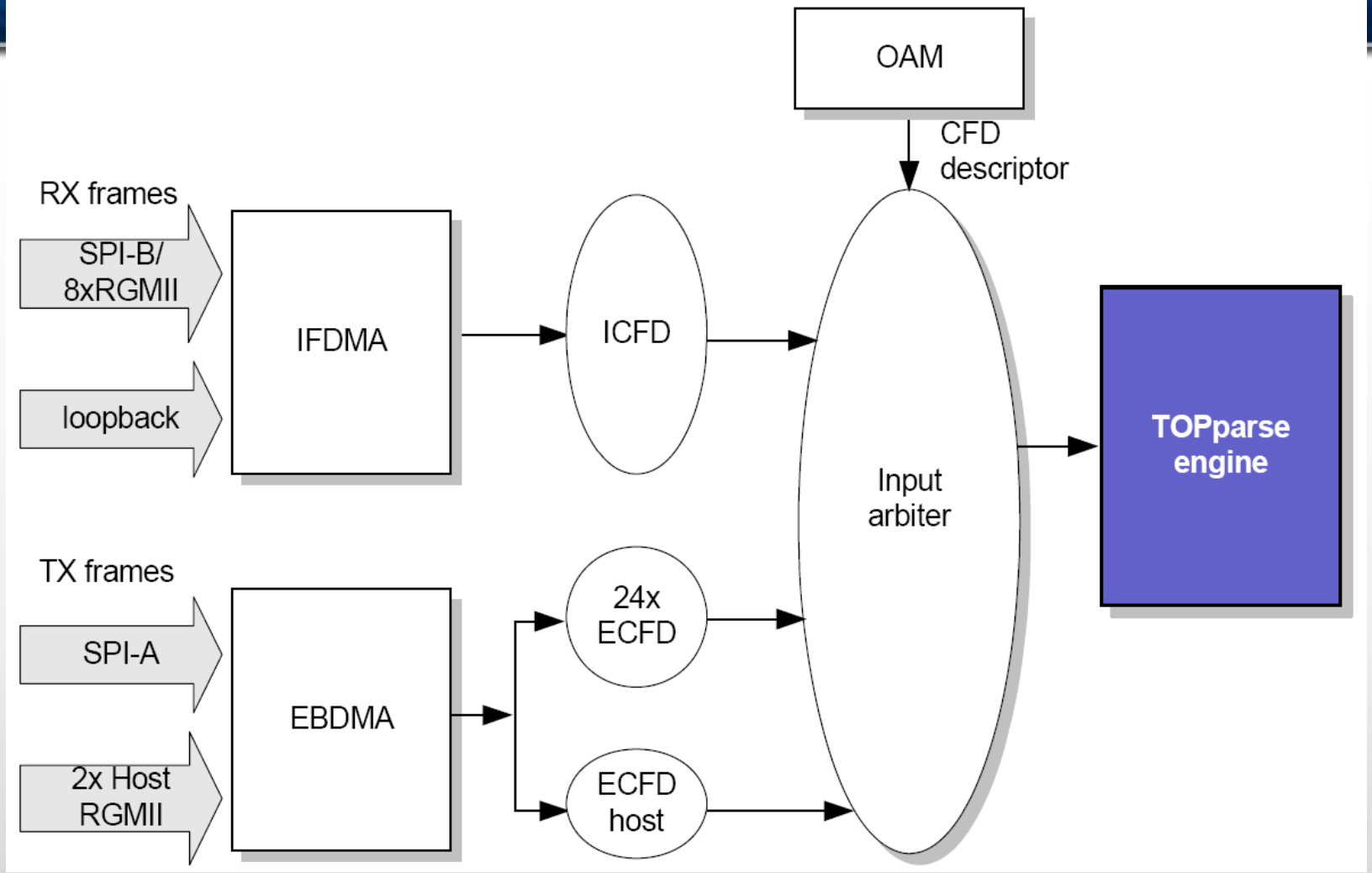
Sposoby adresowania – c.d.

Direct	*number* **(0)** or **(***number***)+**	Can be used for the Get or Copy commands. For src in TOPparse copy and get instructions, use *number* **(0)+**.
Memory (byte-specific)	**hreg [***HBS***].byte[***num2***]**	
Memory (bit-specific)	**hreg [***HBS***].bit[***num2***]**	
Memory (four-bit)	**hreg [***HBS***].bits [***n1***,***n2***,***n3***,***n4***]**	
Device group	**bcam8 [***num1***] or bcam8 [ureg[0]]** **bcam16[***num1***] or bcam16[ureg[0]]** **tcam [***num1***]** or **tcam [ureg[0]]**	*num1* - immediate 3 bits
Control registers in result memory (bit specific access with group registers)	**sreg[8].byte[***n1***].index[***n2***].bit[***n3***]**	*n1* - (0, 1) - group number register *n2* - (0, 1,..7) - structure index *n3* - (0, 1,…31) - bit offset
Control registers in result memory (bit specific access without group registers)	**strnum [***num1***].bit[***num2***]**	*num1* - (0,1…63) structure number *num2* - (0,1,…31) - bit offset …
Structure number-offset	*offset* **(***str* **)** or *offset* **(***str***)+**	*str* - register or immediate. Access to result memory. *offset* - immediate or some register.

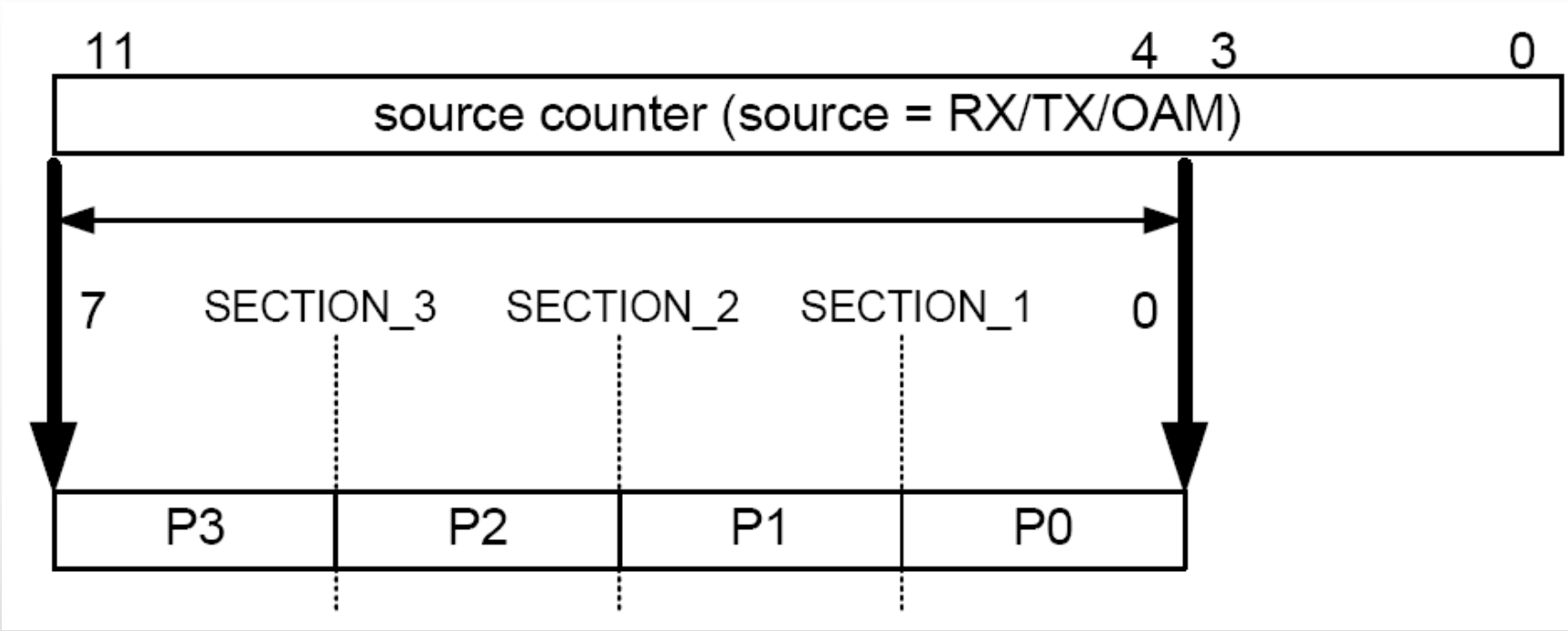


TOPparse

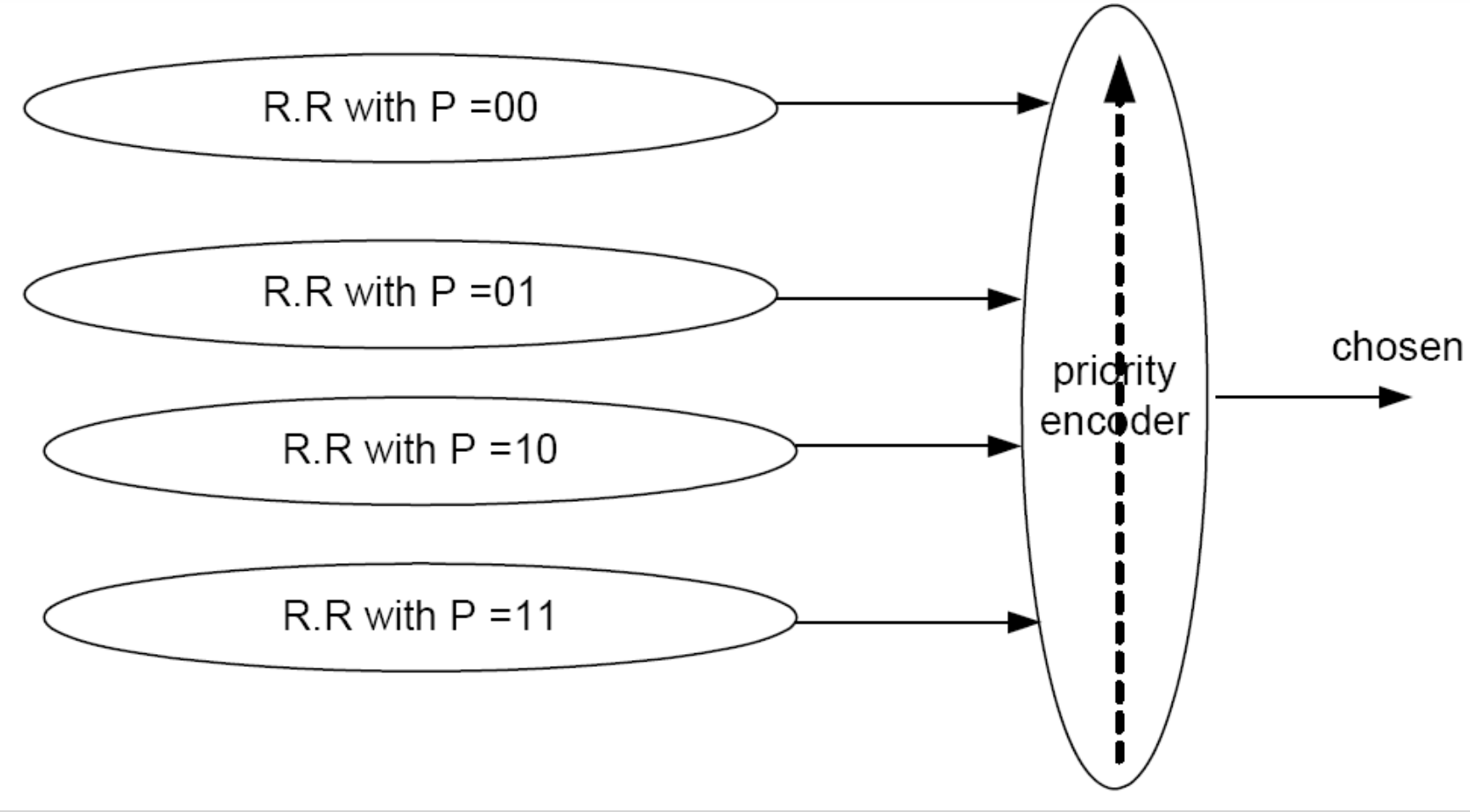
Wejściowy układ planowania



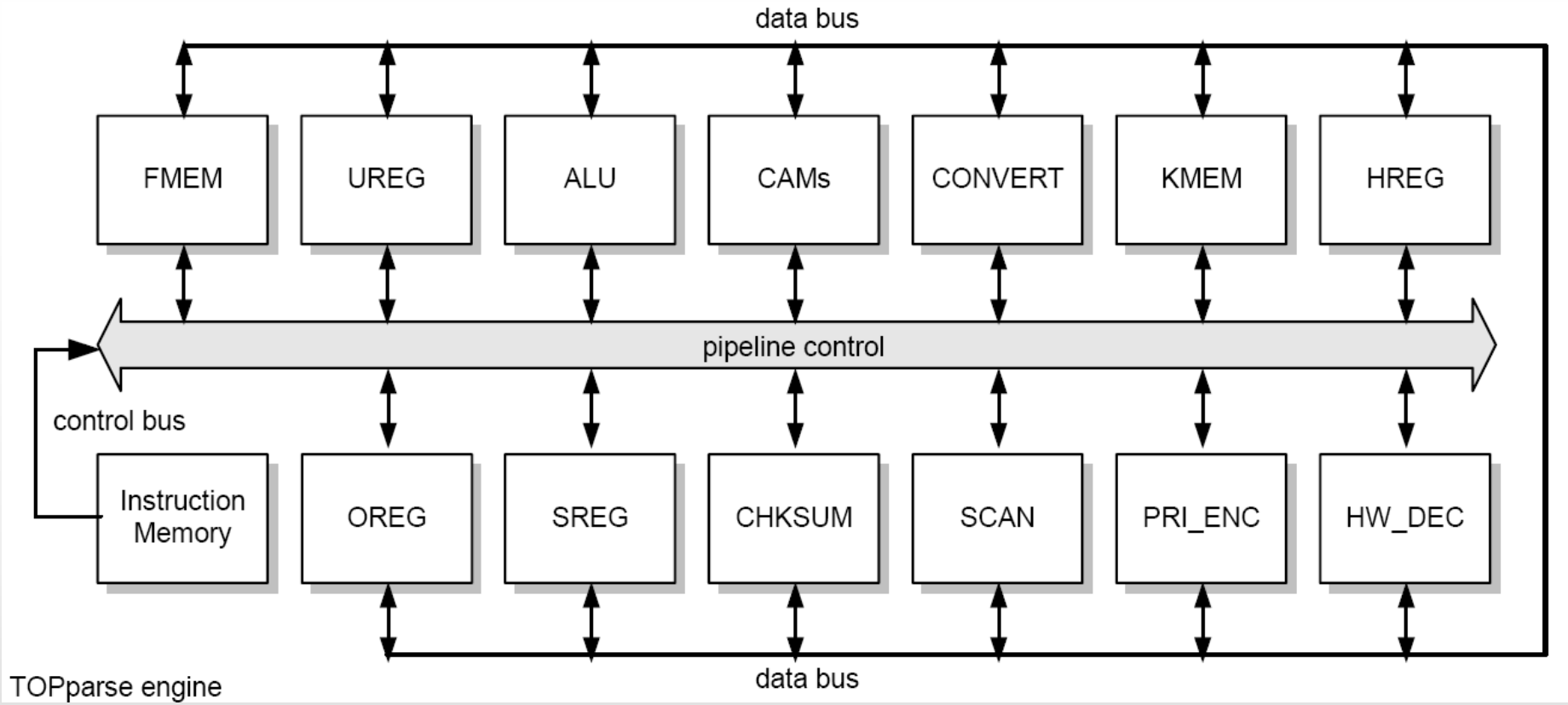
Licznik wejściowy



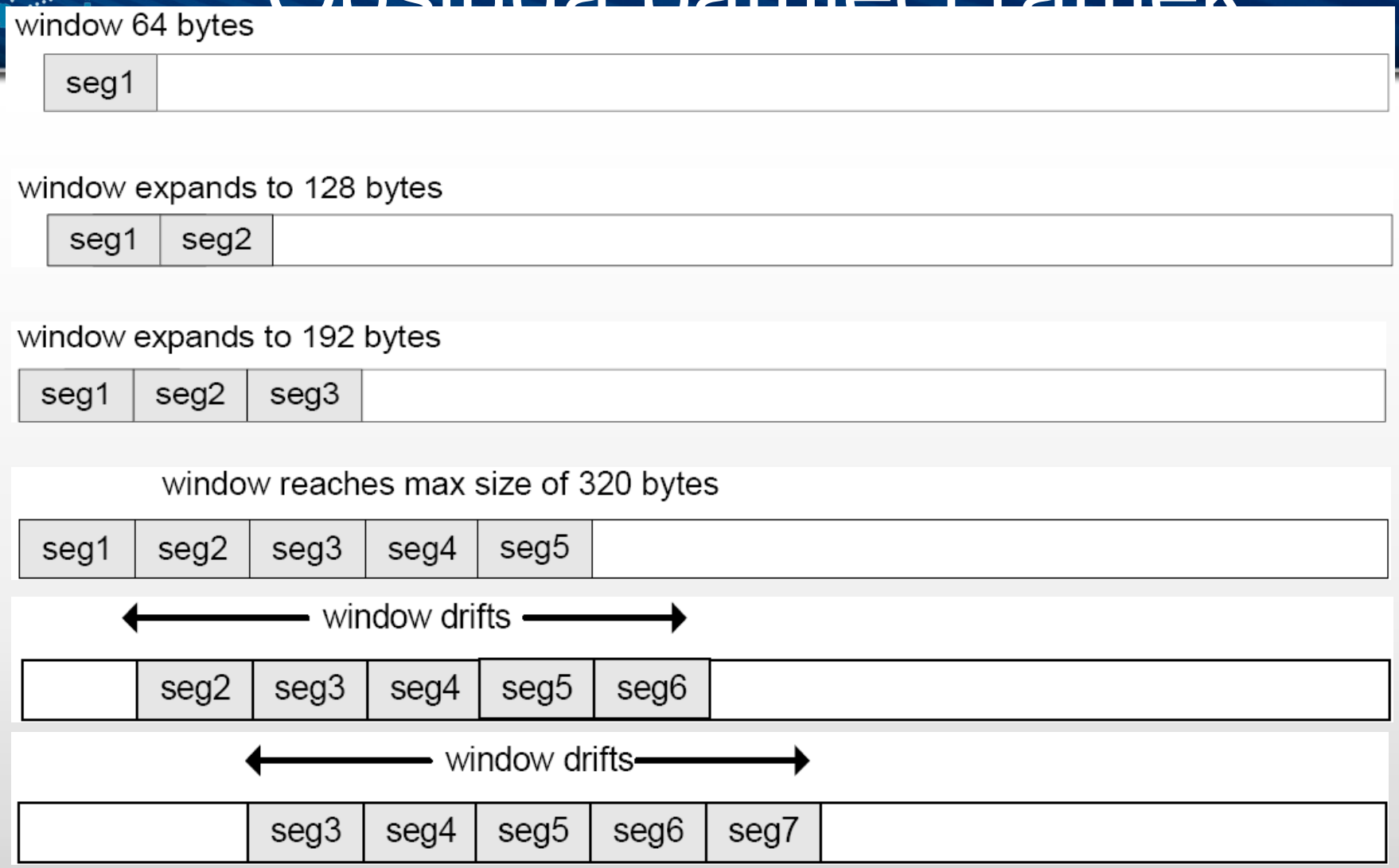
Wybór ramki


R.R with P =00
R.R with P =01
R.R with P =10
R.R with P =11
priority encoder
chosen

Bloki wewnętrzne



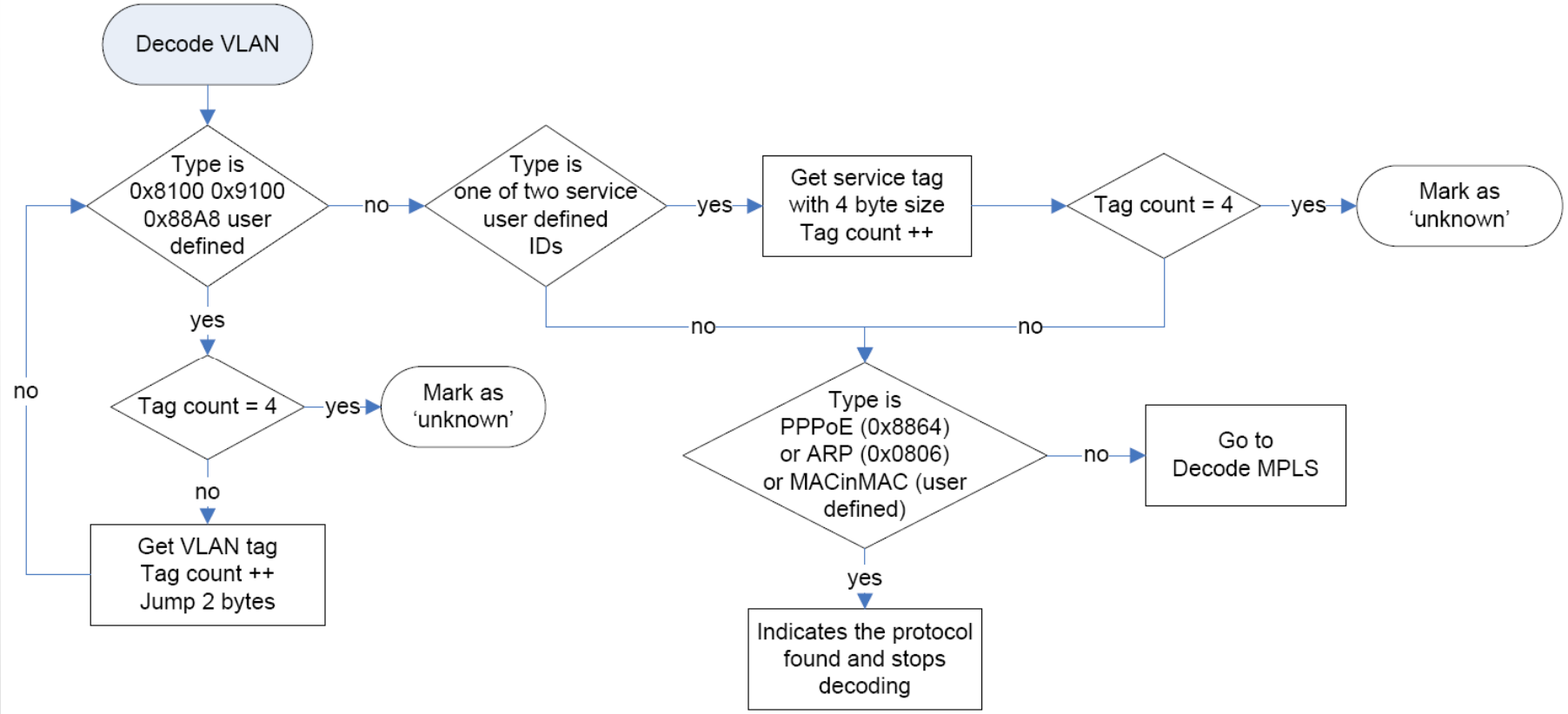
Obsługa pamięci ramek



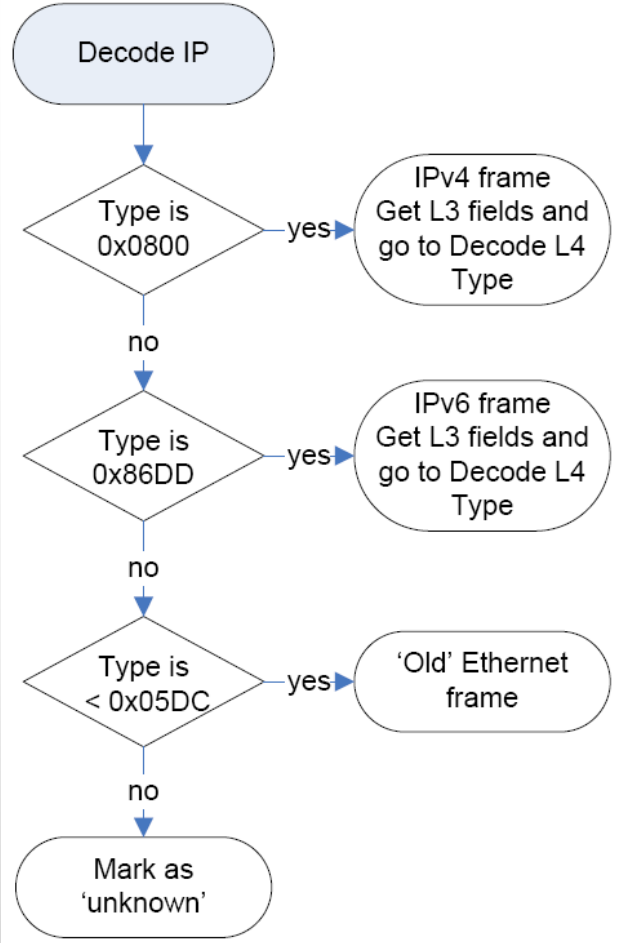
Sprzętowy dekoder protokołów

- Layer 2 protocol type Ethernet/PPP
- Layer 2 header size
- DA is broadcast/multicast/control (BPDU, STP, OAM)
- Layer 3 type IPv4/IPv6/MPLS
- DIP broadcast / multicast / well-known multicast
- Number of MPLS label (up to 4 labels)
- TTL expired
- IPv4 error detection
- Number of VLAN tags (up to 3 tags)
- VLAN TPID can be configured
- VLAN0 (2 bytes)
- VLAN1 (2 bytes)
- VLAN2 (2 bytes or 4 bytes in case of service tag)
- DA match MY MAC
- Ethernet over Ethernet frame
- ARP frame
- PPPoE frame
- TOS if IPv4 or traffic class in IPv6
- Layer 3 header size
- Layer 4 type: TCP/UDP/ICMP/IGMP
- L3 control frame
- Folded xor result (8 bits) for each of the following fields: DA, SA, DIP and SIP

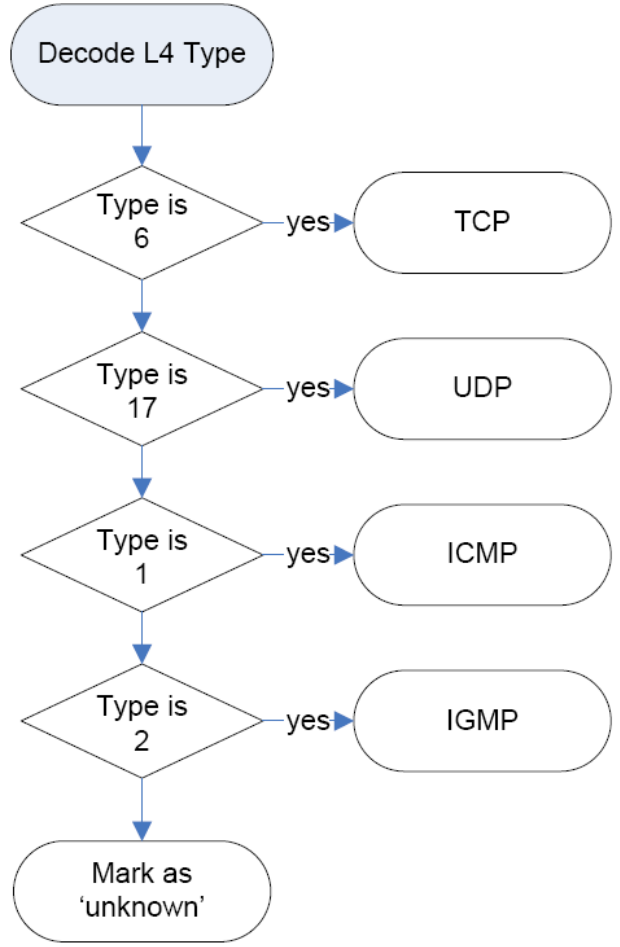
Dekodowanie VLAN



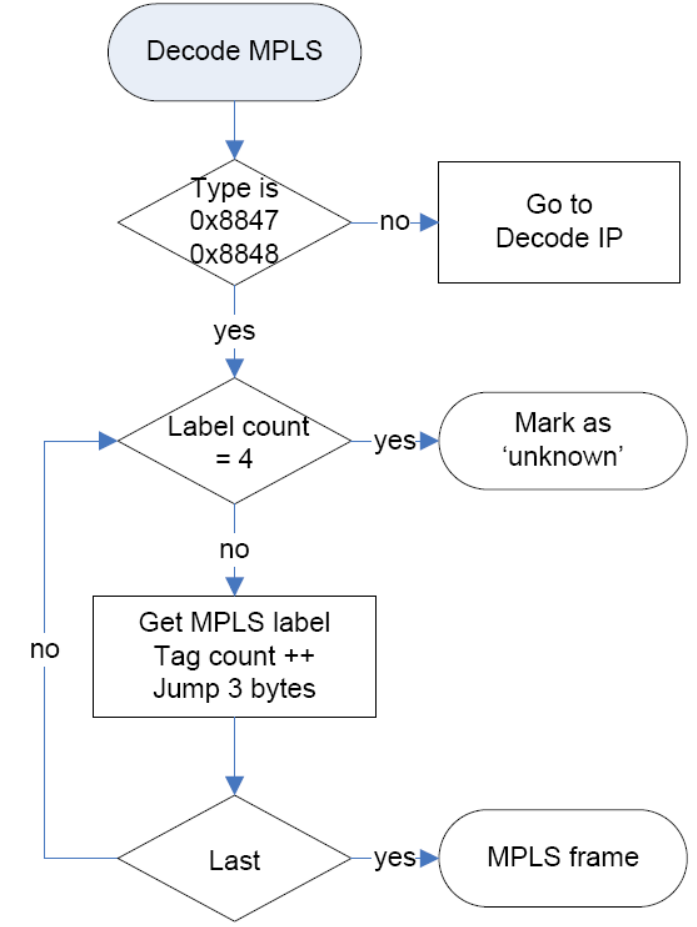
Dekodowanie IP



Dekodowanie protokołu warstwy 4



Dekodowanie MPLS





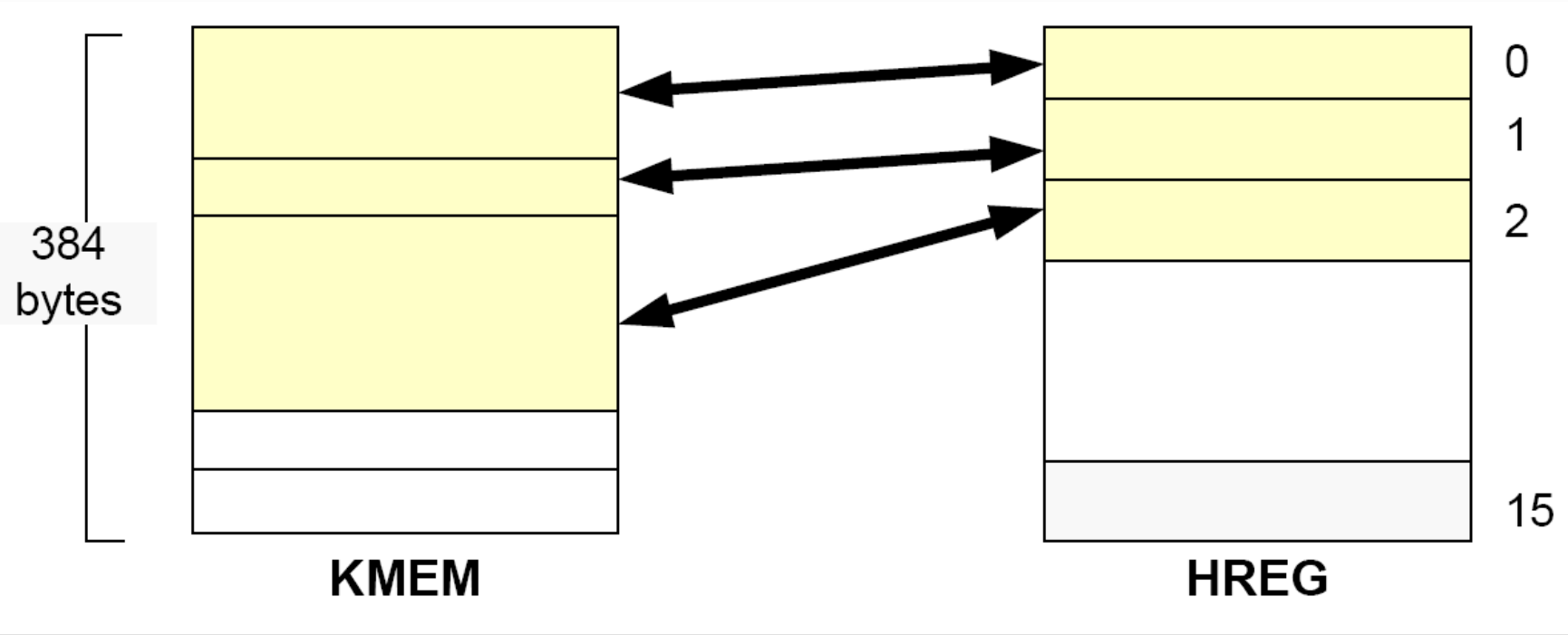
Dekodowanie IPv4

```
copy h_decode_ipv4 0(rd_ptr)+, 8 , 1;    // Read 20B starting at IP header
copy h_decode_ipv4 0(rd_ptr)+, 8;
copy h_decode_ipv4 0(rd_ptr)+, 8;
```

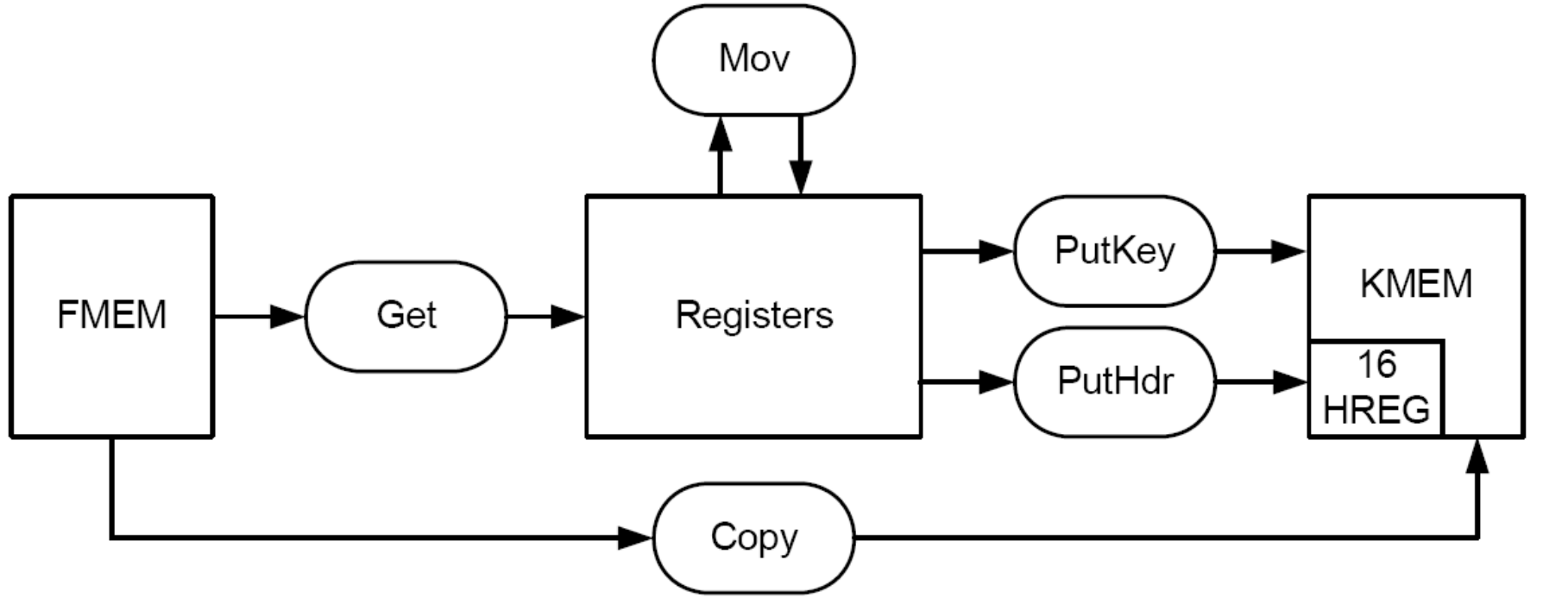
Dekodowanie IPv4 - wyniki

CONTENTS AND DESCRIPTION	*OFFSET (BITS)*	*SIZE (BITS)*
OREG4		
Next protocol	0-7	8
TTL –1	8-15	8
TOS	16-23	8
Header Length	24-29	6
Reserved, set to 0.	30-31	2
OREG5		
SIP+DIP hash result of 16 bits	0-15	16
CAMO		
IPv4 control: (DIP)		
E0-00-00-xx	0	1
E0-00-01-xx	1	1
DMAC match DIP (match IP multicast DIP >= E0-00-00-00 and DMAC multicast 01-00-5e-xx-xx-xx and 24 lsb equal). (DMAC is stored in UREG[11] and UREG[12])	2	1
Control frame is:		
ICMP	3	1
IGMP	4	1
Reserved, set to 0.	5-7	3

Obsługa kluczy



Polecenia przemieszczania danych



Polecenia przemieszczania danych – c.d.

Mov	Move bytes from immediate value or register to register
MovBits	Move bits from immediate value or register to register
Mov4Bits	Move 4 separate bits
MovMul	Multiple move
Get	Move bytes from memory to register
PutKey	Move bytes from register or immediate value to memory
Copy	Move bytes from memory to memory
PutHdr	Write bytes from immediate value or register to header
PutHdrBits	Write bits group from immediate value or register to header
PutHdr4Bits	Write 4 bits with immediate values or from register to header

Polecenia skoku

```
if ( CONDITION )
      command LABELS;

or

command LABELS;
```

Rodzaje poleceń skoków

NAME	DESCRIPTION	CONDITIONS FOR JUMP
Jmp	Jump unconditionally.	always
Jcam	Indirect jump – address jump from CAMO register	MH
Jstack	Indirect jump – address from PC_STACK register	always
Return	Indirect jump – address from PC_STACK register. This is equivalent to Jstack. When a RETURN command is conditional on the CAM match (MH flag), it behaves as JCAM instead of Jstack.	always
Jmul	Multiple jump	PR
Loop	Jump with loop counter decrement	LP
Call	Call and save current address in PC stack.	always
CallCam	JCam and save current address in PC stack	MH
CallStack	JStack and save current address in PC stack	always

Makra skoków

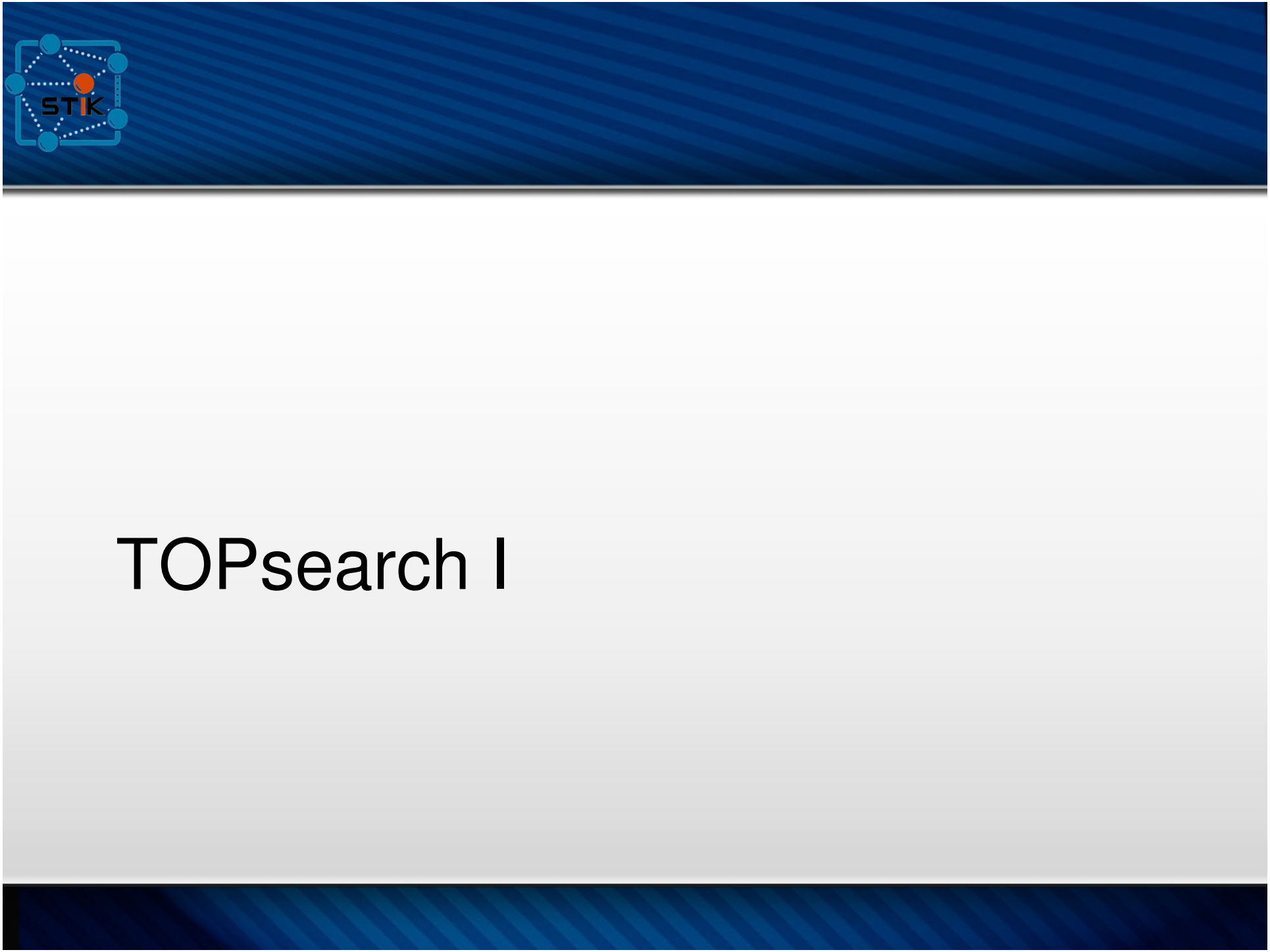
JB	Jump if below (CY)
JC	Jump if carry (CY)
JNAE	Jump if not above or equal (CY)
JAE	Jump if above or equal (!CY)
JNB	Jump if not below (!CY)
JNC	Jump if not carry (!CY)
JE	Jump if equal (ZR)
JZ	Jump if zero (ZR)
JNE	Jump if not equal (!ZR)
JNZ	Jump if not zero (!ZR)
JO	Jump if overflow (OV)
JNO	Jump if not overflow (!OV)
JS	Jump if sign (SN)
JNS	Jump if not sign (!SN)
JA	Jump if above (!CY & !ZR)
JNBE	Jump if not below or equal (!CY & !ZR)



Polecenia ALU

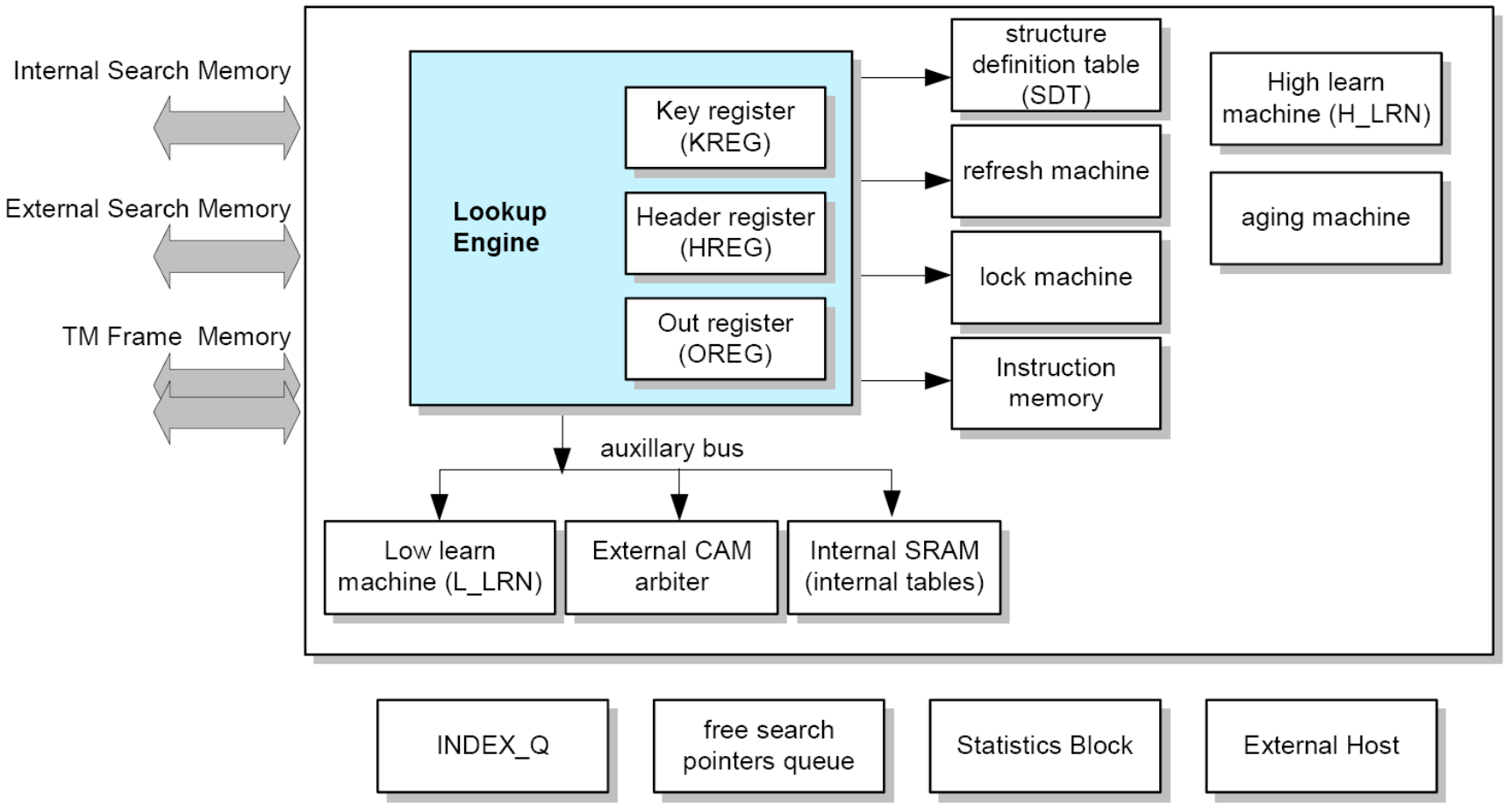
Command DST, SRC1, SRC2, SIZE, [,MREG [, MODE]];

NAME	DESCRIPTION	FLAG DEPENDENCIES
And		OV, ZR, SN
Xor		OV, ZR, SN
Or	(Performed by a macro, hence do not support the mask related parameters (MREG, MODE))	OV, ZR, SN
Not		OV, ZR, SN
Add		OV, ZR, SN
Sub		OV, ZR, SN
Addc	DST = SRC1 + SRC2 + Carry	OV, ZR, CY, SN
Subb	DST = SRC1 - SRC2 - Carry	OV, ZR, CY, SN
Decode	Set bit k in the DST, when k = SRC	ZR
Encode	DST = k when bit k is the most significant set bit in SRC	ZR
NumOnes	DST = k when k is the number of bits = 1 in SRC. 1 byte only.	ZR
Fxor	Folded Xor function with 8 bit result.	OV, ZR, SN
Fxor16	Folded Xor function with 16 bit result. Size operand must be 2 or 4.	OV, ZR, SN

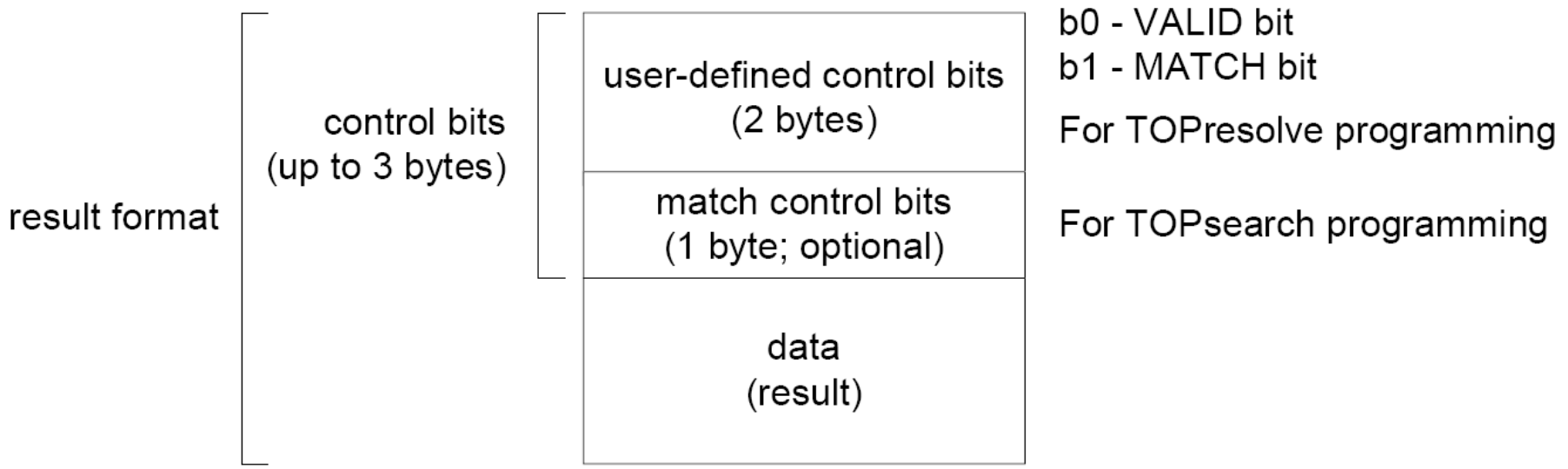


TOPsearch I

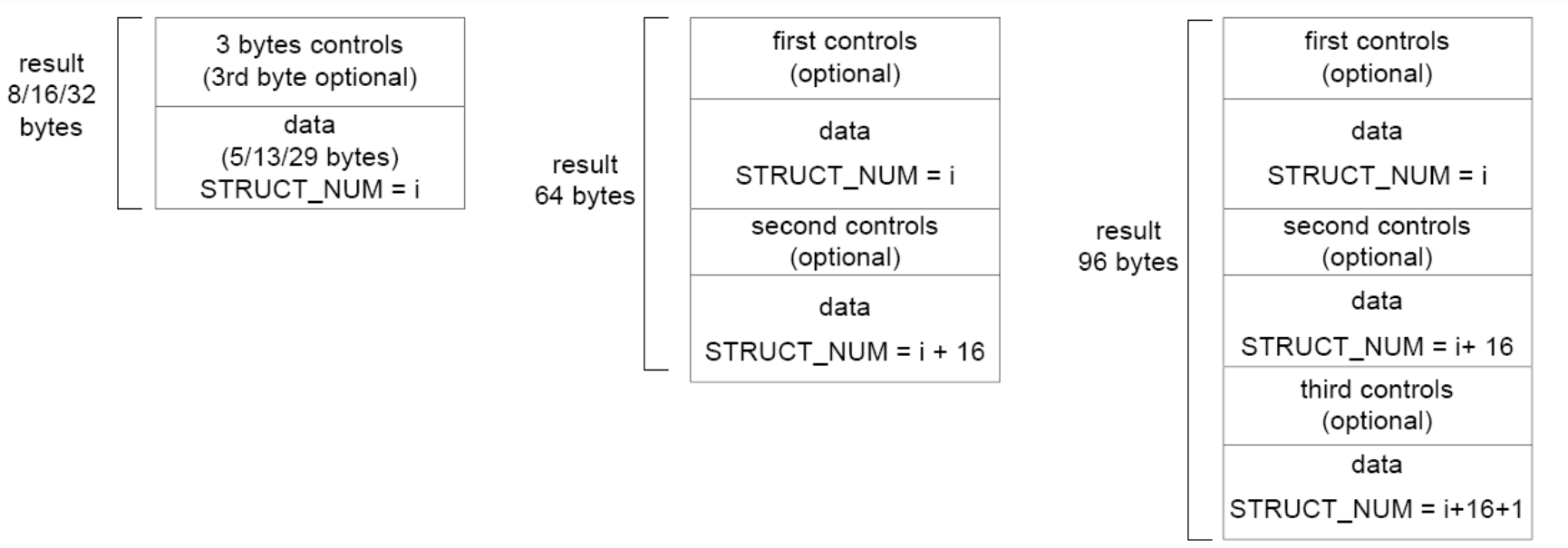
Bloki wewnętrzne



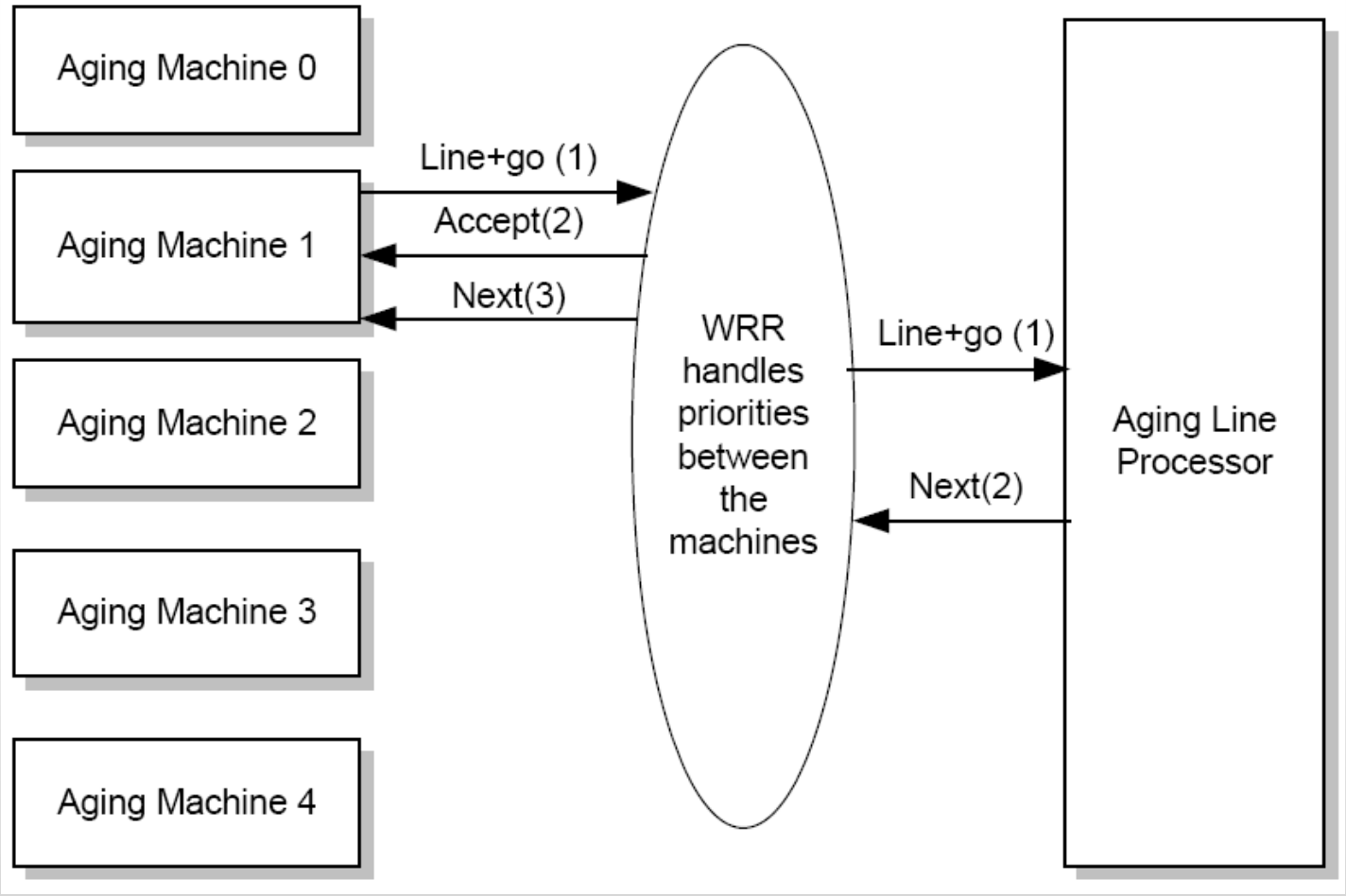
Styk z TOPresolove



Formy rezultatów



Starzenie wpisów



Polecenia

- Polecenia przeszukiwania

```
Lookup TREG, 6, KREG.BYTE[0], 2, KREG, 0;
```

- Polecenia zapisu

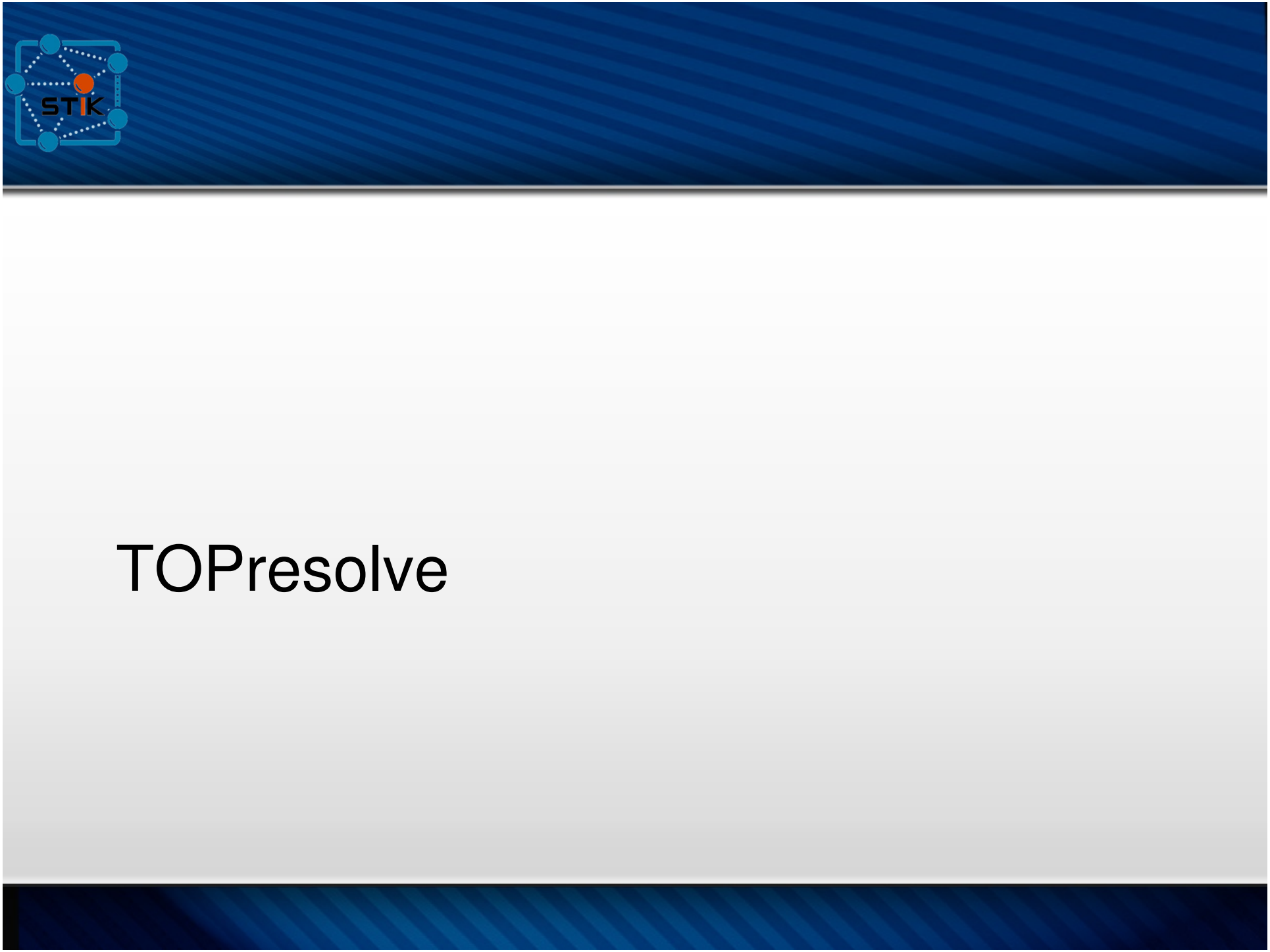
```
Lookup TREG.byte[16];     // First lookup command
Lookup TREG.byte[0] ;     // Second lookup command
...
WriteCond TREG.byte[18], 1, 0x3; // Copies the conditional bits (TREG.byte[18]) from
                                 // the first lookup to be used as the conditional
                                 // bits in the next command. Sets the MATCH bit and
                                 // MATCH_ALL bit to one.
JMul ...;     // Jumps regardless of whether there was a match or not in the last
               // (second) lookup
```



Sterowanie przepływem

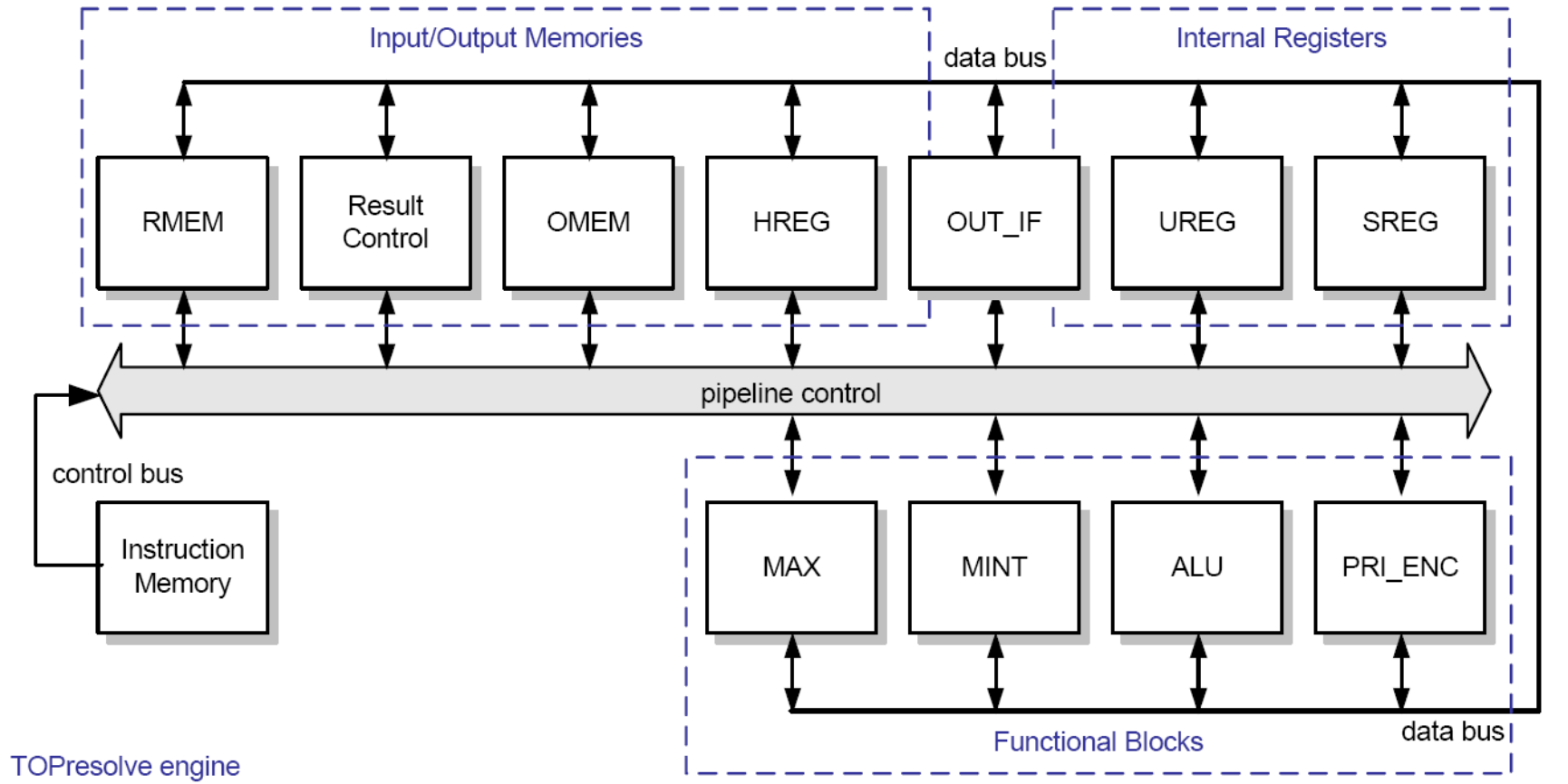
```
    Lookup TOREG, 20, KREG, 8, KREG, 0, 0, 0xF, _WR_L_NJMP, 0, 0;
    Jmatch  GET_COLOR_LBL;
    halt;

GET_COLOR_LBL:
    Lookup OREG, 60, KREG.byte[8], 3, TREG.byte[4], 3, 0, 0xF, _WR_LAST, 0, 0;
    halt;
```

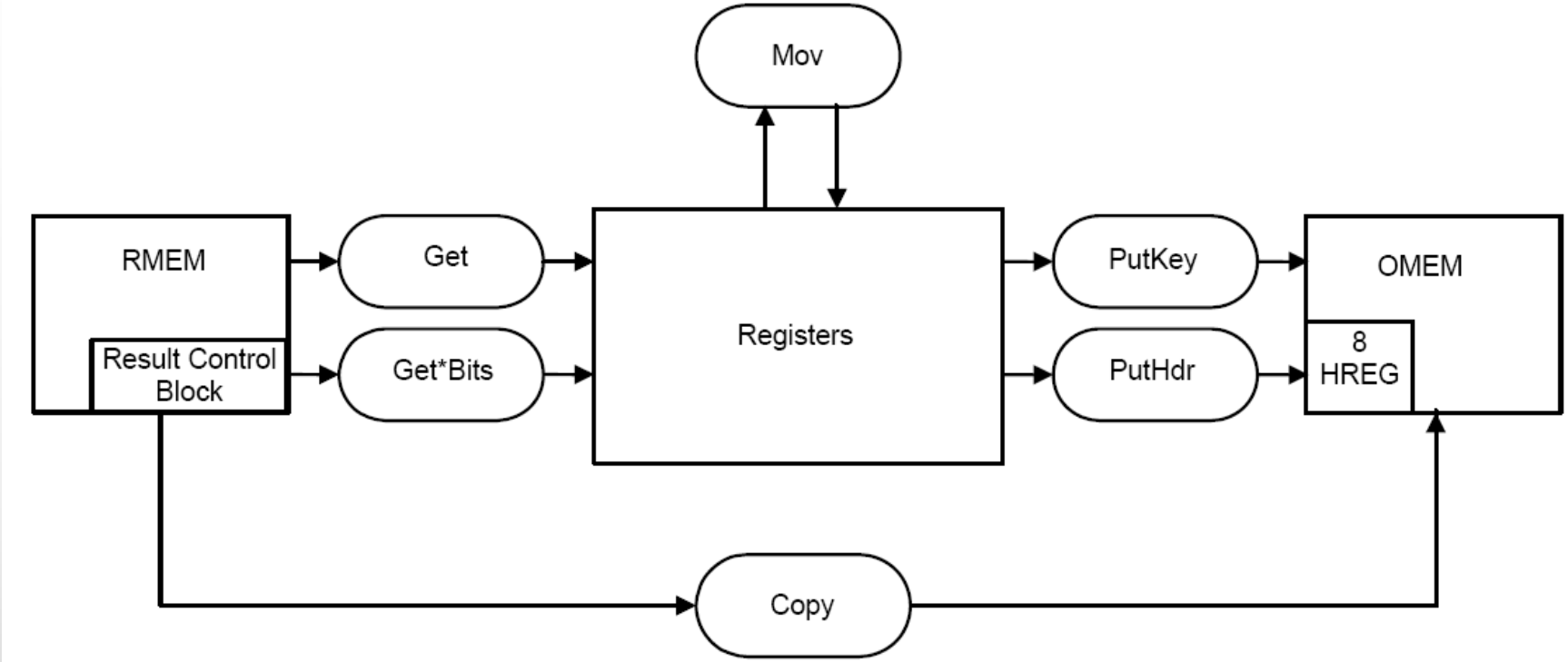


TOPresolve

Bloki wewnętrzne



Polecenia przemieszczania danych



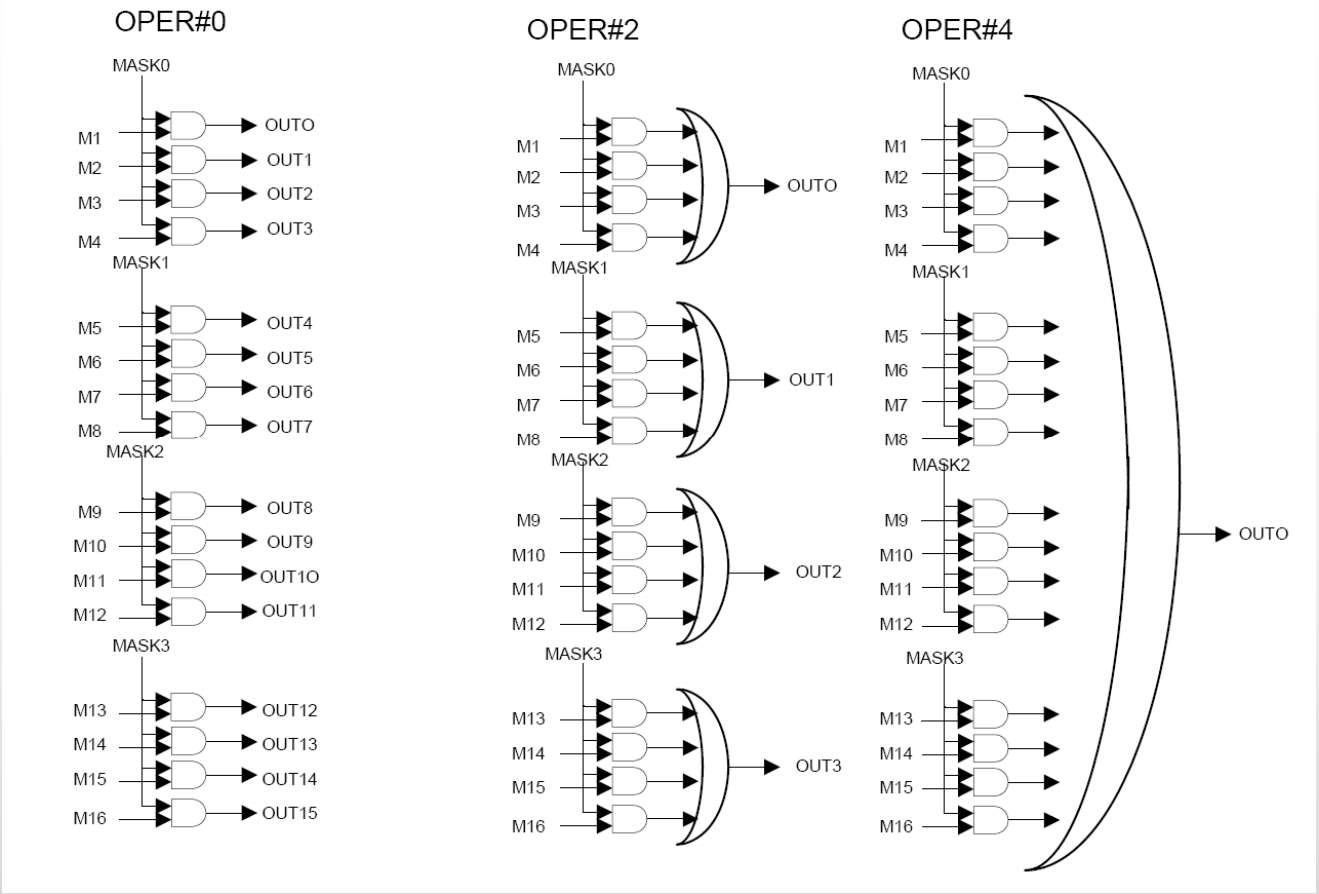


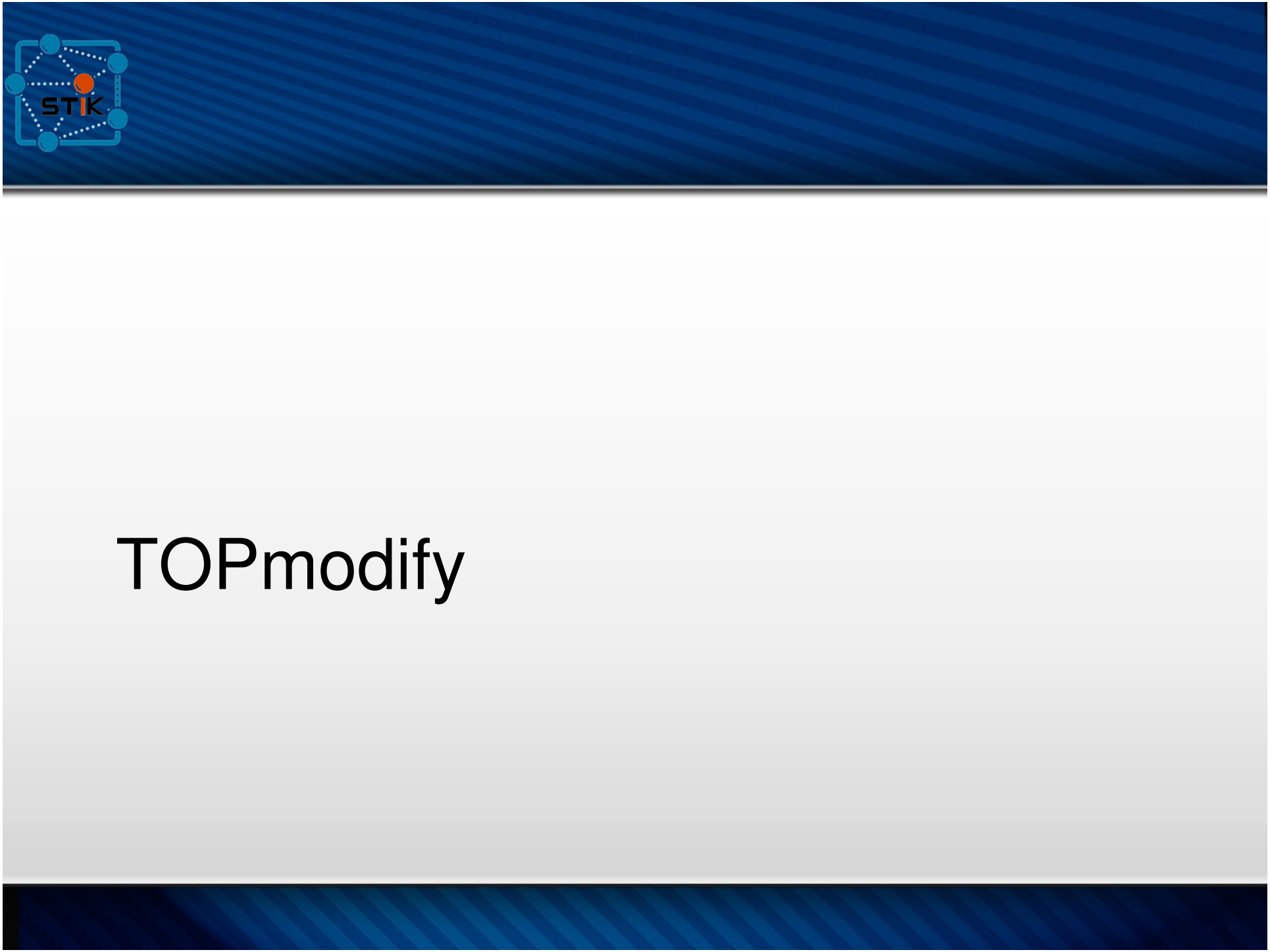
Przemieszczanie danych

Mov	Move bytes from immediate value or register to register
MovBits	Move bits from immediate value or register to register
Mov4Bits	Move 4 separate bits
MovMul	Multiple move
Get	Move bytes from memory to register
PutKey	Move bytes from register or immediate value to memory
Copy	Move bytes from memory to memory
PutHdr	Write bytes from immediate value or register to header
PutHdrBits	Write bits group from immediate value or register to header
PutHdr4Bits	Write 4 bits with immediate values or from register to header
GetRndBits	Move bits from result control
GetFixBits	Move bits from result control
GetRnd4Bits	Move bits from result control

Minitermy

Minterm DST, OPER, INVMASK, MASK4, CTRL, SIZE





TOPmodify

Przemieszczanie danych

